

СОДЕРЖАНИЕ

Введение.....	7
1 Обзор литературы	8
1.1 Программный интерфейс TBDb	8
1.2 Рекомендательная система.....	11
1.3 Платформы машинного обучения	16
1.4 Среда разработки Visual Studio Code	17
1.5 Архитектура приложения.....	18
1.6 Обзор аналогов	19
2 Системное проектирование.....	25
2.1 Блок клиентского приложения	25
2.2 Блок сервера.....	26
2.3 Блок загрузки предпочтений.....	26
2.5 Блок локальной базы данных.....	27
2.6 Блок анализа данных.....	27
2.7 Блок обучения модели	27
2.8 Блок базы данных фильмов.....	28
2.9 Блок подготовки исходных данных	28
3 Функциональное проектирование	29
3.1 Модель анализа данных.....	29
3.2 Модель обучения модели	30
3.3 Модель подготовки исходных данных	35
3.4 Модель реляционной базы данных	38
3.5 Описание таблиц локальной базы данных	42
3.6 Модель клиентского приложения	44
4 Разработка программных модулей.....	48
4.1 Общая архитектура системы.....	48
4.2 Ключевые компоненты	48
4.3 Ключевые алгоритмы	57
5 Программа и методика испытаний.....	64
5.1 Тесты графического интерфейса	64
5.2 Тесты функциональности.....	69
6 Руководство пользователя.....	71
6.1 Системные требования	71
6.2 Доступ к приложению	71
6.3 Краткое руководство пользователя.....	71
7 Технико-экономическое обоснование разработки и реализации на рынке приложения «Сервис рекомендаций фильмов с использованием ИИ»	76
7.1 Характеристика программного средства, разрабатываемого для реализации на рынке.....	76
7.2 Расчёт инвестиций в разработку программного средства	76
7.3 Расчёт экономического эффекта от реализации программного средства на рынке	79
7.4 Расчёт показателей экономической эффективности разработки и	

реализации программного средства на рынке	80
7.5 Вывод об экономической целесообразности реализации проектного решения	81
Заключение	82
Список использованных источников	83
Приложение А	84
Приложение Б	120
Приложение В.....	121

ВВЕДЕНИЕ

Данный дипломный проект посвящён разработке программного средства для рекомендаций фильмов пользователям на основе их предпочтений.

Целью дипломного проектирования является создание сервиса рекомендаций фильмов с использованием технологий искусственного интеллекта. На основе введенных данных пользователя алгоритм будет подбирать и предлагать фильмы, что обеспечит персонализированный подход к рекомендациям, улучшая опыт просмотра и помогая пользователям находить новые интересные фильмы, соответствующие их вкусам.

В соответствии с целью определены следующие задачи:

- изучить необходимую литературу;
- провести анализ аналогов;
- разработать серверную часть программы, отвечающую за обработку данных;
- разработать клиентскую часть программы для взаимодействия с пользователями;
- протестировать программу.

Практическим применением разработки сервиса рекомендаций фильмов является создание удобной платформы для пользователей, позволяющей быстро и эффективно находить новые фильмы на основе их предпочтений. Сервис улучшает пользовательский опыт, предлагая структурированную информацию о фильмах, жанрах и рейтингах, что позволяет пользователям легко ориентироваться в обширном мире кино. Кроме того, интеграция алгоритмов искусственного интеллекта обеспечивает высокую степень точности в рекомендациях, помогая пользователям открывать для себя новые интересные фильмы, которые могут не быть на поверхности.

Разработка приложения для различных типов устройств является одной из главных задач данного дипломного проекта. Это требует создания адаптивного дизайна, который будет корректно отображаться как на настольных компьютерах, так и на мобильных устройствах.

В рамках разработки проекта также будет предусмотрена возможность интеграции с различными API для получения данных о фильмах, что позволит динамически обновлять контент и рекомендации. Кроме того, создание серверной части позволит эффективно обрабатывать запросы пользователей, управлять базами данных и реализовывать алгоритмы машинного обучения для генерации персонализированных рекомендаций.

Данный дипломный проект выполнен мной лично, проверен на заимствования, процент оригинальности составляет 98,6% (отчет о проверке на заимствования прилагается).

1 ОБЗОР ЛИТЕРАТУРЫ

1.1 Программный интерфейс TMDb

The Movie Database (TMDb) – это созданная сообществом база данных фильмов и телепередач. Она содержит описания большинства фильмов и обложки на русском языке, что делает ее удобной для русскоязычных пользователей. TMDb предоставляет обширную информацию о фильмах и сериалах, включая актёрский состав, рейтинги, отзывы и многое другое [1].

Основные функциональные возможности TMDb включают следующее:

1 Информационное наполнение о фильмах и сериалах:

- подробные описания произведений (оригинальное название, локализованное название, жанр, год выпуска, страна производства, продолжительность, статус);
- аннотации и краткие сюжеты;
- пользовательские и профессиональные рейтинги;
- сведения о наградах и фестивалях;
- информация о сериалах, разбитая по сезонам и эпизодам.

2 Данные о персоналиях:

- полные профили актёров, режиссёров, сценаристов, продюсеров и других участников съёмочного процесса;
- биографическая информация, фильмография и фотографии;
- структурированные связи между персоналиями и произведениями.

3 Визуальный контент:

- обложки фильмов и сериалов (в том числе локализованные на русском языке);
- фоновые изображения, постеры, логотипы, кадры из фильмов;
- различные разрешения изображений, подходящие как для пользовательских, так и для коммерческих целей.

4 Пользовательское редактирование контента:

- возможность зарегистрированным пользователям добавлять и редактировать информацию о фильмах, сериалах, эпизодах и участниках съёмочной группы;
- краудсорсинговая модель наполнения данных, обеспечивающая высокую степень актуальности информации;
- модерация всех изменений для обеспечения достоверности.

5 Официальное API для разработчиков:

- доступ к структурированной информации о фильмах, актёрах, обзорах, списках, трендах и т. д.;
- возможности фильтрации, поиска и получения связанных данных;
- поддержка мультиязычности, в том числе русского языка;
- интеграция TMDb API в веб-сайты, мобильные и десктопные приложения, рекомендательные системы и аналитические платформы.

6 Социальные и пользовательские функции:

- создание персонализированных списков фильмов и сериалов (например, «Избранное», «Просмотрено», «Хочу посмотреть»);
- возможность оставлять пользовательские оценки, комментарии и рецензии;
- подписка на обновления и просмотр актуальных новинок кино и телевидения;
- обсуждение контента внутри пользовательского сообщества.

7 Дополнительные особенности:

- регулярное обновление базы за счёт активного сообщества;
- поддержка мультиязычного интерфейса;
- высокая степень доступности и совместимости с различными цифровыми решениями.

API TMDb предоставляет возможность получения данных в формате JavaScript Object Notation (JSON), что облегчает интеграцию с различными приложениями, так как JSON является нативно поддерживаемым форматом для большинства языков программирования, включая Python и JavaScript [2]. Использование JSON позволяет разработчикам легко парсить и обрабатывать данные, не прибегая к сторонним библиотекам.

Далее в данном разделе представлены структуры ответов API TMDb на различные запросы. В каждой структуре указаны названия ключей и типы значений. Если значение является опциональным, это будет отмечено в отдельном столбце таблицы. Если такой столбец отсутствует, все значения являются обязательными. Полями в данном разделе будут называться пары ключа и значения.

Типы данных также могут варьироваться в зависимости от конкретного запроса, и далее будут описаны примеры структур данных для наиболее распространенных запросов, таких как получение информации о фильмах и поисковые запросы.

1.1.1 Информация о фильме.

В ответе за запрос, связанный с информацией о фильме, полученной из базы данных TMDb, содержатся ключевые данные, такие как название, описание, жанры, актеры, дата релиза, компания производитель и рейтинги. Эти данные являются важными для понимания содержания и контекста фильма. Структура ответа на запрос о предоставлении информации о фильме представлена в таблице 1.1. Данная информация помогает зрителям сделать осознанный выбор при просмотре, а также дает возможность глубже оценить художественные и технические аспекты произведения.

Таблица 1.1 – Структура ответа на запрос о получении информации о фильме

Ключ	Тип	Описание
1	2	3
id	Число	Идентификатор
title	Строка	Название

Продолжение таблицы 1.1

1	2	3
original_language	Строка	Язык
genres	Объект	Жанры
release_data	Дата	Дата выпуска
production_company	Объект	Компания производитель
runtime	Число	Продолжительность
poster_path	Строка	Ссылка на обложку фильма
budget	Число	Бюджет фильма
overview	Строка	Описание фильма
homepage	Строка	Домашняя страница
popularity	Число	Популярность фильма
status	Строка	Статус фильма
vote_average	Число	Рейтинг фильма

Пример JSON-ответа на запрос о информации о фильме представлен в таблице 1.2.

Таблица 1.2 – Структура ответа с информацией о фильме

Поле	Значение
title	Inception
genres	[{ "id": 28, "name": "Action" }]
release_data	2010-07-16
runtime	148
budget	160000000
vote_average	8.3
poster_path	/9gk7adHYeDvHkCSEqAvQNLV5Uge.jpg
production_company	[{ "name": "Legendary Pictures", "country": "US" }]

1.1.2 Поиск фильма.

Поисковый функционал TMDb позволяет находить фильмы по ключевым словам, названиям или их фрагментам. В ответ на такой запрос API возвращает пагинированный список результатов, содержащий основные сведения о каждом найденном фильме. Структура ответа за запрос поиска фильма представлена в таблице 1.2.

Таблица 1.3 – Структура ответа на поиск фильма

Ключ	Тип	Описание
page	Число	Номер страницы
results	Объект	Информация о фильме
total_page	Число	Количество страниц
total_results	Число	Количество результатов поиска

Каждый элемент массива `results` обычно содержит поля: `id`, `title`, `release_date`, `overview`, `poster_path`, `vote_average`. Пример одного результата из массива `results` представлен в таблице 1.4.

Таблица 1.4 – Структура ответа с информацией о фильме

Поле	Значение
<code>title</code>	Inception
<code>id</code>	27205
<code>release_data</code>	2010-07-16
<code>vote_average</code>	8.3
<code>poster_path</code>	/9gk7adHYeDvHkCSEqAvQNLV5Uge.jpg
<code>overview</code>	"A thief who steals corporate secrets through dream-sharing..."

1.2 Рекомендательная система

Рекомендательные системы играют ключевую роль в современных цифровых сервисах, таких как стриминговые платформы, интернет-магазины, новостные сайты и социальные сети. Их основная цель – повысить вовлеченность пользователей путём персонализированного предложения контента, который наиболее вероятно заинтересует конкретного пользователя. Для достижения этой цели системы активно используют алгоритмы машинного обучения и обработки больших данных (Big Data).

Такие алгоритмы анализируют разнообразную информацию, включая:

- историю взаимодействия пользователя с сервисом (просмотры фильмов, прослушивания музыки, клики, лайки и т. д.);
- временные характеристики поведения (длительность просмотра, время отклика, частота использования);
- демографические данные (при наличии);
- отзывы и оценки контента.

На основе собранных данных система прогнозирует, какой именно контент (например, фильм, сериал, трек, статья, товар) будет наиболее релевантен конкретному пользователю, и формирует персонализированные рекомендации. В зависимости от задач, доступных данных и особенностей платформы, могут использоваться различные подходы:

- коллаборативная фильтрация;
- матричная факторизация;
- контентная фильтрация;
- гибридная система.

1.2.1 Коллаборативная фильтрация.

Коллаборативная фильтрация (Collaborative Filtering) [3] является одним из наиболее популярных и широко применяемых подходов в построении

рекомендательных систем. Её ключевая идея заключается в том, что предпочтения пользователей можно предсказывать, исходя из сходства между ними или между объектами, с которыми они взаимодействуют. В отличие от контентных методов, коллаборативная фильтрация не требует анализа характеристик самих объектов (например, жанра фильма или описания товара), а использует поведенческие данные, такие как рейтинги, просмотры, покупки или лайки.

Метод коллаборативной фильтрации делится на два основных направления: user-based подход (основанный на пользователях) и item-based подход (основанный на объектах).

В рамках user-based метода система анализирует поведение одного пользователя и сравнивает его с поведением других пользователей для выявления так называемых «соседей» – пользователей с похожими вкусами. Система рассчитывает метрику схожести (например, косинусное расстояние или коэффициент корреляции Пирсона) между пользователями на основе их взаимодействия с контентом (рейтинги, просмотры и т. д.). Затем контент, который был положительно оценён похожими пользователями, но ещё не просмотрен данным пользователем, предлагается в качестве рекомендаций.

Пример: если пользователь А и пользователь В смотрели и положительно оценили одни и те же фильмы, а затем пользователь А посмотрел новый фильм Х, который пользователь В ещё не видел, система может порекомендовать фильм Х пользователю В. Таким образом, рекомендации строятся на основе «мнений» похожих пользователей.

Преимущества:

- простота реализации;
- хорошие результаты при наличии большого числа пользователей и взаимодействий.

Недостатки:

- проблема масштабируемости, при которой при большом количестве пользователей вычисление схожести может быть ресурсоемким;
- проблема холодного старта, где эффективность снижается при недостатке данных о новом пользователе.

Item-based метод строит рекомендации, ориентируясь не на пользователей, а на товары (или объекты, такие как фильмы, книги, товары и т. п.). Он анализирует схожесть между объектами на основе оценок и взаимодействий с ними со стороны пользователей. Если пользователю понравился определённый объект, система предложит другие объекты, которые получили схожие отклики от тех же пользователей.

Пример: если пользователь оценил фильм А, а большинство пользователей, которые также оценили фильм А, положительно оценили фильм В, то система может порекомендовать фильму В этому пользователю. Это особенно удобно, если у пользователя мало взаимодействий, но много данных по объектам.

Преимущества:

- лучше масштабируется в условиях большого количества

пользователей;

- устойчив к проблеме «холодного старта» при появлении новых пользователей (если объекты уже оценены другими).

Недостатки:

- меньшая персонализация по сравнению с user-based подходом;
- требует хорошей заполненности матрицы взаимодействий.

1.2.2 Матричная факторизация.

Матричная факторизация является одним из наиболее мощных и распространённых подходов в построении рекомендательных систем, особенно при работе с задачами предсказания предпочтений пользователей [4]. Данный метод получил широкое распространение благодаря своей высокой эффективности при работе с большими и разреженными наборами данных, характерными для большинства онлайн-сервисов, включая стриминговые платформы и интернет-магазины.

Суть метода заключается в том, чтобы представить поведенческие данные в виде прямоугольной матрицы, где строки соответствуют пользователям, а столбцы – объектам (например, фильмам, книгам или товарам). Элементы этой матрицы отражают уровень взаимодействия пользователя с конкретным объектом: это могут быть оценки, лайки, просмотры или покупки. Однако на практике большая часть элементов матрицы остаётся пустой, так как каждый пользователь взаимодействует лишь с небольшой частью всего доступного контента.

Матричная факторизация решает задачу заполнения этих пропущенных значений путём разложения исходной матрицы на две матрицы меньших размерностей:

- матрицу пользовательских признаков, где каждая строка описывает предпочтения конкретного пользователя в скрытом (латентном) пространстве признаков;

- матрицу объектных признаков, где каждая строка представляет свойства объекта в том же скрытом пространстве.

Затем путём перемножения этих двух матриц можно получить аппроксимацию исходной матрицы с заполненными пропущенными значениями, то есть предсказанными рейтингами, которые можно использовать для формирования персонализированных рекомендаций.

Одним из наиболее известных и часто используемых алгоритмов матричной факторизации является SVD (Singular Value Decomposition, сингулярное разложение), которое разлагает матрицу на произведение трёх матриц: матрицы левых сингулярных векторов (пользователи), диагональной матрицы сингулярных значений и матрицы правых сингулярных векторов (объекты). В рекомендательных системах часто используется упрощённая или модифицированная версия SVD, адаптированная под задачи машинного обучения и обработки разреженных данных, например, Funk-SVD или SVD++.

Преимущества метода матричной факторизации:

- высокая точность предсказаний, особенно в условиях большого числа

пользователей и объектов;

- способность находить скрытые зависимости и предпочтения, которые невозможно выявить с помощью простых эвристик;

- эффективность в условиях разреженных данных, что критично для реальных рекомендательных систем.

Недостатки:

- неэффективность при проблеме холодного старта – когда необходимо рекомендовать новые объекты или обслуживать новых пользователей без предварительных данных;

- потребность в предварительной обработке данных и выборе гиперпараметров;

- возможность переобучения, особенно при недостаточном объеме обучающей выборки.

Матричная факторизация лежит в основе многих успешных коммерческих рекомендательных решений. Так, платформа Netflix использовала модифицированные версии SVD в рамках своего знаменитого Netflix Prize, где главной задачей было улучшение точности предсказания оценок фильмов. Amazon, Spotify и другие ведущие IT-компании также активно применяют различные формы факторизации в своих рекомендательных системах.

1.2.3 Контентная фильтрация.

Контентная фильтрация – это другой важный метод, используемый в рекомендательных системах [5]. В отличие от коллаборативной фильтрации, которая основывается на поведении пользователей, контентная фильтрация анализирует характеристики контента и сопоставляет их с предпочтениями пользователя.

Контентная фильтрация использует информацию о свойствах товаров или контента для создания рекомендаций. Например, в случае с книгами это могут быть жанр, автор, язык и т. д. Система создает профиль пользователя на основе его предпочтений и затем сравнивает его с характеристиками доступного контента, чтобы предложить наиболее подходящие варианты. Контентная фильтрация часто используется в системах, где характеристики товаров играют ключевую роль в предпочтениях пользователей. Например, в музыкальных сервисах, таких как Spotify, используются данные о жанрах, исполнителях и альбомах для создания рекомендаций. Если пользователь часто слушает рок-музыку, система будет рекомендовать ему новые рок-альбомы и исполнителей. В случае с фильмами и сериалами, такими как Netflix, система анализирует жанры, режиссеров, актеров и другие метаданные, чтобы предложить пользователю контент, который соответствует его вкусу. Например, если пользователь часто смотрит научно-фантастические фильмы, система порекомендует ему новые фильмы и сериалы в этом жанре.

Контентная фильтрация имеет свои преимущества и недостатки. Одним из главных преимуществ является то, что она не требует данных о других пользователях и может создавать рекомендации даже для новых

пользователей (проблема "холодного старта"). Однако у нее есть и свои ограничения. Например, система может не учитывать, что пользователь может быть заинтересован в контенте, который отличается от его предыдущих предпочтений. Это ограничивает разнообразие рекомендаций и может привести к "эффекту пузыря", когда пользователь получает только однотипные рекомендации.

1.2.4 Гибридная система.

Для достижения наилучших результатов в предоставлении персонализированных рекомендаций всё чаще используются гибридные рекомендательные системы. Эти системы представляют собой комбинацию различных подходов, прежде всего коллаборативной фильтрации и контентной фильтрации, что позволяет одновременно учитывать индивидуальные предпочтения пользователей и характеристики самих объектов рекомендаций. Гибридный подход позволяет преодолеть множество ограничений, присущих каждому из методов по отдельности, и обеспечивает более высокую точность, релевантность и разнообразие рекомендаций.

Основные компоненты гибридного подхода:

- коллаборативная фильтрация;
- контентная фильтрация;

Преимущества гибридного подхода:

- устранение эффекта холодного старта: контентная фильтрация позволяет рекомендовать новые объекты, о которых ещё нет пользовательской информации;
- борьбу с проблемой разреженности данных: комбинирование методов повышает полноту данных, доступных системе;
- улучшение персонализации (система учитывает как поведенческий контекст, так и тематическое содержание, что повышает индивидуальную релевантность рекомендаций);
- снижение влияния "пузырей фильтра" (рекомендации становятся более разнообразными, что способствует открытию новых интересов);
- способы интеграции различных методов.

Гибридные системы могут быть реализованы с использованием нескольких стратегий объединения методов:

1 Последовательное применение методов (каскадный подход). В этом сценарии один метод используется в качестве предварительного фильтра. Например, сначала система с помощью коллаборативной фильтрации формирует первичный список рекомендаций, содержащий объекты, интересные с точки зрения поведения схожих пользователей. Затем к этим объектам применяется контентная фильтрация, которая оценивает соответствие этих объектов индивидуальному профилю пользователя и формирует финальный отсортированный список. Такой подход позволяет улучшить точность за счёт учета дополнительных факторов.

2 Параллельное применение методов (взвешенное объединение). При этом подходе система независимо генерирует два (или более) списка

рекомендаций: один на основе контента, другой — на основе пользовательских оценок. Затем каждый элемент списка получает взвешенную оценку в зависимости от важности используемых методов. Весовые коэффициенты могут быть фиксированными, адаптивными (обучающимися), или зависеть от контекста (например, от количества уже доступной информации о пользователе или объекте). Этот метод позволяет максимально использовать сильные стороны каждого подхода.

3 Уровень мета-обучения (model-based hybridization). В этом случае рекомендации формируются с помощью машинного обучения, которое принимает на вход признаки как от коллаборативной, так и от контентной модели. Такой подход особенно эффективен в сложных сценариях, где необходимо учитывать большое количество факторов, включая временные зависимости, контекст взаимодействия, популярность объектов и даже психологические характеристики пользователей.

Наиболее успешные рекомендательные платформы, такие как Netflix, Spotify, YouTube и Amazon, активно используют гибридные подходы. Например, Netflix применяет гибридную систему, которая сочетает предпочтения пользователей с описательными признаками фильмов, чтобы максимально точно предсказать, что пользователь захочет посмотреть в следующий раз. Это позволяет им не только повысить вовлеченность, но и сократить отток пользователей.

1.3 Платформы машинного обучения

TensorFlow – это открытая программная библиотека для машинного обучения, разработанная компанией Google для решения задач построения и тренировки нейронной сети с целью автоматического нахождения и классификации образов, достигая качества человеческого восприятия. Библиотека предлагает несколько уровней абстракции, большую гибкость, быстрое выполнение обеспечивает немедленную итерацию и интуитивно понятную отладку. Для больших задач обучения ML можно использовать API стратегии распределения для распределенного обучения на различных конфигурациях оборудования без изменения определения модели.

PyTorch – это библиотека глубокого обучения, созданная на основе Python и Torch (фреймворк на основе Lua). Он обеспечивает ускорение графического процессора, динамические вычислительные графики и интуитивно понятный интерфейс для исследователей и разработчиков глубокого обучения. PyTorch использует подход «определения по выполнению», что означает, что его вычислительные графы создаются на лету, что позволяет лучше отлаживать и настраивать модель.

Scikit-learn – один из наиболее широко используемых пакетов Python для Data Science и Machine Learning. Он позволяет выполнять множество операций и предоставляет множество алгоритмов. Scikit-learn также предлагает отличную документацию о своих классах, методах и функциях, а также описание используемых алгоритмов.

Keras это библиотека, предоставляющая интерфейс Python для искусственных нейронных сетей. Keras следует рекомендациям по снижению когнитивной нагрузки: он предлагает согласованные и простые API, минимизирует количество действий пользователя, необходимых для обычных случаев использования, и предоставляет четкие и действенные сообщения об ошибках. Keras также уделяет первостепенное внимание созданию качественной документации и руководств для разработчиков.

CNTK это стандартизированный инструментарий для проектирования и развития сетей разнообразных видов, применяет искусственный интеллект для работы с большими объёмами данных путем глубокого обучения, использует внутреннюю память для обработки последовательностей произвольной длины. CNTK развивает неуклонное расширение, скорость и точность с качеством коммерческого уровня применения. Имеет выраженную и простую архитектуру, совместимую с популярными языками и сетями.

1.4 Среда разработки Visual Studio Code

Visual Studio Code (VS Code) – это современный, кроссплатформенный, открытый редактор исходного кода, разработанный корпорацией Microsoft. Он был впервые представлен в 2015 году и с тех пор получил широкое распространение среди разработчиков программного обеспечения благодаря своей легкости, расширяемости и поддержке большого числа языков программирования. Несмотря на то, что VS Code не является полноценной интегрированной средой разработки (IDE) в традиционном смысле, по своей функциональности и возможностям он приближается к полноценным IDE и часто используется в профессиональной среде для создания программ различной сложности.

Одна из главных особенностей VS Code – это расширяемая архитектура. Пользователи могут устанавливать тысячи расширений через встроенный маркетплейс, тем самым добавляя поддержку новых языков, фреймворков, инструментов отладки, тем оформления, сниппетов и многого другого. Среди наиболее популярных расширений – поддержка Python, C++, Java, JavaScript, Node.js, Docker, Git и GitHub, а также интеграция с системами управления базами данных и облачными сервисами.

Встроенная поддержка системы контроля версий Git делает VS Code удобным инструментом для командной разработки. Прямо из редактора можно выполнять основные операции с репозиториями: коммиты, создание веток, разрешение конфликтов, просмотр истории изменений. Также редактор предлагает гибкие средства настройки, включая поддержку пользовательских сниппетов кода, настройку горячих клавиш и конфигурацию задач сборки.

VS Code активно используется в образовательных и исследовательских проектах благодаря своей простоте и доступности. Он поддерживает разработку как веб-приложений (HTML, CSS, JavaScript, React, Vue и др.), так и серверной логики (Python, Node.js, PHP и пр.). Поддержка языка разметки Markdown делает его удобным инструментом для ведения технической

документации и конспектов. Для работы с Python, например, доступны средства отладки, запуск Jupyter-ноутбуков и интеграция с виртуальными средами.

Кроме того, VS Code поддерживает удалённую разработку – через SSH, контейнеры или WSL (Windows Subsystem for Linux). Это позволяет разработчику работать с проектами, находящимися на удалённом сервере, как если бы они были локальными, что особенно важно при создании распределённых систем или сервисов с высокими требованиями к вычислительным ресурсам.

Таким образом, Visual Studio Code представляет собой гибкий и мощный инструмент, активно применяемый как в промышленной разработке, так и в научно-исследовательской и учебной практике. Благодаря открытому исходному коду, широкой поддержке сообщества и непрерывному развитию, редактор остаётся одним из самых популярных средств разработки на сегодняшний день.

1.5 Архитектура приложения

Архитектура приложения – это структурное определение его компонентов и их взаимодействий, которое служит основой для разработки, поддержки и масштабирования программного обеспечения. Правильно спроектированная архитектура обеспечивает гибкость, устойчивость и возможность интеграции новых функций.

Существует несколько типов архитектур приложений, каждая из которых имеет свои особенности и подходит для различных сценариев использования. К основным типам относятся:

- монолитная архитектура;
- микросервисная архитектура;
- клиент-серверная архитектура;
- сервисная архитектура;
- архитектура на основе событий;
- архитектура с использованием очередей сообщений.

Каждый тип архитектур имеет свои сильные и слабые стороны, и выбор подходящей архитектуры зависит от требований проекта, его масштабируемости, сложности и других факторов. Среди существующих архитектур, клиент-серверная архитектура выделяется своим четким разделением задач и возможностью использования разных технологий на клиенте и сервере, что делает её особенно подходящей для поставленных в проекте задач.

1.5.1 Клиент-серверная архитектура.

Клиент-серверная архитектура позволяет четко разделить обязанности между клиентской и серверной частями. Это упрощает процесс разработки, так как команды могут сосредоточиться на своих областях, используя различные технологии и инструменты. Клиент, отвечающий за

взаимодействие с пользователем, может быть разработан с учетом удобства и отзывчивости, в то время как серверная часть будет сосредоточена на обработке бизнес-логики и управлении данными.

Кроме того, данный подход обеспечивает гибкость в масштабировании. По мере роста нагрузки на приложение есть возможность независимо масштабировать серверную часть, добавляя новые ресурсы, не затрагивая клиентскую логику. Это особенно важно в условиях динамично меняющихся требований и увеличивающегося числа пользователей.

Также клиент-серверная архитектура предоставляет возможности для улучшения безопасности. Сервер может централизованно управлять аутентификацией и авторизацией пользователей, что минимизирует риски, связанные с утечкой данных.

Таким образом, учитывая все вышеперечисленные преимущества, клиент-серверная архитектура является оптимальным выбором для данного проекта. Она обеспечит необходимую гибкость, производительность и безопасность, что позволит успешно реализовать задуманное и адаптироваться к будущим изменениям.

1.6 Обзор аналогов

Существует множество онлайн-сервисов и платформ, использующих рекомендательные системы для подбора фильмов, каждая из которых обладает своими уникальными особенностями, алгоритмами и подходами к взаимодействию с пользователем. Такие платформы анализируют предпочтения зрителей, историю их просмотров, оценки и другие поведенческие факторы, чтобы предоставить максимально персонализированные рекомендации. На сегодняшний день наибольшую популярность среди пользователей получили следующие рекомендательные сервисы:

- Netflix – международная стриминговая платформа, широко известная своими интеллектуальными алгоритмами рекомендаций и ориентацией на удержание зрителя;
- Кинопоиск – российский сервис с обширной базой данных и развитой системой аналитики пользовательских предпочтений;
- MovieTop – более узкоспециализированный сервис, ориентированный на быструю настройку рекомендаций на основе пользовательских тестов и интересов.

1.6.1 Netflix.

Netflix – это один из крупнейших в мире стриминговых онлайн-сервисов, предоставляющий пользователям возможность смотреть огромный спектр аудиовизуального контента: от художественных и документальных фильмов до анимации, сериалов и интерактивных проектов [6]. Доступ к просмотру осуществляется через подключённые к интернету устройства, включая телевизоры, смартфоны, планшеты, ноутбуки и игровые консоли, что

делает платформу универсальной и удобной для широкой аудитории по всему миру.

Одним из важнейших конкурентных преимуществ Netflix является его высокоразвитая и постоянно совершенствующаяся система персонализированных рекомендаций. Главная цель этой системы – помочь каждому пользователю найти интересный и релевантный контент среди тысяч предложений, доступных на платформе. В условиях перенасыщенности медиaproстранства подобные интеллектуальные алгоритмы становятся особенно актуальными, так как они существенно сокращают время поиска нужного контента и увеличивают вовлечённость зрителя.

Работа системы рекомендаций основывается на анализе большого количества факторов, включающих как поведенческие данные самого пользователя, так и обобщённые данные о предпочтениях других пользователей с аналогичными вкусами. В частности, учитываются такие аспекты, как история просмотров, частота и продолжительность взаимодействия с теми или иными видео, оценка фильмов и сериалов, а также технические параметры использования платформы – язык интерфейса, тип и модель устройства, геолокация, время суток, когда совершается просмотр, и даже длительность пауз между сессиями.

Кроме того, при формировании рекомендаций используется информация о характеристиках самого контента: жанровая принадлежность, поджанры, категории, участие определённых актёров или режиссёров, год выхода, рейтинг, продолжительность и другие метаданные. Это позволяет не только учитывать прошлый опыт пользователя, но и предлагать ему новый, потенциально интересный контент, который ранее не попадался в его поле зрения.

Система рекомендаций Netflix работает по принципу персонализированного ранжирования, формируя уникальную стартовую страницу для каждого пользователя. Все ряды на главном экране, которые представляют собой горизонтальные полосы с подборками контента, подбираются и упорядочиваются индивидуально. Персонализация осуществляется на нескольких уровнях: выбираются сами ряды (например, «Вы не досмотрели», «Продолжить просмотр», «Популярное сейчас», «Фильмы, похожие на...»), затем формируется индивидуальный набор видео внутри каждого ряда, и, наконец, сортировка внутри ряда производится по степени предполагаемой релевантности для конкретного пользователя. Если пользователь выбрал арабский или иврит в качестве основного языка интерфейса, порядок отображения контента внутри ряда меняется с привычного «слева направо» на «справа налево».

Примечательно, что алгоритмы Netflix обладают способностью к непрерывному обучению. Это означает, что система динамически адаптируется к изменениям вкусов и поведения пользователя. Каждый просмотр или пропуск, каждая оценка и каждое действие в рамках платформы учитываются в общей модели, которая регулярно переобучается. Таким образом, рекомендации со временем становятся всё более точными и

персонализированными. Это обеспечивает не только повышение пользовательского удовлетворения, но и стратегические преимущества для самой компании – в виде увеличения времени использования сервиса и повышения лояльности аудитории.

На рисунке 1.1 представлен интерфейс главной страницы платформы Netflix, на которой визуализирована структура персонализированных рекомендаций и ряды с контентом, предложенным пользователю в соответствии с его предпочтениями и историей взаимодействия с сервисом.

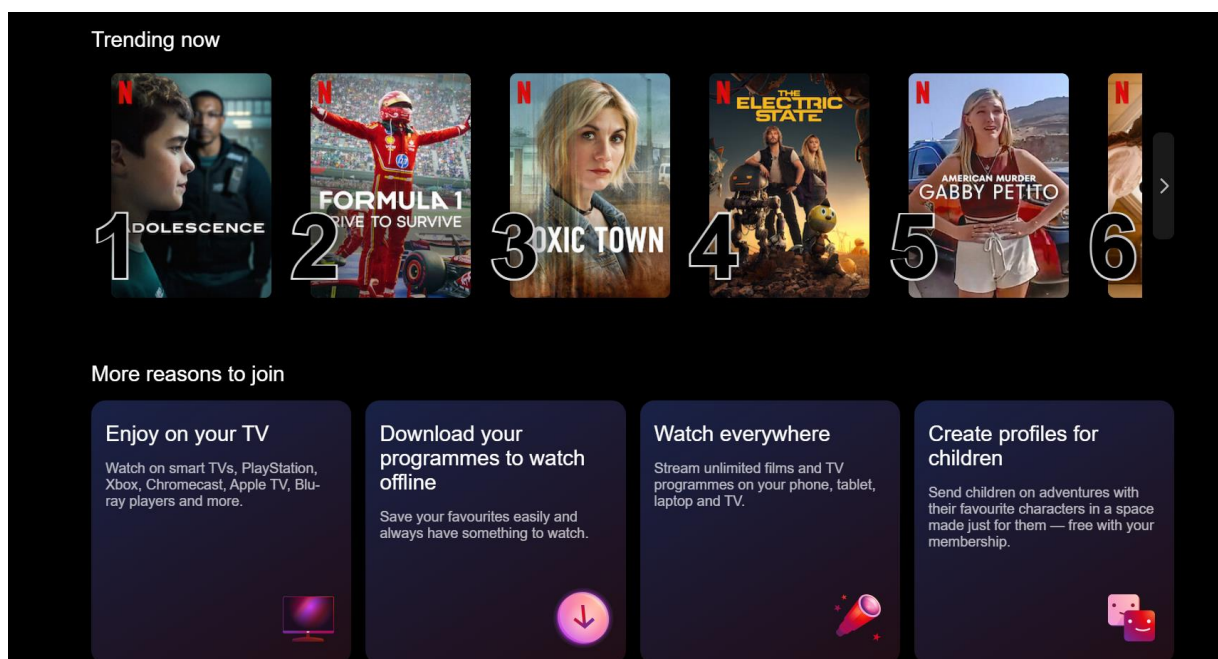


Рисунок 1.1 – Интерфейс сервиса Netflix

1.6.2 Кинопоиск.

Кинопоиск – это мультимедийный онлайн-сервис, специализирующийся на предоставлении информации о фильмах, сериалах, актёрах и режиссёрах, а также на стриминге аудиовизуального контента [7]. Помимо функции просмотра, платформа выполняет роль интеллектуального рекомендательного помощника, подбирающего персонализированные тайтлы для каждого пользователя. В рамках работы с рекомендациями Кинопоиск активно использует алгоритмы машинного обучения и нейросетевые технологии, которые обрабатывают и интерпретируют большой массив данных, поступающих от пользователей.

Для формирования максимально точных и релевантных рекомендаций, платформа собирает и анализирует широкий спектр пользовательской информации. В частности, Кинопоиск учитывает:

- поисковые запросы, введённые пользователем в строку поиска;
- историю просмотров, включая длительность просмотра каждого тайтла;
- историю переходов по страницам сервиса и карточкам фильмов или сериалов;

- пользовательские оценки, выставленные различным тайтлам;
- действия по добавлению тайтлов в личные коллекции, списки «Буду смотреть», «Любимое» и другие.

Важно отметить, что тайтлами на платформе называются отдельные единицы контента – фильмы, телесериалы, телеканалы, документальные ленты, мультфильмы и т.д. Для повышения точности рекомендаций Кинопоиск не ограничивается только пользовательскими действиями. Система также анализирует метаданные о самих тайтлах – жанры, названия, краткие и полные описания сюжетов, состав актёрского и режиссёрского состава, а также сведения о полученных наградах и номинациях. Таким образом, строится многомерное представление как о поведении пользователя, так и о характеристиках контента, что позволяет значительно улучшить релевантность выдачи рекомендательной системы в сервисе.

Система рекомендаций Кинопоиска опирается на гибридные алгоритмы, в которых используются как коллаборативные методы (учёт схожести между пользователями), так и контентные подходы (анализ признаков самих тайтлов). Важную роль играет применение матричных факторизаций – математических моделей, позволяющих выделять скрытые зависимости между пользователями и тайтлами. Эти модели обучаются на большом количестве обратной связи: лайках, дизлайках, просмотрах, отложенных просмотрах, прерванных фильмах и т.п.

Отдельное внимание уделяется метрике качества рекомендаций, которая формируется с прицелом на максимально возможную вовлечённость пользователя. Алгоритмы подбирают тайтлы таким образом, чтобы не только вызвать интерес пользователя к отдельному произведению, но и стимулировать его к регулярному использованию сервиса. При этом учитываются такие показатели, как:

- частота обращений к контенту в течение дня;
- продолжительность отдельных сеансов просмотра;
- количество дней с активностью в течение недели или месяца.

Итоговая цель алгоритма – построение рекомендательной ленты, способствующей удлинению времени пребывания пользователя на платформе и увеличению регулярности взаимодействия. То есть, система стремится не только угадать предпочтения, но и сформировать устойчивую привычку потребления контента.

Важно подчеркнуть, что точные механизмы формирования рекомендательной метрики, равно как и все параметры модели, не раскрываются публично. Это обусловлено коммерческой тайной, а также желанием предотвратить манипуляции со стороны пользователей или сторонних разработчиков.

На рисунке ниже представлен интерфейс приложения Кинопоиск, демонстрирующий структуру отображения рекомендаций, персонализированные подборки, карточки тайтлов и возможности навигации по контенту.

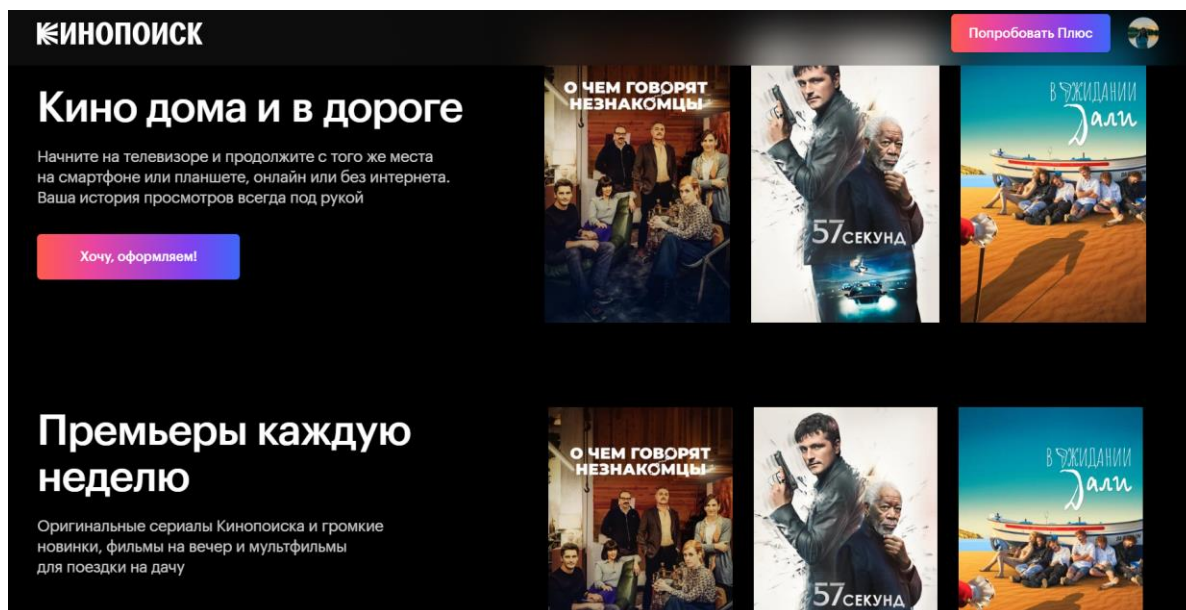


Рисунок 1.2 – Интерфейс платформы Кинопоиск

1.6.3 MovieTon.

MovieTon – это онлайн-сервис, ориентированный на предоставление персонализированных рекомендаций фильмов и сериалов [8]. Главная цель и ключевая особенность платформы заключается в точечном определении интересов каждого пользователя, что достигается посредством прохождения краткого интерактивного теста. На основе полученных ответов интеллектуальный алгоритм формирует индивидуальную подборку кинолент, максимально соответствующую вкусовым предпочтениям конкретного человека. Благодаря этой системе, даже при минимальных вводных данных пользователь получает качественные и персональные предложения к просмотру.

Механизм взаимодействия с платформой прост и интуитивно понятен: чтобы воспользоваться интеллектуальным подбором, достаточно нажать кнопку «Подобрать фильм», после чего пользователю будет предложено ответить на серию коротких вопросов, касающихся его предпочтений – например, желаемого жанра, настроения, длительности просмотра и прочего. После завершения теста сервис практически мгновенно подбирает и предоставляет список фильмов, максимально соответствующих указанным критериям. Для тех пользователей, кто предпочитает элемент неожиданности или не желает проходить опрос, доступна функция «Случайный фильм», которая позволяет получить рандомную киноленту из коллекции платформы.

Кроме автоматического подбора, MovieTon предоставляет и возможность самостоятельного поиска фильмов. Для удобства навигации контент на сайте структурирован по различным признакам: например, по тематическим подборкам, жанрам, а также по участию конкретных актёров. После выбора любого из направлений поиска открывается доступ к обширному каталогу фильмов, где каждый тайтл сопровождается необходимой справочной информацией.

Каждая карточка фильма на платформе содержит подробные сведения: официальный постер (афишу), краткое описание сюжета, список актёров, исполнивших как главные, так и второстепенные роли, имя режиссёра, год выпуска, продолжительность фильма и, при наличии, трейлер. Таким образом, пользователь получает достаточно информации, чтобы принять решение о просмотре, не переходя на сторонние ресурсы.

Отдельное внимание разработчики MovieTon уделили удобству интерфейса и навигации. Сайт обладает интуитивно понятной и наглядной визуальной структурой, что делает его использование комфортным даже для новых пользователей. Интерактивный формат взаимодействия с контентом способствует лучшему восприятию информации, а визуальные элементы и цветовая палитра делают пребывание на платформе приятным.

Зарегистрированные пользователи получают доступ к расширенному функционалу: возможность оставлять отзывы на просмотренные фильмы, добавлять любимые тайтлы в «Избранное», вести собственную подборку кинолент, а также сохранять результаты тестов и подборок. Это позволяет не только удобно организовывать собственную коллекцию контента, но и влиять на дальнейшие рекомендации системы.

На рисунке 1.3 представлен интерфейс приложения MovieTon, демонстрирующий основные элементы пользовательского взаимодействия, структуру навигации и визуальную подачу информации о тайтлах.

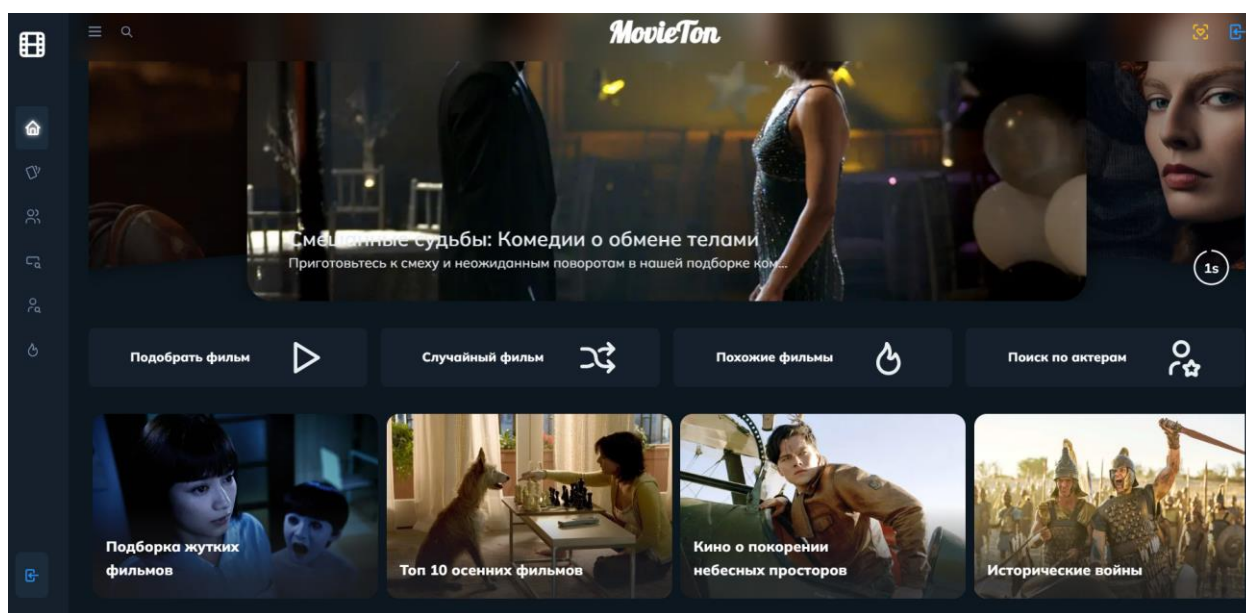


Рисунок 1.3 – Интерфейс сервиса MovieTon

2 СИСТЕМНОЕ ПРОЕКТИРОВАНИЕ

Перед тем, как приступить к разработке приложения для рекомендаций фильмов необходимо выделить определенную структуру проекта. Для эффективной работы сервиса можно выделить несколько блоков, каждый из которых отвечает за свою функцию. Перечень основных компонентов, которые будут формировать архитектуру разрабатываемого приложения:

- блок клиентского приложения;
- блок сервера;
- блок загрузки предпочтений;
- блок локальной базы данных;
- блок анализа данных;
- блок обучения модели;
- блок базы данных фильмов;
- блок подготовки исходных данных.

Подбор персональных рекомендаций будет происходить следующим образом:

- пользователь вводит предпочтения через клиентское приложение;
- клиентское приложение отправляет данные на сервер;
- сервер передает данные в блок загрузки предпочтений;
- блок загрузки предпочтений отправляет данные в блок анализа данных;
- блок анализа данных использует блок машинного обучения, которая анализирует предпочтения и генерирует рекомендации;
- рекомендации возвращаются на сервер;
- сервер отправляет рекомендации в клиентское приложение;
- клиентское приложение отображает рекомендации пользователю;
- блок обучения модели обновляет модель на основе новых пользовательских данных, полученных из локальной базы данных или базы данных фильмов.

Взаимосвязь между блоками проекта отражена на структурной схеме ГУИР.400201.043 С1

2.1 Блок клиентского приложения

Этот блок представляет собой удобный интерфейс пользователю. Он отвечает за отображение информации и обработку пользовательских действий.

Интерфейс приложения представляет собой совокупность визуальных элементов, таких как кнопки, меню, поля ввода, и списки, которые позволяют пользователю взаимодействовать с приложением. Дизайн интерфейса должен быть интуитивно понятным и удобным, что позволит легко находить фильмы и сохранять предпочтения.

Основными компонентами интерфейса являются форма ввода предпочтений и отображение рекомендаций. Форма ввода представляет собой

карточку с незаполненными постером и полем с названием, чтобы при последующем вводе в поле название предпочитаемого фильма, пользователь мог выбрать необходимый фильм и выпадающего списка, после чего установится название и постер выбранного фильма. На основе введенного пользователем фильма происходит отображение списка рекомендаций, который представлен в виде карточек с информацией рекомендуемых фильмов: описанием, постером, жанрами, актерами и прочее.

Этот блок непосредственно взаимодействует с сервером для отправки и получения данных.

2.2 Блок сервера

Блок сервера является центральным компонентом архитектуры приложения и отвечает за обработку запросов от клиентского приложения, управление данными и взаимодействие с другими модулями системы. Он принимает входящие данные от клиента, такие как предпочтения пользователей, и обрабатывает их, используя бизнес-логику и алгоритмы для генерации рекомендаций.

Сервер управляет связью с базами данных, обеспечивая хранение, извлечение и обновление информации о фильмах и предпочтениях пользователей. Он также осуществляет вызовы к модулям машинного обучения, передавая им данные для анализа и получения рекомендаций. После обработки данных сервер формирует ответ и отправляет его обратно клиенту, где результаты отображаются пользователю.

Кроме того, сервер отвечает за безопасность приложения, включая аутентификацию пользователей и защиту данных. Он может также реализовывать механизмы кэширования для ускорения доступа к часто запрашиваемым данным, что улучшает производительность приложения. В целом, блок сервера обеспечивает надежную и эффективную работу всего приложения, координируя взаимодействие между клиентом, базами данных и модулями анализа данных.

2.3 Блок загрузки предпочтений

Этот блок является компонентом серверной части приложения, отвечающий за прием и обработку пользовательских данных о предпочтениях в фильмах. Когда пользователь вводит свои предпочтения через клиентское приложение, этот блок получает эти данные и подготавливает их для дальнейшей обработки.

Он выполняет валидацию входящих данных, чтобы убедиться, что они корректны и соответствуют ожидаемым форматам. После проверки блок загружает предпочтения в соответствующий формат, который может быть использован в алгоритмах анализа данных. Затем он передает эти данные в модуль анализа, который использует их для формирования рекомендаций.

Этот компонент отвечает за установление связи между

пользовательским вводом и алгоритмами, обеспечивающими персонализированный опыт.

2.5 Блок локальной базы данных

Блок локальной базы данных представляет собой компонент, который отвечает за хранение и управление данными о фильмах, метаданными и пользовательскими данными в пределах приложения. Эта база данных обеспечивает быстрый доступ к информации, что критически важно для эффективной работы системы рекомендаций.

Локальная база данных хранит различные данные, включая названия фильмов, жанры, описания, рейтинги и обложки, а также информацию о пользовательских данных, а также их оценках и предпочтениях. Она позволяет быстро извлекать данные при выполнении запросов от клиентского приложения или других модулей, что значительно ускоряет процессы поиска и генерации рекомендаций.

Блок локальной базы данных также обеспечивает возможности для обновления информации, например, при добавлении новых фильмов или изменении пользовательских предпочтений.

2.6 Блок анализа данных

Блок анализа данных отвечает за обработку и интерпретацию пользовательских предпочтений с целью генерации персонализированных рекомендаций. Этот блок использует алгоритмы машинного обучения и статистические методы для анализа собранных данных о фильмах и пользовательских оценках.

Когда блок загрузки предпочтений передает данные о предпочтениях пользователя, блок анализа начинает обрабатывать эту информацию, выявляя паттерны и зависимости. Он учитывает различные факторы, такие как жанры, рейтинги и другие характеристики фильмов, чтобы определить, какие фильмы могут быть интересны конкретному пользователю.

В итоге, этот компонент обеспечивает основу для создания персонализированного опыта пользователя, позволяя системе адаптироваться к изменениям в предпочтениях и предоставлять рекомендации, которые соответствуют индивидуальным вкусам.

2.7 Блок обучения модели

Блок обучения модели является важным компонентом системы, он отвечает за создание и оптимизацию алгоритмов машинного обучения, используемых для генерации рекомендаций. Этот блок принимает данные о предпочтениях пользователей и характеристиках фильмов, чтобы обучить модель на основе этих данных.

Процесс обучения включает в себя несколько этапов: выбор алгоритма,

подготовка данных, обучение модели и оценка её производительности. На первом этапе выбирается подходящий алгоритм, такой как коллаборативная фильтрация, контентная фильтрация или гибридные методы. Затем данные подготавливаются, включая очистку, нормализацию и разделение на обучающую и тестовую выборки.

После этого модель обучается на обучающей выборке, где она анализирует паттерны и взаимосвязи в данных. Затем происходит тестирование модели на тестовой выборке для оценки её точности и способности делать предсказания.

2.8 Блок базы данных фильмов

Этот блок представляет собой компонент, который отвечает за хранение и управление информацией о фильмах, полученной из TMDb (The Movie Database). Блок включает в себя разнообразные данные: названия, жанры, описания, рейтинги, даты выхода и обложки, которые предоставляются API TMDb.

База данных фильмов обеспечивает быстрый доступ к информации при выполнении запросов от других компонентов системы, таких как блок поиска фильмов и блок анализа данных. Она поддерживает различные операции, включая извлечение данных о фильмах, обновление информации при изменениях на стороне TMDb и синхронизацию данных.

Этот компонент также обеспечивает целостность и согласованность данных, что позволяет системе функционировать эффективно и предоставлять пользователю актуальную информацию о фильмах.

2.9 Блок подготовки исходных данных

Блок подготовки исходных данных отвечает за обработку и трансформацию необработанных данных о фильмах перед их использованием в других компонентах системы, таких как блок анализа данных или блок обучения модели. Процесс начинается с извлечения необходимой информации о фильмах из различных источников, таких как API TMDb, базы данных или другие внешние ресурсы.

После этого происходит очистка данных, которая включает в себя удаление дубликатов, исправление ошибок и недостающих значений, а также фильтрацию неактуальной или нерелевантной информации. Затем данные преобразуются в удобный для анализа формат, включая нормализацию значений и создание новых переменных.

Блок подготовки исходных данных обеспечивает высокое качество и релевантность информации, используемой для анализа, обучения моделей и генерации рекомендаций. Это, в свою очередь, значительно улучшает точность и эффективность работы остальных компонентов системы, что критически важно для достижения успешных результатов.

3 ФУНКЦИОНАЛЬНОЕ ПРОЕКТИРОВАНИЕ

В данном разделе пояснительной записки подробно рассматривается принцип функционирования программного обеспечения, разработанного в рамках данного проекта. С целью достижения поставленных задач предоставляется детализированный обзор архитектурного устройства создаваемой системы. В частности, будет представлен анализ логической структуры приложения, включая описание основных компонентов, моделей, а также их взаимосвязей и зависимостей. Особое внимание будет уделено ключевым классам, их полям, методам и константам, которые определяют поведение системы. Также будет рассмотрена структура и содержимое таблиц, используемых в базе данных, на основе которых осуществляется хранение и обработка информации.

Разработка программного обеспечения ведется с применением объектно-ориентированного подхода, в соответствии с которым вся система разделена на логически обособленные модели, сгруппированные по выполняемым функциям. Такая организация позволяет повысить читаемость и масштабируемость кода, а также упростить дальнейшую модификацию и сопровождение продукта. На диаграмме ГУИР.400201.043 РР.1 представлена диаграмма классов программного обеспечения, что дает наглядное представление об их иерархии и взаимодействии между классами.

Ключевыми компонентами, формирующими основу разрабатываемого программного продукта, являются следующие модели:

- модель анализа данных, отвечающая за обработку, очистку и агрегирование пользовательской информации для последующего анализа;
- модель обучения модели, реализующая алгоритмы машинного обучения и содержащая методы настройки и переобучения рекомендательной системы;
- модель подготовки исходных данных, включающая инструменты предварительной обработки данных, нормализации и кодирования признаков;
- модель реляционной базы данных, обеспечивающая взаимодействие с хранилищем информации, включая создание, чтение, обновление и удаление записей;
- модель клиентского приложения, реализующая пользовательский интерфейс и взаимодействие с серверной частью.

3.1 Модель анализа данных

Модель анализа данных отвечает за следующие задачи:

- подготовка данных;
- анализ предпочтений пользователей;
- генерация рекомендаций;
- анализирует данные пользователя для улучшения рекомендаций.

Для реализации перечисленных выше функций предусмотрены классы, отвечающие за отдельно взятые задачи. Подробное описание классов

рассмотрено далее.

3.1.1 Класс **DataAnalyzer**

Класс `DataAnalyzer` является ключевым компонентом системы анализа данных, отвечающим за глубокий анализ пользовательских предпочтений и паттернов в фильмах. Этот класс играет важную роль в формировании персонализированных рекомендаций и выявлении общих трендов в кинематографе. Его основная цель – извлечение ценных инсайтов из данных о пользователях и фильмах для улучшения качества рекомендаций.

Основные поля и функции класса:

1 `analyze_user_preferences(self, user_id)` – функция анализа предпочтений пользователя. Выполняет комплексный анализ избранных фильмов пользователя, включая анализ жанровых предпочтений с подсчетом частоты встречаемости каждого жанра, анализ предпочтений по актерам, выявляя наиболее часто встречающихся актеров в избранных фильмах, анализ предпочтений по странам производства и анализ временных предпочтений по годам выпуска фильмов. Результаты анализа используются для создания персонализированного профиля пользователя.

2 `analyze_movie_patterns(self)` – функция анализа общих паттернов в фильмах. Выполняет глобальный анализ всей базы фильмов, включая анализ популярности жанров с подсчетом количества фильмов в каждом жанре, анализ популярности актеров с подсчетом количества фильмов с их участием, анализ популярности стран производства и анализ статистики рейтингов (среднее, минимальное, максимальное значение). Эти данные помогают выявить общие тренды и популярные направления в кинематографе.

3 `generate_insights(self, user_id=None)` – функция генерации инсайтов на основе проведенного анализа. Объединяет результаты анализа глобальных паттернов и пользовательских предпочтений, находит пересечение предпочтений пользователя с глобальными трендами, формирует рекомендации по жанрам и актерам и создает комплексный отчет с инсайтами для дальнейшего использования в системе рекомендаций.

4 `get_trending_movies(self, limit=10)` – функция определения трендовых фильмов. Анализирует популярность фильмов на основе количества добавлений в избранное, учитывает историю просмотров и возвращает список наиболее популярных фильмов. Эта функция помогает выявлять актуальные тренды и популярный контент в системе.

Класс `DataAnalyzer` тесно интегрирован с базой данных и использует различные методы агрегации и группировки данных для получения точных статистических показателей.

3.2 Модель обучения модели

Модель обучения модели является важной частью проекта, так как он отвечает за разработку и оптимизацию алгоритмов, которые генерируют

рекомендации для пользователей. Этот блок включает в себя несколько ключевых классов, обеспечивающих создание, обучение и оценку моделей на основе данных о пользователях и фильмах.

Он реализует методы коллаборативной и контентной фильтрации, обучаясь на исторических данных о предпочтениях пользователей. Процесс включает извлечение и подготовку признаков, таких как жанры фильмов и актерский состав, что позволяет моделям эффективно предсказывать интересы пользователей.

После обучения модели проходят тщательную оценку с использованием метрик, таких как RMSE и MAE, что позволяет точно определить качество рекомендаций. Настройка гиперпараметров, включая число соседей и коэффициенты регуляризации, обеспечивает максимальную производительность моделей. Кроме того, блок гарантирует возможность сохранения и загрузки обученных моделей, что существенно экономит время на повторное обучение.

Регулярное обновление моделей с использованием новых данных позволяет системе поддерживать актуальность рекомендаций, учитывая изменяющиеся предпочтения пользователей и новые поступления контента.

Классы блока подробно описаны далее.

3.2.1 Класс `ModelTrainer`

Класс `ModelTrainer` является центральным компонентом системы обучения моделей рекомендаций, отвечающим за управление процессом обучения, оценки и обновления моделей. Этот класс координирует работу различных типов рекомендательных систем и обеспечивает их эффективное взаимодействие для получения качественных рекомендаций. Основные поля и функции класса:

1 `model_path` – путь к директории, где хранятся обученные модели. По умолчанию установлен как `'recommender/data'`. Это поле определяет местоположение файлов моделей в файловой системе.

2 `hybrid_recommender` – экземпляр класса `HybridRecommender`, отвечающий за гибридную систему рекомендаций. Инициализируется при создании объекта `ModelTrainer`.

3 `movie_recommender` – экземпляр класса `MovieRecommender`, отвечающий за контентную фильтрацию. Инициализируется с указанным путем к данным.

4 `metrics` – словарь для хранения метрик качества моделей. Содержит результаты оценки как гибридной, так и контентной моделей, а также временные метки обновлений.

5 `prepare_training_data(self)` – функция подготовки данных для обучения. Создает матрицу взаимодействия пользователь-фильм на основе данных о избранных фильмах пользователей. Эта матрица является основой для обучения моделей рекомендаций и отражает предпочтения пользователей в структурированном виде. Функция собирает все избранные

фильмы пользователей и формирует из них обучающую выборку.

6 `train_recommendation_model(self)` – функция обучения моделей рекомендаций. Выполняет полный цикл обучения, включая подготовку данных, обучение гибридной модели, создание матрицы схожести для контентной модели, сохранение обученных моделей и оценку их качества. Функция обеспечивает согласованное обучение различных типов моделей и их последующее сохранение.

7 `update_model(self, new_data)` – функция обновления модели на основе новых данных. Позволяет переобучать модель с учетом новых данных о предпочтениях пользователей. Функция обновляет обучающую выборку, переобучает модель и логирует процесс обновления для последующего анализа. Это обеспечивает актуальность рекомендаций при появлении новых данных.

8 `evaluate_model(self)` – функция оценки качества моделей. Выполняет комплексную оценку как гибридной, так и контентной моделей, собирая различные метрики качества. Функция использует тестовые данные для проверки эффективности моделей и сохраняет результаты оценки для последующего анализа. Это позволяет отслеживать качество рекомендаций и выявлять области для улучшения.

9 `save_models(self)` и `load_models(self)` – функции сохранения и загрузки моделей. Обеспечивают персистентность обученных моделей, позволяя сохранять их в файловой системе и загружать при необходимости. Функции работают как с гибридной, так и с контентной моделями, обеспечивая их целостность при сохранении и загрузке.

10 `log_model_update(self)` – функция логирования обновлений модели. Создает записи в базе данных о каждом обновлении модели, сохраняя метрики качества и временные метки. Это позволяет отслеживать историю изменений моделей и анализировать их эффективность во времени.

Класс `ModelTrainer` тесно интегрирован с базой и использует различные компоненты системы рекомендаций, такие как `HybridRecommender` и `MovieRecommender`. Он обеспечивает надежное управление процессом обучения моделей и их последующего использования в системе рекомендаций. Результаты работы класса используются для генерации персонализированных рекомендаций фильмов для пользователей.

3.2.2 Класс `MovieRecommender`

Класс `MovieRecommender` является компонентом системы рекомендаций, реализующим контентную фильтрацию фильмов на основе их характеристик и описаний. Этот класс анализирует различные аспекты фильмов, такие как жанры, актеры, режиссеры и описания, для создания персонализированных рекомендаций. Основные поля и функции класса:

1 `data_path` – путь к директории с данными. По умолчанию установлен как `'data'`. Определяет местоположение файлов с данными о фильмах и сохраненных моделей.

2 `movies` – объект, содержащий основную информацию о фильмах (`id`, `title`, `original_title`, `tags`). Используется для хранения и обработки данных о фильмах.

3 `movies_data` – полный набор данных о фильмах, включающий все характеристики и описания. Содержит расширенную информацию для анализа.

4 `similarity_matrix` – матрица схожести фильмов, вычисленная на основе их характеристик. Используется для определения похожих фильмов.

5 `ps` – экземпляр `PorterStemmer` для стемминга слов в текстовых данных. Помогает нормализовать текстовые описания фильмов.

6 `_preprocess_text(self, text)` – внутренняя функция предобработки текста. Выполняет комплексную обработку текстовых данных, включая стемминг, удаление стоп-слов, пунктуации и коротких слов. Функция преобразует текстовые описания фильмов в нормализованный формат, подходящий для дальнейшего анализа и сравнения.

7 `load_data(self, credits_path, movies_path)` – функция загрузки и подготовки данных. Загружает данные о фильмах и их создателях из CSV файлов, объединяет различные характеристики фильмов в единый набор признаков и выполняет предобработку текстовых данных. Функция создает структурированный набор данных, готовый для анализа и создания рекомендаций.

8 `create_similarity_matrix(self)` – функция создания матрицы схожести фильмов. Использует TF-IDF векторизацию для преобразования текстовых признаков фильмов в числовой формат и вычисляет косинусное сходство между фильмами. Функция создает основу для определения похожих фильмов и генерации рекомендаций.

9 `save_model(self, filename='movie_recommender.pkl')` и `load_model(self, filename='movie_recommender.pkl')` – функции сохранения и загрузки модели. Обеспечивают персистентность обученной модели, сохраняя и загружая матрицу схожести и данные о фильмах. Это позволяет использовать обученную модель без необходимости повторного обучения.

10 `get_recommendations(movie_ids, n_recommendations=10)` – функция получения рекомендаций на основе ID фильмов. Анализирует схожесть между указанными фильмами и остальными фильмами в базе данных, используя матрицу схожести. Функция возвращает список ID рекомендованных фильмов, отсортированных по степени схожести. Поддерживает рекомендации на основе от 1 до 3 фильмов.

Класс `MovieRecommender` использует различные библиотеки для обработки данных и машинного обучения, такие как `pandas`, `numpy`, `scikit-learn` и `nlTK`. Он обеспечивает эффективную контентную фильтрацию фильмов на основе их характеристик и описаний, что позволяет создавать персонализированные рекомендации для пользователей. Результаты работы класса используются в общей системе рекомендаций для улучшения качества

и разнообразия предлагаемых фильмов.

3.2.3 Класс HybridRecommender

Класс HybridRecommender является компонентом системы рекомендаций, реализующим гибридный подход к рекомендациям фильмов, сочетающий контентную фильтрацию с коллаборативной фильтрацией. Этот класс анализирует как характеристики самих фильмов, так и поведение пользователей для создания более точных и разнообразных рекомендаций. Основные поля и функции класса:

1 `app` – экземпляр Flask-приложения. Используется для доступа к контексту приложения и базе данных.

2 `tfidf` – экземпляр TfidfVectorizer для преобразования текстовых данных в TF-IDF векторы. Настроен на работу с английским языком и удаление стоп-слов.

3 `content_similarity_matrix` – матрица схожести фильмов, вычисленная на основе их контентных характеристик. Используется для определения похожих фильмов по их содержанию.

4 `movie_id_to_index` – словарь для сопоставления ID фильмов с их индексами в матрице схожести. Обеспечивает быстрый доступ к данным о фильмах.

5 `index_to_movie_id` – обратный словарь для сопоставления индексов с ID фильмов. Используется для преобразования результатов анализа обратно в ID фильмов.

6 `_get_movie_features(self, movie)` – внутренняя функция извлечения признаков фильма. Собирает и объединяет различные характеристики фильма, включая жанры, актеров, страны производства и описание. Функция создает комплексное представление фильма, учитывающее все его важные аспекты для последующего анализа.

7 `build_content_similarity_matrix(self)` – функция построения матрицы схожести фильмов на основе контента. Извлекает признаки для каждого фильма, создает TF-IDF матрицу и вычисляет косинусное сходство между фильмами. Функция также создает сопоставления между ID фильмов и их индексами в матрице схожести для эффективного доступа к данным.

8 `get_content_based_recommendations(self, movie_id, n_recommendations=10)` – функция получения рекомендаций на основе контента. Использует матрицу схожести для определения фильмов, наиболее похожих на указанный фильм. Функция учитывает различные аспекты фильмов и возвращает список ID рекомендованных фильмов, отсортированных по степени схожести.

9 `init_app(self, app)` – функция инициализации приложения. Устанавливает связь с Flask-приложением для доступа к контексту и базе данных. Эта функция необходима для корректной работы класса в контексте веб-приложения.

Класс `HybridRecommender` обеспечивает комплексный подход к рекомендациям, учитывая как характеристики фильмов, так и предпочтения пользователей. Результаты работы класса используются для создания более точных и разнообразных рекомендаций, учитывающих различные аспекты фильмов и предпочтений пользователей.

3.3 Модель подготовки исходных данных

Модель подготовки исходных данных ориентирован на извлечение и обработку информации о фильмах из API TMD. Он обеспечивает надежный и эффективный процесс извлечения, обработки и подготовки данных, что критически важно для последующего обучения моделей машинного обучения. Этот блок включает несколько классов, каждый из которых выполняет специфические задачи, направленные на обеспечение качества и актуальности данных.

К основным функциям компонента относятся:

- извлечение данных;
- обработка и трансформация данных;
- нормализация данных;
- валидация данных;
- сохранение данных.

За каждую из перечисленных функций отвечает свой класс. Подробнее про классы далее.

3.3.1 Класс `TMDBDownloader`

Класс `TMDBDownloader` является компонентом системы, отвечающим за загрузку и сохранение данных о фильмах из The Movie Database (TMDB) API. Этот класс обеспечивает эффективную и надежную загрузку большого объема данных с учетом ограничений API и возможных ошибок. Основные поля класса:

1 `api_key` – ключ для доступа к TMDB API. Очищается от лишних кавычек при инициализации.

2 `base_url` – базовый URL для API запросов к TMDB.

3 `language` – язык, на котором будут загружаться данные (по умолчанию "ru-RU").

4 `movies_data` – список для хранения загруженных данных о фильмах.

5 `error_log` – список для записи ошибок, возникающих при загрузке.

6 `max_retries` – максимальное количество попыток повторного запроса при ошибке (по умолчанию 5).

7 `retry_delay` – задержка между повторными попытками в секундах (по умолчанию 5).

8 `current_page` – текущая страница пагинации при загрузке данных.

9 `script_dir` – абсолютный путь к директории скрипта.

10 `progress_file` – путь к файлу для сохранения прогресса загрузки.
11 `error_log_file` – путь к файлу для сохранения лога ошибок.
12 `data_dir` – путь к директории для сохранения загруженных данных.
13 `last_save_time` – время последнего сохранения прогресса.
14 `save_interval` – интервал сохранения прогресса в секундах (по умолчанию 30).
15 `headers` – HTTP заголовки для запросов к API.
16 `handle_interrupt(self, signum, frame)` – функция обработки прерываний. Обеспечивает корректное завершение работы при получении сигналов прерывания (SIGINT, SIGTERM). Сохраняет текущий прогресс загрузки перед завершением работы, что позволяет возобновить загрузку с места остановки.
17 `save_progress(self)` – функция сохранения прогресса загрузки. Сохраняет текущее состояние загрузки, включая номер последней страницы, количество загруженных фильмов и временную метку. Это позволяет возобновить загрузку с места остановки в случае прерывания.
18 `log_error(self, error_type, details)` – функция логирования ошибок. Записывает информацию об ошибках, возникающих при загрузке данных, включая тип ошибки, детали и временную метку. Это помогает отслеживать и анализировать проблемы при загрузке.
19 `get_popular_movies(self, client, page=1)` – функция получения списка популярных фильмов. Выполняет запрос к API для получения списка популярных фильмов с указанной страницы. Обрабатывает возможные ошибки и повторяет запрос при необходимости.
20 `get_movie_details(self, client, movie_id)` – функция получения детальной информации о фильме. Запрашивает полную информацию о фильме, включая кредиты, даты выпуска, внешние идентификаторы, ключевые слова, видео и изображения. Обрабатывает возможные ошибки и логирует их.
21 `process_movie_batch(self, client, movie_ids)` – функция обработки пакета фильмов. Параллельно обрабатывает несколько фильмов для ускорения загрузки. Использует асинхронные запросы для эффективной работы с API.
22 `download_data(self, num_movies=5000)` – основная функция загрузки данных. Управляет процессом загрузки данных о фильмах, включая создание необходимых директорий, загрузку сохраненного прогресса, последовательную загрузку данных с учетом пагинации, обработку пакетов фильмов, сохранение прогресса.

3.3.2 Класс `DataPreprocessor`

Класс `DataPreprocessor` является компонентом системы обработки данных, отвечающим за предварительную обработку и подготовку данных о фильмах для последующего использования в системе рекомендаций. Этот

класс обеспечивает очистку, нормализацию и трансформацию данных из различных источников в единый формат, пригодный для анализа и обучения моделей. Основные поля и функции класса:

1 `movies_df` – `DataFrame` с данными о фильмах. Содержит основную информацию о фильмах, включая названия, годы выпуска, жанры и другие метаданные.

2 `ratings_df` – `DataFrame` с пользовательскими оценками. Содержит информацию о том, как пользователи оценивали различные фильмы.

3 `genres_df` – `DataFrame` с жанрами фильмов. Содержит информацию о жанровой принадлежности фильмов.

4 `keywords_df` – `DataFrame` с ключевыми словами. Содержит ключевые слова, связанные с фильмами.

5 `actors_df` – `DataFrame` с данными об актерах. Содержит информацию об актерах, участвовавших в фильмах.

6 `directors_df` – `DataFrame` с данными о режиссерах. Содержит информацию о режиссерах фильмов.

7 `processed_data` – словарь с обработанными данными. Хранит результаты предварительной обработки для последующего использования.

8 `config` – конфигурация предобработки. Содержит настройки и параметры для процесса предварительной обработки данных.

9 `preprocess_movies(self)` – функция предобработки данных о фильмах. Выполняет очистку данных, удаление дубликатов, заполнение пропущенных значений и нормализацию числовых признаков. Функция обеспечивает приведение данных о фильмах к единому формату.

10 `preprocess_ratings(self)` – функция предобработки пользовательских оценок. Выполняет очистку данных, удаление выбросов и нормализацию оценок. Функция подготавливает данные о пользовательских оценках для последующего анализа.

11 `preprocess_genres(self)` – функция предобработки жанров. Создает `one-hot encoding` для жанров фильмов, преобразуя категориальные данные в числовой формат. Функция обеспечивает представление жанровой информации в виде, пригодном для машинного обучения.

12 `preprocess_keywords(self)` – функция предобработки ключевых слов. Создает `TF-IDF` представление ключевых слов, преобразуя текстовые данные в числовой формат. Функция обеспечивает представление ключевых слов в виде, пригодном для анализа.

13 `preprocess_actors(self)` – функция предобработки данных об актерах. Создает признаки на основе популярности актеров и их участия в фильмах. Функция извлекает полезную информацию из данных об актерах.

14 `preprocess_directors(self)` – функция предобработки данных о режиссерах. Создает признаки на основе популярности режиссеров и их работ. Функция извлекает полезную информацию из данных о режиссерах.

15 `preprocess_all(self)` – функция выполнения полной

последовательности предобработки. Координирует выполнение всех этапов предварительной обработки данных. Функция обеспечивает комплексную подготовку данных для последующего использования.

16 `save_processed_data(self)` – функция сохранения обработанных данных. Сохраняет результаты предварительной обработки в файлы для последующего использования. Функция обеспечивает сохранение промежуточных результатов.

17 `load_processed_data(self)` – функция загрузки обработанных данных. Загружает ранее сохраненные результаты предварительной обработки из файлов. Функция обеспечивает возможность использования ранее обработанных данных.

Класс `DataPreprocessor` обеспечивает комплексную подготовку данных для системы рекомендаций, выполняя необходимые преобразования и очистку данных из различных источников. Результаты работы класса используются для создания качественных входных данных для моделей рекомендаций, что способствует повышению точности и эффективности системы рекомендаций в целом.

3.4 Модель реляционной базы данных

Модель реляционной базы данных играет критическую роль в структуре системы, обеспечивая надежное хранение и управление данными о фильмах, пользователях и других связанных сущностях. Этот блок отвечает за организацию данных, их целостность и доступность, что позволяет эффективно извлекать и обрабатывать информацию для дальнейшего анализа и использования в приложении.

Для взаимодействия с реляционной базой данных на стороне бэкенда будут реализованы несколько классов, каждый из которых отвечает за различные аспекты работы с данными.

3.4.1 Класс `User`

Класс `User` является ключевым компонентом системы аутентификации и управления пользователями. Этот класс реализует функциональность для работы с пользовательскими данными, обеспечивая безопасное хранение учетных данных и управление сессиями пользователей. Класс наследуется от `UserMixin` и `db.Model`, что позволяет использовать его как с системой аутентификации `Flask-Login`, так и с ORM `SQLAlchemy` для работы с базой данных.

Основные функции класса:

1 `get_id()` – функция получения уникального идентификатора пользователя. Возвращает строковое представление ID пользователя, что необходимо для работы с `Flask-Login`.

2 `is_active()` – свойство, определяющее активность пользователя. Возвращает `True`, если пользователь активен и может использовать систему.

3 `is_authenticated()` – свойство, определяющее аутентификацию пользователя. Возвращает `True`, если пользователь успешно аутентифицирован в системе.

4 `is_anonymous()` – свойство, определяющее анонимность пользователя. Возвращает `False`, так как этот класс представляет аутентифицированных пользователей.

5 `set_password()` – функция установки пароля пользователя. Принимает пароль в открытом виде, хеширует его с использованием Werkzeug и сохраняет хеш в базе данных. Обеспечивает безопасное хранение паролей.

6 `check_password()` – функция проверки пароля пользователя. Сравнивает предоставленный пароль с сохраненным хешем, используя Werkzeug. Обеспечивает безопасную проверку паролей.

Класс `User` обеспечивает надежную основу для системы аутентификации, предоставляя необходимый функционал для управления пользовательскими данными и обеспечения безопасности. Он интегрируется с Flask-Login для управления сессиями и с SQLAlchemy для работы с базой данных, что позволяет эффективно хранить и обрабатывать информацию о пользователях.

3.4.2 Класс `Movie`

Класс `Movie` является центральным компонентом системы хранения и управления данными о фильмах. Этот класс представляет собой модель фильма в базе данных и содержит всю необходимую информацию о кинематографических произведениях, включая их основные характеристики и связи с другими сущностями системы. Класс наследуется от `db.Model`, что обеспечивает его интеграцию с ORM SQLAlchemy для эффективной работы с базой данных.

Основные поля и функции класса:

- 1 `id` – уникальный идентификатор фильма в базе данных;
- 2 `tmdb_id` – идентификатор фильма в базе данных TMDb;
- 3 `title` – название фильма;
- 4 `overview` – описание фильма;
- 5 `poster_url` – URL постера фильма;
- 6 `release_date` – дата выпуска фильма;
- 7 `rating` – рейтинг фильма;
- 8 `genres` – отношение к жанрам фильма;
- 9 `actors` – отношение к актерам фильма;
- 10 `directors` – отношение к режиссерам фильма;
- 11 `countries` – отношение к странам производства;
- 12 `favorites` – отношение к избранным фильмам пользователей;
- 13 `recommendation_logs` – отношение к логам рекомендаций.

Класс `Movie` служит основой для хранения и управления данными о фильмах в системе, обеспечивая структурированное хранение информации и

эффективные связи с другими сущностями.

3.4.3 Класс Genre

Класс Genre является компонентом системы категоризации фильмов, отвечающим за хранение и управление информацией о жанрах кинематографических произведений. Этот класс представляет собой модель жанра в базе данных и обеспечивает структурированное хранение жанровой информации, что является важным аспектом для системы рекомендаций и поиска фильмов. Класс наследуется от `db.Model`, что обеспечивает его интеграцию с ORM SQLAlchemy для эффективной работы с базой данных.

Основные поля и функции класса:

- 1 `id` – уникальный идентификатор жанра в базе данных;
- 2 `name` – название жанра;
- 3 `movies` – отношение к фильмам данного жанра.

Класс Genre обеспечивает основу для жанровой классификации фильмов в системе, позволяя эффективно категоризировать кинематографические произведения и осуществлять поиск по жанрам. Эта классификация является важным фактором при формировании рекомендаций и фильтрации контента для пользователей.

3.4.4 Класс Actor

Класс Actor является компонентом системы хранения информации об актерах, участвующих в кинематографических произведениях. Этот класс представляет собой модель актера в базе данных и обеспечивает структурированное хранение информации о персоналиях, что является важным аспектом для системы рекомендаций и поиска фильмов. Класс наследуется от `db.Model`, что обеспечивает его интеграцию с ORM SQLAlchemy для эффективной работы с базой данных.

Основные поля и функции класса:

- 1 `id` – уникальный идентификатор актера в базе данных;
- 2 `name` – имя актера;
- 3 `movies` – отношение к фильмам, в которых участвовал актер.

Класс Actor обеспечивает основу для хранения информации об актерах в системе, позволяя эффективно отслеживать участие актеров в различных фильмах и осуществлять поиск по актерскому составу. Эта информация является важным фактором при формировании рекомендаций и анализе предпочтений пользователей.

3.4.5 Класс Director

Класс Director является компонентом системы хранения информации о режиссерах, создающих кинематографические произведения. Этот класс представляет собой модель режиссера в базе данных и обеспечивает структурированное хранение информации о создателях фильмов, что является важным аспектом для системы рекомендаций и поиска. Класс наследуется от

db.Model, что обеспечивает его интеграцию с ORM SQLAlchemy для эффективной работы с базой данных.

Основные поля и функции класса:

1 id – уникальный идентификатор режиссера в базе данных;

2 name – имя режиссера;

3 movies – отношение к фильмам, снятым режиссером.

Класс Director обеспечивает основу для хранения информации о режиссерах в системе, позволяя эффективно отслеживать творчество режиссеров и осуществлять поиск по создателям фильмов. Эта информация является важным фактором при формировании рекомендаций и анализе предпочтений пользователей, особенно для тех, кто интересуется работами конкретных режиссеров.

3.4.6 Класс Country

Класс Country является компонентом системы хранения информации о странах производства кинематографических произведений. Этот класс представляет собой модель страны в базе данных и обеспечивает структурированное хранение географической информации о фильмах, что является важным аспектом для системы рекомендаций и поиска. Класс наследуется от db.Model, что обеспечивает его интеграцию с ORM SQLAlchemy для эффективной работы с базой данных.

Основные поля класса:

1 id – уникальный идентификатор страны в базе данных;

2 name – название страны;

3 movies – отношение к фильмам, произведенным в данной стране.

Класс Country обеспечивает основу для хранения информации о странах производства фильмов в системе, позволяя эффективно фильтровать фильмы по географическому происхождению и осуществлять поиск по странам. Эта информация является важным фактором при формировании рекомендаций и анализе предпочтений пользователей, особенно для тех, кто интересуется кинематографом определенных стран.

3.4.7 Класс Favorite

Класс Favorite является компонентом системы управления избранными фильмами пользователей. Этот класс представляет собой модель избранного фильма в базе данных и обеспечивает структурированное хранение информации о предпочтениях пользователей, что является важным аспектом для персонализации рекомендаций. Класс наследуется от db.Model, что обеспечивает его интеграцию с ORM SQLAlchemy для эффективной работы с базой данных.

Основные поля и функции класса:

1 id – уникальный идентификатор записи в базе данных;

2 user_id – идентификатор пользователя;

3 movie_id – идентификатор фильма;

- 4 `added_at` – дата и время добавления фильма в избранное;
- 5 `user` – отношение к пользователю, добавившему фильм в избранное;
- 6 `movie` – отношение к фильму, добавленному в избранное.

Класс `Favorite` обеспечивает основу для хранения информации об избранных фильмах пользователей в системе, позволяя эффективно отслеживать предпочтения пользователей и использовать эту информацию для формирования персонализированных рекомендаций. Эта информация является ключевым фактором при анализе пользовательских предпочтений и улучшении качества рекомендаций.

3.5 Описание таблиц локальной базы данных

3.5.1 Таблица `users`

Таблица отвечает за хранение информации о пользователях системы.

Основные поля:

- `id` (`SERIAL PRIMARY KEY`) – уникальный идентификатор пользователя;
- `username` (`VARCHAR(50) UNIQUE NOT NULL`) – имя пользователя;
- `password_hash` (`VARCHAR(255) NOT NULL`) – хеш пароля;
- `email` (`VARCHAR(100)`) – электронная почта;
- `created_at` (`TIMESTAMP DEFAULT CURRENT_TIMESTAMP`) – дата регистрации.

3.5.2 Таблица `movies`

Таблицы с основной информацией о фильмах.

Основные поля:

- `id` (`SERIAL PRIMARY KEY`) – уникальный идентификатор фильма;
- `tmdb_id` (`INTEGER UNIQUE NOT NULL`) – идентификатор в базе TMDB;
- `title` (`VARCHAR(255) NOT NULL`) – название фильма;
- `overview` (`TEXT`) – описание фильма;
- `poster_url` (`VARCHAR(255)`) – ссылка на постер;
- `release_date` (`DATE`) – дата выпуска;
- `rating` (`FLOAT`) – рейтинг фильма.

3.5.3 Таблица `genres`

Хранение информации о жанрах фильмов, содержит следующие поля:

- `id` (`SERIAL PRIMARY KEY`) – уникальный идентификатор жанра;
- `name` (`VARCHAR(50) UNIQUE NOT NULL`) – название жанра.

3.5.4 Таблица `actors`

Таблица с информацией об актерах.

Основные поля:

- id (SERIAL PRIMARY KEY) – уникальный идентификатор актера;
- name (VARCHAR(100) NOT NULL) – имя актера.

3.5.5 Таблица **directors**

Информации о режиссерах фильмов со следующими полями:

- id (SERIAL PRIMARY KEY) – уникальный идентификатор режиссера;
- name (VARCHAR(100) NOT NULL) – имя режиссера.

3.5.6 Таблица **countries**

Хранение информации о странах производства.

Основные поля:

- id (SERIAL PRIMARY KEY) – уникальный идентификатор страны;
- name (VARCHAR(100) UNIQUE NOT NULL) – название страны.

3.5.7 Таблица **movies_genres**

Связь многие-ко-многим между фильмами и жанрами.

Основные поля:

- movie_id (INTEGER REFERENCES movies(id)) – идентификатор фильма;
- genre_id (INTEGER REFERENCES genres(id)) – идентификатор жанра.

3.5.8 Таблица **movies_actors**

Связь многие-ко-многим между фильмами и актерами.

Основные поля:

- movie_id (INTEGER REFERENCES movies(id)) – идентификатор фильма;
- actor_id (INTEGER REFERENCES actors(id)) – идентификатор актера.

3.5.9 Таблица **movies_directors**

Связь многие-ко-многим между фильмами и режиссерами, содержит следующие поля:

- movie_id (INTEGER REFERENCES movies(id)) – идентификатор фильма;
- director_id (INTEGER REFERENCES directors(id)) – идентификатор режиссера.

3.5.10 Таблица **movies_countries**

Связь многие-ко-многим между фильмами и странами.

Основные поля:

– movie_id (INTEGER REFERENCES movies(id)) – идентификатор фильма;
– country_id (INTEGER REFERENCES countries(id)) – идентификатор страны.

3.5.11 Таблица **favorites**

Таблица хранит информацию об избранных фильмах пользователей.
Основные поля:

– id (SERIAL PRIMARY KEY) – уникальный идентификатор записи;
– user_id (INTEGER REFERENCES users(id)) – идентификатор пользователя;
– movie_id (INTEGER REFERENCES movies(id)) – идентификатор фильма;
– added_at (TIMESTAMP DEFAULT CURRENT_TIMESTAMP) – дата добавления.

3.5.12 Таблица **search_history**

Истории поисковых запросов пользователей.

Основные поля:

– id (SERIAL PRIMARY KEY) – уникальный идентификатор записи;
– user_id (INTEGER REFERENCES users(id)) – идентификатор пользователя;
– query (TEXT) – поисковый запрос;
– date (TIMESTAMP DEFAULT CURRENT_TIMESTAMP) – дата запроса.

3.5.13 Таблица **recommendation_logs**

Хранение логов рекомендаций фильмов, содержит поля:

– id (SERIAL PRIMARY KEY) – уникальный идентификатор записи;
– user_id (INTEGER REFERENCES users(id)) – идентификатор пользователя;
– movie_id (INTEGER REFERENCES movies(id)) – идентификатор фильма;
– based_on_movies (JSONB) – фильмы, на основе которых сделаны рекомендации;
– recommended_ids (JSONB) – рекомендованные фильмы;
– created_at (TIMESTAMP DEFAULT CURRENT_TIMESTAMP) – дата создания рекомендации.

3.6 Модель клиентского приложения

Модель клиентского приложения представляет собой фронтенд-часть системы, которая обеспечивает взаимодействие пользователей с

функциональностью, реализованной на бэкенде. В данном случае, фронтенд будет разрабатываться с использованием Vue.js, что позволит создать отзывчивый и интерактивный интерфейс. Этот блок отвечает за отображение данных, обработку пользовательских запросов и взаимодействие с API.

К основным функциям и компонентам блока относятся:

- интерфейс пользователя;
- маршрутизация;
- взаимодействие с сервером.

3.6.1 Интерфейс пользователя

Интерфейс пользователя (UI) является одним из основных аспектов этого блока. Он состоит из различных компонентов Vue, которые отображают информацию о фильмах, такие как карточки с описанием, изображения, рейтинги и другие ключевые атрибуты. Каждый компонент разрабатывается с учетом принципа переиспользования, что позволяет легко изменять и настраивать интерфейс в зависимости от потребностей пользователей. Стилизация интерфейса осуществляется с использованием CSS-фреймворков, таких как Bootstrap или Tailwind CSS, что помогает создать современный и привлекательный дизайн. Это делает приложение более интуитивным и удобным для пользователей, позволяя им легко ориентироваться в доступном контенте.

К основным компонентам приложения относятся:

- `SearchBar.vue` – компонент поиска фильмов;
- `AppBar.vue` – шапка приложения с логотипом и навигацией;
- `NavigationMenu.vue` – меню навигации с избранным и аккаунтом;
- `MovieCarousel.vue` – карусель популярных фильмов;
- `MovieCard.vue` – карточка отдельного фильма;
- `MovieList.vue` – список фильмов с возможностью слайдера;
- `SimilarMoviePicker.vue` – компонент для подбора похожих фильмов;
- `FilmDetailsModal.vue` – модальное окно с детальной информацией о фильме;
- `MoviePickerModal.vue` – модальное окно для подбора фильмов по параметрам;
- `PickedMovieModal.vue` – модальное окно для отображения подобранных фильмов;
- `RandomFilm.vue` – модальное окно для случайного фильма;
- `HomePage.vue` – главная страница приложения, объединяющая все компоненты.

3.6.2 Маршрутизация

Маршрутизация в приложении организована с помощью Vue Router. Это позволяет пользователю перемещаться между различными страницами, такими как главная страница, страница деталей фильма и страница поиска.

Динамические маршруты обеспечивают возможность отображения информации о конкретных фильмах на основе их идентификаторов, что делает навигацию более удобной и логичной. Пользователи могут быстро получать доступ к интересующим их данным, что улучшает общий опыт работы с приложением.

В файле `src/router/router.js` настроена основная конфигурация маршрутизации с использованием Vue Router:

```
const routes = [
  { path: "/", component: HomePage },
  {
    path: "/favorites",
    component: FavoritesPage,
    meta: { requiresAuth: true },
  },
  { path: "/login", component: LoginPage },
  { path: "/register", component: RegisterPage },
];
```

К основным маршрутам в приложении относятся:

- / – главная страница (HomePage);
- /favorites – страница избранных фильмов (требуется аутентификации);
- /login – страница входа;
- /register – страница регистрации.

В шапке приложения реализована адаптивная навигация с иконками для избранного и аккаунта, которая корректно работает на разных устройствах.

Такая организация маршрутизации обеспечивает:

- четкую структуру навигации по приложению;
- защиту контента, требующего аутентификации;
- плавные переходы между страницами;
- удобный доступ к основным функциям приложения;
- эффективное управление состоянием пользователя.

3.6.3 Взаимодействие с сервером

Взаимодействие с сервером осуществляется с помощью библиотеки `Axios`, которая позволяет выполнять HTTP-запросы к бэкенду, получая данные о фильмах, жанрах и рекомендациях. Обработка ответов от API включает в себя управление возможными ошибками и отображение пользователям соответствующих уведомлений. Это обеспечивает плавный и бесперебойный процесс получения данных, позволяя пользователям получать актуальную информацию без задержек.

Методы для взаимодействия с сервером делятся на GET, POST и DELETE. К основным GET-методам относятся:

- получение информации о фильме (`/api/movies/{id}`);
- получение списка фильмов (`/api/movies`);

- поиск фильмов (/api/movies/search);
- получение рекомендаций(/api/recommendations);
- получение похожих на выбранные пользователем фильмов (/api/recommendations/similar/{movie_id});
- получение избранных фильмов (/api/favorites).

POST-методы:

- регистрация пользователя (/api/auth/register);
- вход в систему (/api/auth/login);
- добавление в избранное (/api/favorites);
- подбор фильмов по параметрам (/api/movie-picker).

К DELETE-методам относятся:

- удаление из избранных (/api/favorites/{id});
- выход из системы (/api/auth/logout).

Функционал поиска и фильтрации также играет важную роль в приложении. Пользователи могут легко находить фильмы по названию, жанру или актеру, что значительно улучшает их опыт. Результаты поиска отображаются в реальном времени, что делает процесс поиска более интерактивным. Возможность фильтрации фильмов по различным критериям, таким как рейтинг или дата выхода, позволяет пользователям быстро находить нужный контент, учитывая их предпочтения и интересы.

Адаптивный дизайн приложения обеспечивает его корректное отображение на различных устройствах, включая мобильные телефоны и планшеты. Использование медиазапросов и гибкой верстки гарантирует, что интерфейс будет выглядеть привлекательно и оставаться функциональным на экранах разных размеров. Тестирование на различных устройствах и браузерах позволяет выявить и устранить возможные проблемы с совместимостью, что дополнительно улучшает пользовательский опыт.

В целом, блок клиентского приложения, реализованный на Vue.js, обеспечивает надежное и эффективное взаимодействие пользователей с системой. Он сочетает в себе интуитивный интерфейс, мощные функции поиска и фильтрации, а также надежное управление состоянием и взаимодействие с API, что делает его ключевым элементом успешного проекта.

На диаграмме последовательности ГУИР.400201.043 РР.2 представлено общее взаимодействие между основными компонентами системы, на диаграммах ГУИР.400201.043 РР.3 и ГУИР.400201.043 РР.4 наглядно показано взаимодействие пользователя с системой при выполнении основных операций, таких как регистрация и инициализация, а также добавление и получение избранных фильмов.

4 РАЗРАБОТКА ПРОГРАММНЫХ МОДУЛЕЙ

В данном разделе представлено несколько ключевых алгоритмов программного средства, а также приведено описание нескольких ключевых представлений, реализованных в рамках системы рекомендаций фильмов с использованием искусственного интеллекта.

4.1 Общая архитектура системы

Программное средство реализовано с использованием языка программирования Python и представляет собой современную рекомендательную систему фильмов. Система построена на основе гибридного подхода, сочетающего контентную фильтрацию и коллаборативную фильтрацию, что обеспечивает более точные и персонализированные рекомендации.

В качестве основного инструмента для построения рекомендательной системы использованы библиотеки:

- scikit-learn для реализации алгоритмов машинного обучения и обработки текстовых данных;
- pandas и numpy для эффективной обработки и анализа данных;
- NLTK для предобработки текстовых данных;
- Flask для реализации REST API серверной части.

Интерфейс пользователя реализован на фреймворке Vue.js, что обеспечивает современный и отзывчивый веб-интерфейс. Особое внимание уделено визуализации рекомендаций, где каждый фильм сопровождается постерным изображением для более привлекательного представления контента.

4.2 Ключевые компоненты

В программном средстве реализована модульная архитектура пользовательского интерфейса на основе компонентов Vue.js. Основные представления реализованы в виде компонентов Vue, которые могут быть переиспользованы в различных частях приложения.

4.2.1 Навигационная панель

Во многих представлениях существует необходимость в навигационной панели с основными элементами управления, при этом важно обеспечить единообразный внешний вид и функциональность на всех страницах приложения. Для этого используется стандартизированный хидер с адаптивной структурой и интерактивными элементами.

Исходный код представления:

```
<template>  
  <header>
```

```

<h1 @click="$router.push('/')">MovieAs</h1>
<button class="button-icon" @click="goToFavorites">
  <IconFavorite />
</button>
<button class="button-icon" @click="goToAccount">
  <IconUser />
</button>
</header>
<router-view />
</template>

```

Вычисляемое свойство `isAuthenticated` проверяет состояние авторизации пользователя через `localStorage`. Если пользователь авторизован, то значение свойства является истинным, в ином случае – ложным.

Метод `goToFavorites` используется для обработки перехода в раздел избранного. Если пользователь авторизован, происходит переход на страницу избранного, в противном случае отображается уведомление с просьбой войти в систему. Метод `goToAccount` управляет состоянием авторизации: если пользователь авторизован, происходит выход из системы, если нет – переход на страницу входа. Оба метода интегрированы с системой уведомлений, которая отображает соответствующие сообщения пользователю.

Адаптивность компонента обеспечивается через медиа-запросы, которые динамически изменяют размеры элементов и отступы в зависимости от размера экрана. При уменьшении ширины экрана до 768px и 480px происходит перераспределение пространства в сетке и уменьшение размеров кнопок, что обеспечивает удобство использования на мобильных устройствах.

4.2.2 Главная страница

Во многих представлениях существует необходимость в отображении основного контента приложения с различными интерактивными элементами и фильтрами. Для этого используется главная страница, которая объединяет в себе поисковую строку, карусель фильмов, кнопки управления и систему модальных окон.

Исходный код представления:

```

<template>
  <div class="card-main">
    <main class="home">
      <SearchBar :on-select="selectSearchedMovie" />
      <MovieCarousel @select-movie="openDetails" />
      <section class="buttons">
        <button @click="showPicker = true">Подобрать
фильм</button>
        <button @click="openMovieModal">Случайный
фильм</button>
      </section>
      <SimilarMoviePicker :on-recommend=
"fetchRecommendedMovies" />
    </main>
  </div>
</template>

```

```

    <MovieList
      v-if="recommendedMovies.length > 0"
      :movies="recommendedMovies"
      :favorites="favorites"
      @toggle-favorite="addToFavorites"
    />
  </main>
</div>
</template>

```

Представление использует компонентный подход, где каждый элемент интерфейса является отдельным компонентом. Основной контейнер `card-main` создает визуальное оформление страницы с помощью теней и скругленных углов. Внутри него располагаются все основные элементы: поисковая строка, карусель фильмов, кнопки управления и список рекомендованных фильмов.

Метод `fetchRecommendedMovies` используется для получения рекомендаций на основе выбранных фильмов. Он принимает массив идентификаторов фильмов и делает параллельные запросы к API для получения рекомендаций. Результаты объединяются, удаляются дубликаты, и для каждого фильма запрашивается детальная информация.

Метод `openMovieModal` реализует функционал случайного выбора фильма. Он запрашивает популярные фильмы с случайной страницы, выбирает один из них и загружает его детальную информацию, включая актеров и жанры. Результат отображается в модальном окне.

Система управления избранным реализована через метод `addToFavorites`, который проверяет авторизацию пользователя и управляет списком избранных фильмов в `localStorage`. При добавлении или удалении фильма из избранного отображаются соответствующие уведомления.

Адаптивность компонента обеспечивается через медиа-запросы, которые корректируют размеры элементов, отступы и расположение компонентов в зависимости от размера экрана. При уменьшении ширины экрана до 768px и 480px происходит изменение структуры сетки, размеров карточек фильмов и кнопок, что обеспечивает удобство использования на мобильных устройствах.

4.2.3 Компонент поисковой строки

Поисковая строка представляет собой интерактивный элемент интерфейса, который позволяет пользователям быстро находить фильмы и получать к ним доступ. Компонент реализует мгновенный поиск с отображением результатов в виде выпадающего списка, содержащего миниатюры постеров и основную информацию о фильмах.

Исходный код представления:

```
<template>
```

```

<div class="search-bar-wrapper">
  <div class="search-icon">
  </div>
  <input
    type="text"
    class="search-bar"
    placeholder="Поиск фильмов..."
    v-model="searchQuery"
    @input="onSearchInput"
  />
  <ul v-if="searchResults.length > 0" class="search-
suggestions">
    <li v-for="movie in searchResults" :key="movie.id"
@click="selectSearchedMovie(movie)">
      
      <div>
        <div>{{ movie.title }}</div>
        <div class="search-year">{{ new
Date(movie.release_date).getFullYear() }}</div>
      </div>
    </li>
  </ul>
</div>
</template>

```

Представление использует реактивную систему Vue.js для управления состоянием поиска. Компонент содержит поисковую строку с иконкой, которая визуально указывает на функциональность поиска. При вводе текста в поле поиска происходит мгновенный запрос к API через метод `onSearchInput`, который обновляет список результатов поиска.

Метод `selectSearchedMovie` обрабатывает выбор фильма из списка результатов. При выборе фильма происходит запрос детальной информации о фильме, включая актеров и жанры, после чего открывается модальное окно с подробной информацией.

Визуальное оформление компонента реализовано с помощью CSS, где `search-bar-wrapper` создает контейнер с полупрозрачным фоном и эффектом тени. При фокусе на поле поиска происходит анимация изменения тени, что создает эффект "поднятия" элемента. Список результатов поиска (`search-suggestions`) отображается в виде выпадающего меню с миниатюрами постеров и информацией о фильмах.

Адаптивность компонента обеспечивается через медиа-запросы, которые корректируют размеры элементов, отступы и шрифты в зависимости от размера экрана. При уменьшении ширины экрана до 768px и 480px происходит изменение размеров иконки поиска, поля ввода и списка результатов, что обеспечивает удобство использования на мобильных устройствах.

4.2.4 Компонент списка фильмов

Компонент списка фильмов представляет собой универсальный элемент интерфейса для отображения коллекции фильмов с возможностью добавления их в избранное. Компонент реализует сетку карточек фильмов с анимацией при наведении и интерактивными элементами управления.

Исходный код представления:

```
<template>
  <div v-for="movie in movies" :key="movie.id"
class="movie-card">
    <img
      @click="openDetails(movie)"/>
    <div class="movie-info">
      <h3>{{ movie.title }}</h3>
      <div class="movie-meta">
        <span class="year">{{ new
Date(movie.release_date).getFullYear() }}</span>
        <span class="rating">{{
movie.vote_average.toFixed(1) }}</span>
      </div>
      <button
        class="favorite-btn"
        @click="toggleFavorite(movie)"
        :class="{ 'is-favorite': isFavorite(movie) }">
      </button>
    </div>
  </div>
</template>
```

Каждый фильм представлен в виде карточки, содержащей постер, название, год выпуска, рейтинг и кнопку добавления в избранное. Компонент принимает массив фильмов через проп `movies` и массив избранных фильмов через проп `favorites`.

Метод `openDetails` обрабатывает клик по постеру фильма, открывая модальное окно с детальной информацией о фильме. Метод `toggleFavorite` управляет добавлением и удалением фильма из избранного, при этом обновляя состояние через событие `toggle-favorite`.

Визуальное оформление компонента реализовано с помощью CSS Grid (двумерная система сеток), что обеспечивает адаптивное расположение карточек фильмов.

Адаптивность компонента обеспечивается через медиа-запросы, которые корректируют размеры карточек, отступы и шрифты в зависимости от размера экрана.

4.2.5 Компонент карточки фильма

Компонент карточки фильма представляет собой универсальный элемент интерфейса для отображения информации об отдельном фильме. Карточка содержит постер фильма, основную информацию и интерактивные

элементы управления, обеспечивая быстрый доступ к деталям и управлению избранным.

Исходный код представления:

```
<template>
  <div class="movie-card">
    <div class="movie-poster" @click="openDetails">
      <img:src="movie.poster_path"                                ?
      `https://image.tmdb.org/t/p/w300${movie.poster_path}`      :
      'https://via.placeholder.com/300x450?text=No+Image' "
      :alt="movie.title"/>
      <span class="movie-rating">{{
movie.vote_average.toFixed(1) }}</span>
    </div>
    <div class="movie-info">
      <h3 class="movie-title">{{ movie.title }}</h3>
      <span class="movie-year">{{ new
Date(movie.release_date).getFullYear() }}</span>
      <span class="movie-genres">{{ movie.genres.join(',
') }}</span>
      <button class="favorite-btn" @click="toggleFavorite"
      :class="{ 'is-favorite': isFavorite }">
    </button>
    </div>
  </div>
</template>
```

Представление использует компонентный подход Vue.js для создания интерактивной карточки фильма. Компонент принимает объект `movie` через проп, содержащий всю необходимую информацию о фильме. При клике на постер происходит открытие модального окна с детальной информацией через метод `openDetails`.

Метод `toggleFavorite` управляет состоянием избранного для конкретного фильма. При клике на кнопку избранного происходит переключение состояния и отправка события родительскому компоненту для обновления общего списка избранных фильмов.

Визуальное оформление компонента реализовано с помощью CSS, где `movie-card` создает контейнер с эффектом наведения и тенью. Постер фильма занимает основную часть карточки, а поверх него отображается рейтинг фильма в виде оверлея. Информация о фильме располагается под постером в компактном виде, включая название, год выпуска и жанры.

Адаптивность компонента обеспечивается через медиа-запросы, которые корректируют размеры карточки, постера и текстовой информации в зависимости от размера экрана.

4.2.6 Компонент похожих фильмов

Компонент подбора похожих фильмов представляет собой интерактивный элемент интерфейса, позволяющий пользователям выбирать

фильмы для получения персонализированных рекомендаций. Компонент реализует систему выбора фильмов с возможностью добавления и удаления их из списка для подбора.

Исходный код представления:

```
<template>
  <div class="similar">
    <h2>Подберите похожие фильмы</h2>
    <div class="similar-cards">
      <div v-for="(movie, index) in selectedMovies"
:key="index" class="similar-card">
        
        <div class="movie-info">
          <h3>{{ movie.title }}</h3>
          <button
class="remove-btn"
@click="removeMovie(index)">Удалить</button></div></div>
      <div class="similar-card add-card" @click="showSearch
= true">+</div></div>
      <button class="select-movie" @click="getRecommendations"
:disabled="selectedMovies.length === 0"> похожие </button></div>
</template>
```

Компонент содержит сетку карточек выбранных фильмов и кнопку добавления новых фильмов. Каждая карточка отображает постер фильма, его название и кнопку удаления.

Метод `removeMovie` обрабатывает удаление фильма из списка выбранных. При клике на кнопку удаления фильм исключается из массива `selectedMovies`, что автоматически обновляет отображение списка.

Метод `getRecommendations` обрабатывает запрос на получение рекомендаций. При клике на кнопку "Подобрать похожие" происходит отправка выбранных фильмов родительскому компоненту через событие `on-recommend`, которое запускает процесс получения персонализированных рекомендаций.

Визуальное оформление компонента реализовано с помощью CSS Grid, что обеспечивает адаптивное расположение карточек фильмов. Каждая карточка имеет эффект наведения с плавным увеличением и тенью.

Адаптивность компонента обеспечивается через медиа-запросы, которые корректируют размеры карточек, отступы и шрифты в зависимости от размера экрана.

4.2.7 Компонент случайного фильма

Компонент модального окна случайного фильма представляет собой интерактивный элемент интерфейса, который отображает детальную информацию о случайно выбранном фильме. Компонент реализует модальное

окно с анимированным появлением и информативным представлением данных о фильме.

Исходный код представления:

```
<template>
  <div v-if="showModal" class="modal-overlay"
@click="closeModal">
    <div class="modal-content" @click.stop>
      <div class="modal-header">
        <h2>{{ selectedMovie.title }}</h2>
        <button class="close-btn"
@click="closeModal">×</button>
      </div>
      <div class="modal-body">
        <div class="movie-poster">
          
        </div>
        <div class="movie-info">
          <div class="info-row">
            <span class="label">Год выпуска:</span>
            <span class="value">{{ new
Date(selectedMovie.release_date).getFullYear() }}</span>
          </div>
          <div class="info-row">
            <span class="label">Жанры:</span>
            <span class="value">{{ selectedMovie.genres
}}</span>
          </div>
          <div class="info-row">
            <span class="label">Актеры:</span>
            <span class="value">{{ selectedMovie.actors
}}</span>
          </div>
          <div class="info-row">
            <span class="label">Описание:</span>
            <p class="description">{{
selectedMovie.description }}</p>
          </div>
        </div>
      </div>
    </div>
  </template>
```

Представление использует компонентный подход Vue.js для создания модального окна с информацией о фильме. Компонент принимает через пропсы `showModal` для управления видимостью и `selectedMovie` с информацией о фильме. Модальное окно содержит заголовок с названием фильма, постер и информационный блок с данными о фильме (год выпуска, жанры, актеры, описание).

Визуальное оформление компонента реализовано с помощью CSS, обеспечивая полупрозрачный оверлей, центрированное модальное окно с анимацией появления и адаптивную сетку для размещения контента.

Адаптивность компонента обеспечивается через медиа-запросы, корректирующие размеры и расположение элементов на разных устройствах. Компонент реализует обработку кликов вне модального окна для его закрытия.

4.2.8 Компонент страницы входа

Компонент формы авторизации представляет собой элемент пользовательского интерфейса, предназначенный для входа пользователя в систему. Он реализует форму с полями ввода логина и пароля, обработку событий входа и перенаправление пользователя на главную страницу после успешной аутентификации. Также компонент предоставляет возможность перехода к форме регистрации нового пользователя.

Исходный код представления:

```
<template>
  <div class="login-container">
    <div class="login-card">
      <h2>Вход<span</span></h2>
      <form @submit.prevent="login" class="login-form">
        <input
          v-model="username"
          type="text"
          placeholder="Логин" class="login-input" autofocus />
        <input
          v-model="password"
          type="password"
          placeholder="Пароль" class="login-input" />
        <div class="login-actions">
          <button
            type="submit"
            class="login-
            button">Войти</button>
          <button
            type="button"
            class="register-button"
            @click="goToRegister">Регистрация</button>
        </div>
      </form>
    </div>
  </div>
</template>
```

Компонент построен с использованием Composition API фреймворка Vue.js. В нем применяются реактивные переменные username и password, а также методы login и goToRegister для обработки событий входа и перехода на страницу регистрации. В случае незаполненных полей система выводит уведомление с помощью библиотеки vue-toastification.

Визуальное оформление реализовано средствами CSS с применением градиентного фона, стилизованной карточки формы и отзывчивых элементов управления. Кнопки обладают эффектами наведения и акцентными цветами, соответствующими фирменному стилю приложения. Компонент не только обеспечивает базовую авторизацию, но и взаимодействует с localStorage

для хранения состояния пользователя, создавая базовую инфраструктуру клиентской части системы аутентификации.

4.2.9 Компонент формы регистрации

Компонент формы регистрации представляет собой пользовательский интерфейс, позволяющий новым пользователям создать учетную запись в системе. Он реализует форму с полями для ввода логина, пароля и подтверждения пароля, а также обеспечивает проверку введенных данных и визуальное уведомление об успешной или неудачной регистрации.

Исходный код представления:

```
<template>
  <div class="register-container">
    <div class="register-card">
      <h2>Регистрация в 
```

Компонент построен на основе Composition API Vue.js и использует реактивные переменные username, password и confirmPassword. В методе register реализована базовая валидация данных: проверка на заполненность всех полей и совпадение паролей. Сообщения об ошибках и успешной регистрации отображаются с помощью уведомлений из библиотеки vue-toastification.

После успешной регистрации пользователь автоматически перенаправляется на страницу входа. Визуальное оформление формы выполнено в современном стиле: используется затемнённый фон, аккуратная карточка формы, плавные анимации и акцентные цвета, соответствующие фирменному стилю приложения MovieAs.

4.3 Ключевые алгоритмы

В разработанной системе рекомендаций фильмов применены ключевые алгоритмы, обеспечивающие точность и персонализацию выдачи результатов.

Основу составляют методы контентной и коллаборативной фильтрации, реализованные как отдельными модулями, так и в виде гибридного подхода. Контентная фильтрация анализирует характеристики фильмов (жанры, актерский состав, описание), чтобы предложить похожие по содержанию элементы. Коллаборативная фильтрация учитывает поведение пользователей — их оценки и выбор, формируя рекомендации на основе схожих вкусов других пользователей. Дополнительно реализованы алгоритмы динамического поиска, сортировки и фильтрации, которые обеспечивают быстрый отклик системы и удобство в работе с интерфейсом. Все алгоритмы оптимизированы для работы в реальном времени и устойчивы к изменениям пользовательских предпочтений.

4.3.1 Алгоритм преобработки данных

Целью данного алгоритма является подготовка и очистка данных о фильмах для последующего использования в системе рекомендаций.

Шаги алгоритма:

Шаг 1. Начало алгоритма.

Шаг 2. Загрузить данные из CSV файлов (фильмы и кредиты).

Шаг 3. Объединить данные по полю 'title'.

Шаг 4. Удалить неиспользуемые столбцы (homepage, tagline).

Шаг 5. Извлечь основные признаки фильмов.

Шаг 6. Применить функцию преобразования для поля `genres` в список жанров.

Шаг 7. Применить функцию преобразования для поля `keywords` в список ключевых слов.

Шаг 8. Применить функцию преобразования для поля `cast` для извлечения топ-актеров.

Шаг 9. Применить функцию преобразования для поля `crew` для извлечения режиссеров.

Шаг 10. Применить функцию преобразования для поля `production_companies` в список компаний.

Шаг 11. Удалить пробелы между словами в текстовых данных.

Шаг 12. Применить стемминг к текстовым данным.

Шаг 13. Удалить стоп-слова из текстовых данных.

Шаг 14. Удалить пунктуацию из текстовых данных.

Шаг 15. Конец алгоритма.

4.3.2 Алгоритм рекомендаций фильмов

Целью данного алгоритма является генерация рекомендаций фильмов на основе выбранных пользователем фильмов.

Шаги алгоритма:

Шаг 1. Начало алгоритма.

Шаг 2. Проверить, является ли входной параметр `movies` строкой.

Шаг 3. Если `movies` является строкой, преобразовать её в список с

одним элементом.

Шаг 4. Создать пустой список `all_recommendations` для хранения всех рекомендаций.

Шаг 5. Создать пустой список `all_similarities` для хранения оценок схожести.

Шаг 6. Для каждого фильма в списке `movies` выполнить шаги 7-9.

Шаг 7. Найти индекс фильма в базе данных по его названию.

Шаг 8. Загрузить матрицу схожести из файла по указанному пути.

Шаг 9. Получить оценки схожести для текущего фильма из матрицы.

Шаг 10. Отсортировать оценки схожести по убыванию.

Шаг 11. Выбрать топ-25 фильмов с наивысшими оценками схожести.

Шаг 12. Добавить выбранные фильмы и их оценки в список `all_similarities`.

Шаг 13. Если фильм не найден в базе данных, вывести сообщение об ошибке и продолжить со следующим фильмом.

Шаг 14. Если список `all_similarities` пуст, вернуть пустые списки рекомендаций и постеров.

Шаг 15. Создать словарь `combined_scores` для хранения объединенных оценок.

Шаг 16. Если индекс отсутствует в `combined_scores`, инициализировать его значением 0.

Шаг 17. Добавить оценку к существующему значению в `combined_scores`.

Шаг 18. Отсортировать `combined_scores` по убыванию оценок.

Шаг 19. Выбрать топ-25 фильмов с наивысшими объединенными оценками.

Шаг 20. Создать пустой список `rec_movie_list` для хранения названий рекомендованных фильмов.

Шаг 21. Создать пустой список `rec_poster_list` для хранения URL постеров.

Шаг 22. Получить название фильма по индексу из базы данных.

Шаг 23. Проверить, не является ли фильм одним из выбранных пользователем.

Шаг 24. Добавить название фильма в `rec_movie_list`.

Шаг 25. Получить URL постера фильма по его ID.

Шаг 26. Добавить URL постера в `rec_poster_list`.

Шаг 27. Вернуть списки рекомендованных фильмов и их постеров.

Шаг 28. Конец алгоритма.

4.3.3 Алгоритм создания матрицы схожести фильмов

Целью данного алгоритма является создание матрицы, отражающей степень схожести между фильмами на основе их признаков.

Шаги алгоритма:

Шаг 1. Начало алгоритма.

Шаг 2. Создать экземпляр `CountVectorizer` с параметром `max_features` равным 5000.

Шаг 3. Создать экземпляр `CountVectorizer` с параметром `stop_words` равным `'english'`.

Шаг 4. Получить первый фильм из базы данных.

Шаг 5. Создать пустую строку для хранения текстовых признаков.

Шаг 6. Получить жанры фильма.

Шаг 7. Добавить жанры в строку признаков.

Шаг 8. Получить ключевые слова фильма.

Шаг 9. Добавить ключевые слова в строку признаков.

Шаг 10. Получить имена актеров фильма.

Шаг 11. Добавить имена актеров в строку признаков.

Шаг 12. Получить имена режиссеров фильма.

Шаг 13. Добавить имена режиссеров в строку признаков.

Шаг 14. Получить названия компаний фильма.

Шаг 15. Добавить названия компаний в строку признаков.

Шаг 16. Сохранить строку признаков в массив признаков.

Шаг 17. Перейти к следующему фильму.

Шаг 18. Повторить шаги 5-17 для всех фильмов.

Шаг 19. Преобразовать массив признаков в числовые векторы с помощью `CountVectorizer`.

Шаг 20. Создать пустую матрицу размером $N \times N$, где N - количество фильмов.

Шаг 21. Вычислить косинусное сходство между первым вектором и всеми остальными.

Шаг 22. Заполнить первую строку матрицы полученными значениями.

Шаг 23. Перейти к следующему вектору.

Шаг 24. Повторить шаги 21-23 для всех векторов.

Шаг 25. Нормализовать значения в матрице в диапазон $[0, 1]$.

Шаг 26. Создать директорию для сохранения матрицы, если она не существует.

Шаг 27. Сохранить матрицу схожести в файл.

Шаг 28. Создать файл с метаданными.

Шаг 29. Записать количество фильмов в файл метаданных.

Шаг 30. Записать дату создания в файл метаданных.

Шаг 31. Записать тип признаков в файл метаданных.

Шаг 32. Записать параметры `CountVectorizer` в файл метаданных.

Шаг 33. Сохранить файл метаданных.

Шаг 34. Конец алгоритма.

4.3.4 Алгоритм анализа предпочтений пользователя

Целью данного алгоритма является анализ предпочтений пользователя на основе его избранных фильмов.

Шаги алгоритма:

- Шаг 1. Начало алгоритма.
- Шаг 2. Создать пустой словарь для хранения результатов анализа.
- Шаг 3. Получить список избранных фильмов пользователя из базы данных.
- Шаг 4. Создать пустой список для хранения всех жанров.
- Шаг 5. Получить список жанров из первого фильма в списке избранных.
- Шаг 6. Добавить жанры в общий список.
- Шаг 7. Перейти к следующему фильму в списке избранных.
- Шаг 8. Повторить шаги 5-7 для всех фильмов в списке.
- Шаг 9. Подсчитать частоту каждого жанра в общем списке.
- Шаг 10. Отсортировать жанры по убыванию частоты.
- Шаг 11. Сохранить результаты анализа жанров в словарь результатов.
- Шаг 12. Создать пустой список для хранения всех актеров.
- Шаг 13. Получить список актеров из первого фильма в списке избранных.
- Шаг 14. Добавить актеров в общий список.
- Шаг 15. Перейти к следующему фильму в списке избранных.
- Шаг 16. Повторить шаги 13-15 для всех фильмов в списке.
- Шаг 17. Подсчитать частоту каждого актера в общем списке.
- Шаг 18. Отсортировать актеров по убыванию частоты.
- Шаг 19. Выбрать топ-10 актеров с наивысшей частотой.
- Шаг 20. Сохранить результаты анализа актеров в словарь результатов.
- Шаг 21. Создать пустой список для хранения всех стран.
- Шаг 22. Получить список стран из первого фильма в списке избранных.
- Шаг 23. Добавить страны в общий список.
- Шаг 24. Перейти к следующему фильму в списке избранных.
- Шаг 25. Повторить шаги 22-24 для всех фильмов в списке.
- Шаг 26. Подсчитать частоту каждой страны в общем списке.
- Шаг 27. Отсортировать страны по убыванию частоты.
- Шаг 28. Сохранить результаты анализа стран в словарь результатов.
- Шаг 29. Создать пустой список для хранения годов выпуска.
- Шаг 30. Получить год выпуска из первого фильма в списке избранных.
- Шаг 31. Если дата выпуска существует, добавить год в список.
- Шаг 32. Перейти к следующему фильму в списке избранных.
- Шаг 33. Повторить шаги 30-32 для всех фильмов в списке.
- Шаг 34. Подсчитать частоту каждого года в списке.
- Шаг 35. Отсортировать годы по убыванию частоты.
- Шаг 36. Сохранить результаты анализа годов в словарь результатов.
- Шаг 37. Создать пустой словарь для хранения рекомендаций.
- Шаг 38. Получить список предпочитаемых жанров из результатов анализа (Шаг 11).
- Шаг 39. Получить список предпочитаемых актеров из результатов анализа (Шаг 20).
- Шаг 40. Сохранить рекомендации в словарь результатов.
- Шаг 41. Конец алгоритма.

4.3.5 Алгоритм фильтрации и поиска фильмов

Целью данного алгоритма является поиск и фильтрация фильмов по заданным критериям.

Шаги алгоритма:

Шаг 1. Начало алгоритма.

Шаг 2. Получить параметры фильтрации из запроса пользователя.

Шаг 3. Создать пустой набор фильмов для результатов поиска.

Шаг 4. Проверить наличие параметра поиска по названию.

Шаг 5. Получить первый фильм из базы данных.

Шаг 6. Проверить соответствие названия фильма поисковому запросу.

Шаг 7. Если название соответствует, добавить фильм в набор результатов.

Шаг 8. Перейти к следующему фильму.

Шаг 9. Повторить шаги 7-9 для всех фильмов.

Шаг 10. Проверить наличие параметра фильтрации по жанру.

Шаг 11. Получить первый фильм из текущего набора результатов.

Шаг 12. Проверить наличие указанного жанра в фильме.

Шаг 13. Если жанр отсутствует, удалить фильм из набора результатов.

Шаг 14. Перейти к следующему фильму.

Шаг 15. Повторить шаги 14-16 для всех фильмов.

Шаг 16. Проверить наличие параметра фильтрации по году выпуска.

Шаг 17. Если параметр года присутствует

Шаг 18. Получить первый фильм из текущего набора результатов.

Шаг 19. Проверить соответствие года выпуска фильма заданному диапазону.

Шаг 20. Если год не соответствует, удалить фильм из набора результатов.

Шаг 21. Перейти к следующему фильму.

Шаг 22. Повторить шаги 21-23 для всех фильмов.

Шаг 23. Проверить наличие параметра сортировки.

Шаг 24. Получить первый фильм из набора результатов.

Шаг 25. Извлечь значение параметра сортировки из фильма.

Шаг 26. Сохранить значение и индекс фильма.

Шаг 27. Перейти к следующему фильму.

Шаг 28. Повторить шаги 28-30 для всех фильмов.

Шаг 29. Отсортировать фильмы по сохраненным значениям.

Шаг 30. Проверить наличие параметра пагинации.

Шаг 31. Вычислить начальный индекс для текущей страницы.

Шаг 32. Вычислить конечный индекс для текущей страницы.

Шаг 33. Выбрать фильмы в заданном диапазоне индексов.

Шаг 34. Вернуть сформированный ответ.

Шаг 35. Конец алгоритма.

4.3.6 Алгоритм регистрации пользователя

Целью данного алгоритма является регистрация нового пользователя в

системе.

Шаги алгоритма:

Шаг 1. Начало алгоритма.

Шаг 2. Получить JSON-данные из запроса клиента.

Шаг 3. Извлечь из данных поля `username` и `password`.

Шаг 4. Проверить наличие значений в полях `username` и `password`.

Шаг 5. Если хотя бы одно из полей пустое, вернуть ошибку 400 "Missing username or password".

Шаг 6. Выполнить проверку наличия пользователя с таким же именем в базе данных.

Шаг 7. Если пользователь уже существует, вернуть ошибку 400 "Username already exists".

Шаг 8. Создать нового пользователя с переданным именем.

Шаг 9. Установить хэшированный пароль для пользователя.

Шаг 10. Добавить пользователя в текущую сессию базы данных.

Шаг 11. Сохранить изменения в базе данных.

Шаг 12. Вернуть успешный ответ 201 "User registered successfully".

Шаг 13. Конец алгоритма.

5 ПРОГРАММА И МЕТОДИКА ИСПЫТАНИЙ

В данном разделе представлена методика испытания программного средства. Проведено некоторое тестирование функционала и графического интерфейса.

5.1 Тесты графического интерфейса

Для проверки графического интерфейса приложения были использованы инструменты разработчика, встроенные в современные браузеры. Эти инструменты позволяют эмулировать различные условия отображения и взаимодействия, приближённые к реальному пользовательскому опыту.

В ходе тестирования были рассмотрены следующие параметры:

- изменение масштаба интерфейса и плотности пикселей;
- ориентация экрана (портретная и ландшафтная) – переключение между ориентациями позволило убедиться, что элементы интерфейса гибко перестраиваются и остаются функциональными в любом положении устройства;
- различные разрешения экрана, в том числе типичные размеры экранов смартфонов, планшетов и настольных компьютеров;
- кросс-браузерное тестирование, направленное на проверку единообразного отображения интерфейса в наиболее популярных браузерах (Google Chrome, Mozilla Firefox, Microsoft Edge, Safari).

Во всех случаях интерфейс отображался без визуальных искажений, все интерактивные элементы функционировали корректно.

Тестирование проводилось в адаптивном режиме браузера, что позволило в динамике отслеживать поведение интерфейса при смене разрешений и ориентаций.

5.1.1 Адаптивность интерфейса для разных устройств

Для обеспечения корректной работы приложения на различных устройствах было проведено тестирование адаптивности пользовательского интерфейса. Адаптивность интерфейса предполагает, что элементы страницы автоматически подстраиваются под размеры и ориентацию экрана, обеспечивая удобство взаимодействия независимо от устройства, на котором запускается приложение.

В ходе тестирования были использованы устройства с различными разрешениями экрана, такими как стандартный смартфон среднего размера и ноутбук.

На рисунке 5.1 показан внешний вид интерфейса на смартфоне с разрешением 425×920 пикселей. При таких параметрах система корректно перестраивает компоненты интерфейса: меню скрывается в выпадающее, карточки фильмов выстраиваются в одну колонку, а все интерактивные элементы сохраняют доступность и читаемость.

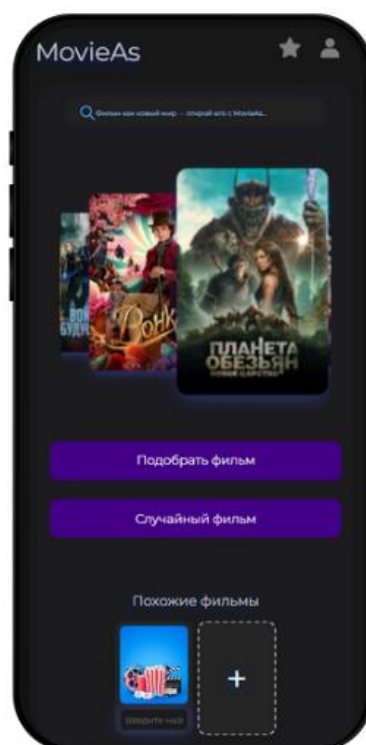


Рисунок 5.1 – Главная страница на смартфоне

На рисунке 5.2 представлен результат тестирования на ноутбуке с разрешением 1920×1080 пикселей. В этом случае интерфейс расширяется, позволяя отобразить больше элементов одновременно – например, карточки фильмов размещаются в несколько колонок, увеличивается область видимости контента, а навигационные элементы распределяются по экрану более свободно.

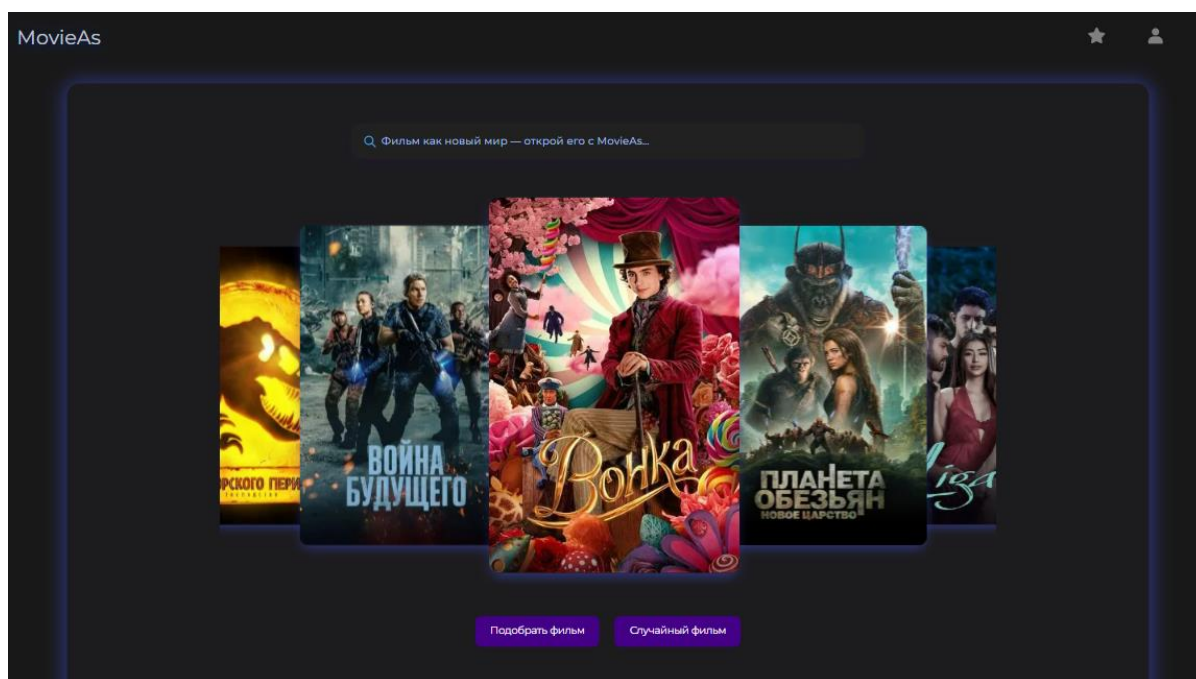


Рисунок 5.2 – Главная страница на ноутбуке

5.1.2 Масштаб интерфейса

В современных браузерах предусмотрены средства масштабирования, позволяющие пользователю изменять общий размер отображаемого контента. Эти настройки особенно важны для пользователей с нарушениями зрения или предпочтениями в отображении информации, поэтому интерфейс веб-приложения должен корректно адаптироваться к разным уровням масштабирования.

В рамках тестирования масштабирования были рассмотрены три уровня, охватывающие наиболее распространённые пользовательские сценарии:

- 75% (уменьшенный масштаб);
- 100% (стандартный масштаб, используемый по умолчанию);
- 125% (увеличенный масштаб).

При масштабировании до 75% элементы интерфейса сохраняли читаемость, текст не сливался, не уменьшался до критически мелких размеров, а структура страниц оставалась удобной для навигации. При стандартном масштабе в 100% все компоненты отображаются в соответствии с изначальным дизайном, без сбоев в разметке. При увеличении масштаба до 125% приложение также функционирует стабильно: тексты автоматически увеличиваются, при этом оставаясь в пределах своих контейнеров, не перекрывая другие элементы интерфейса.

Все основные элементы интерфейса масштабируются корректно, независимо от выбранного пользователем уровня отображения. Интерфейс остаётся адаптивным, логично организованным и визуально читаемым на всех проверенных уровнях масштабирования.

На рисунке 5.3 показано, что при масштабе интерфейса 75% информация отображается корректно, смещения элементов не наблюдается.

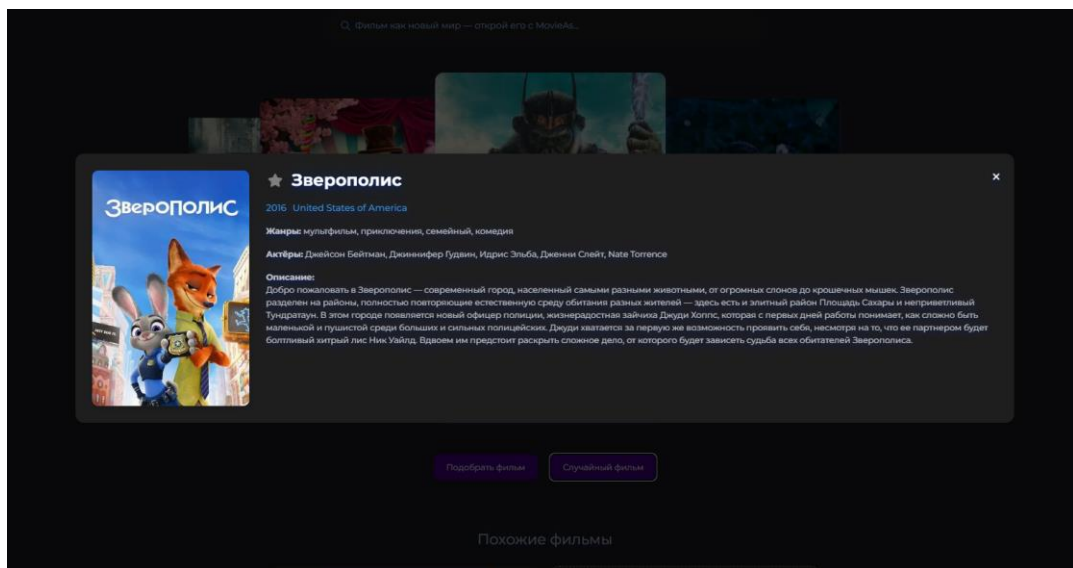


Рисунок 5.3 – Масштаб интерфейса 75%

На рисунке 5.4 при масштабе 100% интерфейс сохраняет корректное

отображение, все элементы расположены аккуратно и читаемо.

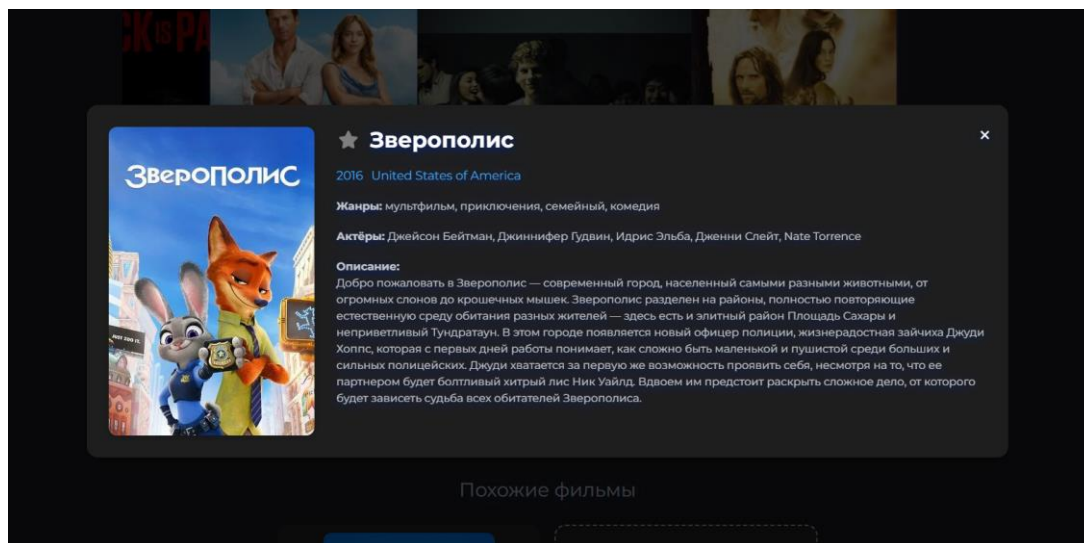


Рисунок 5.4 – Масштаб интерфейса 100%

На рисунке 5.5 при увеличении масштаба до 125% интерфейс также остается функциональным, без искажений и наложений элементов.

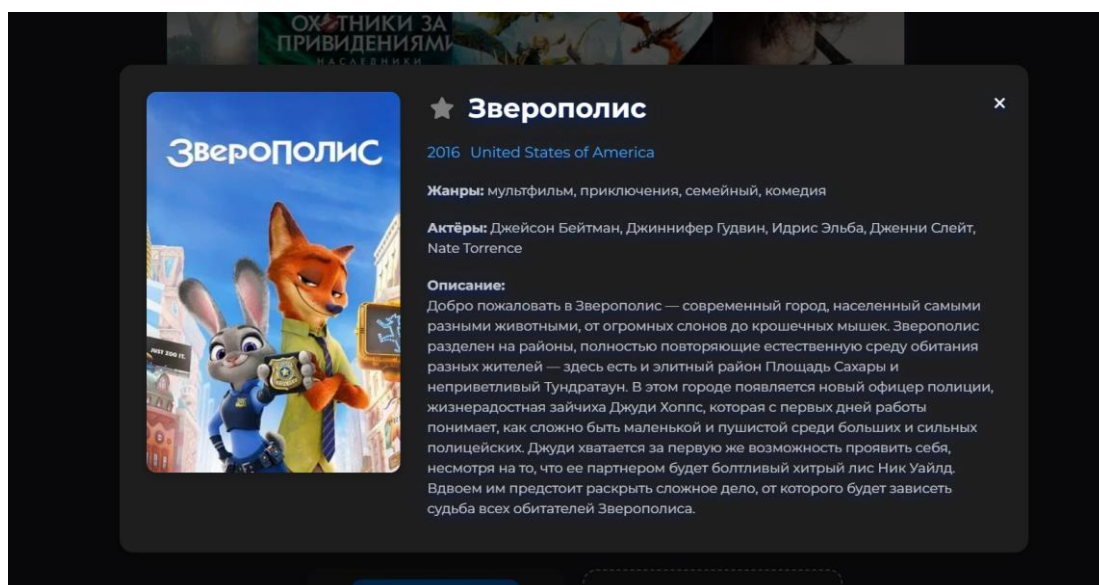


Рисунок 5.5 – Масштаб интерфейса 125%

5.1.3 Ориентация интерфейса

Интерфейс приложения должен корректно адаптироваться к различным ориентациям экрана, чтобы обеспечивать удобство использования на широком диапазоне устройств – от смартфонов до планшетов и ноутбуков. Особенно это важно при переходе между портретной (вертикальной) и ландшафтной (горизонтальной) ориентацией, когда пропорции экрана и доступная площадь изменяются.

Для проверки корректности адаптации интерфейса использовались инструменты разработчика браузера, которые позволяют вручную изменять размеры окна и его пропорции. Тестирование проводилось на всех ключевых представлениях приложения, включая главную страницу, формы подбора фильмов и окно с детальной информацией.

Результаты отображения страницы «Избранные» в вертикальном положении представлен на рисунке 5.6.

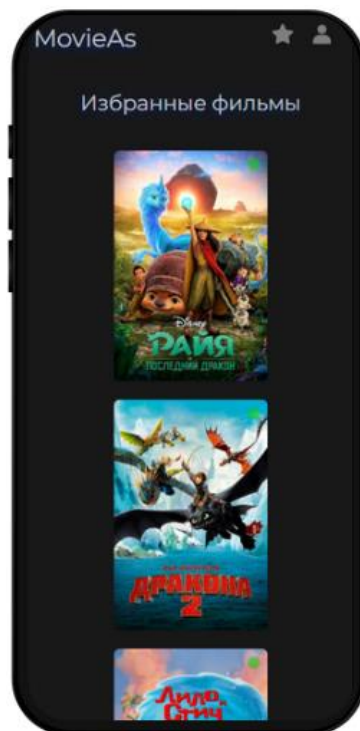


Рисунок 5.6 – Представление избранного при вертикальной ориентации

Проверка показала, что при переходе в ландшафтную ориентацию интерфейс перестраивается должным образом: количество карточек в ряду увеличивается, блоки выравниваются по ширине, и ни один элемент не выходит за пределы видимой области. Результат представлен на рисунке ниже:

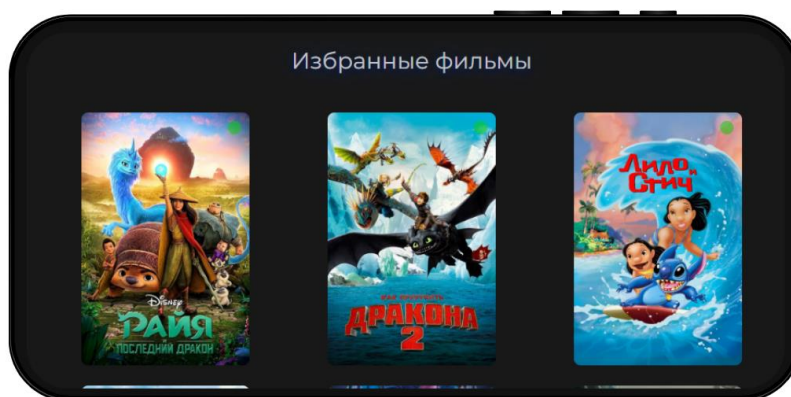


Рисунок 5.7 – Представление избранного при горизонтальной ориентации

5.1.4 Кросс-браузерность

Одной из важных задач при разработке веб-приложения является обеспечение кросс-браузерной совместимости, то есть корректной работы интерфейса и функционала в различных браузерах. Несмотря на то, что современные браузеры стремятся к унификации стандартов, существует множество нюансов в интерпретации HTML, CSS и JavaScript, особенно при работе с анимациями, формами, адаптивной версткой и компонентами пользовательского интерфейса.

В рамках проекта было проведено кросс-браузерное тестирование с использованием встроенных инструментов разработчика и ручной проверки. Основное внимание уделялось следующим популярным браузерам:

- Google Chrome (версия 124 и выше) – как один из наиболее распространённых браузеров с широкой поддержкой веб-стандартов;
- Mozilla Firefox – акцент на корректную работу динамических элементов и форм;
- Microsoft Edge – проверка отображения интерфейса и стилей, так как он использует движок Chromium, но имеет собственные особенности;
- Safari (в рамках доступного тестирования под macOS) – особое внимание уделялось шрифтам, анимациям и адаптивной верстке;
- Opera – дополнительная проверка совместимости и корректности отображения модальных окон.

Тестирование проводилось вручную, без использования автоматизированных систем, с фокусом на корректность отображения основных страниц и блоков: главной страницы, окна подбора фильмов, отображения карточек с результатами и подробной информацией о фильме, а также модальных окон. Особое внимание уделялось работе переходов, навигации и отображению избранных фильмов.

По результатам проверки было установлено, что интерфейс веб-приложения корректно отображается и функционирует во всех вышеуказанных браузерах. Не выявлено критичных различий в отображении: все основные элементы (текст, кнопки, изображения, карточки, формы) сохраняют структуру, стили и поведение, предусмотренное макетом. В некоторых случаях наблюдались незначительные различия в рендеринге шрифтов и сглаживании элементов, что обусловлено особенностями движков браузеров, однако это не повлияло на общее восприятие и пользовательский опыт.

5.2 Тесты функциональности

Тестирование функциональности проводится с целью выявления возможных ошибок в реализации основных возможностей веб-приложения, которые могут привести к некорректной обработке данных или сбоям в работе интерфейса.

Основное внимание уделяется следующим функциям:

- поиск фильмов по названию;

- подбор рекомендаций на основе выбранных фильмов;
- отображение похожих фильмов;
- добавление и удаление фильмов из избранного.

Поскольку приложение использует локальную базу данных, часть функционала может работать и без сетевого соединения, однако доступ к внешнему API (например, для получения обновлённой информации о фильмах) возможен только при наличии подключения к интернету.

Результаты тестирования приведены в таблице:

Таблица 5.1 – Результаты тестирования функциональности приложения

Тест	Ожидаемый результат	Полученный результат
Вход в приложение	Отображается главная страница	Отображается главная страница
Поиск фильма	Отображается список в соответствии с поисковым запросом	Отображается список в соответствии с поисковым запросом
Просмотр детальной информации о фильме	Отображается карточка с информацией	Отображается карточка с информацией
Подбор фильма	Отображается список фильмов по критериям	Отображается список фильмов по критериям
Случайный фильм	Отображается карточка со случайным фильмом	Отображается карточка со случайным фильмом
Похожие фильмы	Отображается 10 карточек с похожими фильмами	Отображается 10 карточек с похожими фильмами
Просмотр избранных	Отображается страница с избранными фильмами	Отображается страница с избранными фильмами

6 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

6.1 Системные требования

Для работы веб-приложения необходимо любое устройство с современным веб-браузером и подключением к сети интернет. Приложение корректно функционирует в последних версиях следующих браузеров:

- Google Chrome (рекомендуется версия 100 и выше);
- Mozilla Firefox (версия 100 и выше);
- Safari (на устройствах с iOS, iPadOS и macOS – версия 15 и выше);
- Microsoft Edge (версия 100 и выше).

Приложение адаптировано под работу на следующих типах устройств:

- настольные компьютеры и ноутбуки (Windows, macOS, Linux);
- планшеты (iPad, Android-планшеты);
- смартфоны (iPhone, Android).

Для полноценного функционирования, включая поиск фильмов, отображение рекомендаций и добавление в избранное, требуется активное подключение к интернету, так как данные загружаются с внешнего API. Без доступа к сети отдельные функции могут быть недоступны.

6.2 Доступ к приложению

После публикации приложения ссылка на его страницу будет размещена в репозитории проекта на GitHub [9].

Для запуска веб-приложения достаточно перейти по указанной ссылке в любом современном браузере – установка не требуется. При первом запуске потребуется активное подключение к интернету для загрузки всех необходимых данных.

6.3 Краткое руководство пользователя

При первом запуске веб-приложения пользователю отображается главная страница. Незарегистрированный пользователь имеет доступ к базовому функционалу, такому как просмотр информации о фильмах и поиск по каталогу.

После авторизации доступный функционал значительно расширяется: становится возможным добавление фильмов в избранное, получение персонализированных рекомендаций.

Полный перечень функциональных возможностей представлен в разделе ниже.

6.3.1 Подбор фильма

На главной странице веб-приложения размещена кнопка «Подобрать фильм», предназначенная для персонализированного поиска кино по заданным критериям. После нажатия на эту кнопку открывается модальное

окно, в котором реализован пошаговый алгоритм выбора параметров. Процесс подбора фильма состоит из следующих этапов:

1 Выбор жанра. На первом этапе пользователь выбирает интересующий его жанр фильма из предложенного списка. Это может быть, например, комедия, драма, фантастика, триллер и др. Жанровая фильтрация помогает сузить круг поиска в соответствии с предпочтениями пользователя.

2 Указание года или диапазона годов выпуска. На следующем этапе предлагается указать либо конкретный год выпуска фильма, либо задать диапазон лет (например, с 2010 по 2020 год). Это позволяет учитывать как современные, так и более старые фильмы, в зависимости от интереса пользователя.

3 Выбор страны производства. Третий шаг включает выбор страны производства. Данный критерий особенно полезен, если пользователь предпочитает фильмы конкретных кинематографий (например, американские, французские, корейские и т. д.).

После завершения всех трёх этапов и выбора параметров пользователь нажимает кнопку «Подтвердить». В результате происходит генерация результатов подбора – на экран выводится список фильмов, соответствующих указанным критериям.

Список фильмов (рисунок 6.1) представлен в виде галереи карточек, каждая из которых содержит основную информацию о фильме: название, постер, год выпуска. Такой формат позволяет быстро просматривать и сравнивать предложенные варианты.

При клике на одну из карточек открывается страница с подробной информацией о фильме. В этом окне пользователь может ознакомиться с более полной информацией, включая описание сюжета, список актёров, рейтинги, трейлер (при наличии), а также добавить фильм в избранное, чтобы вернуться к нему позже.

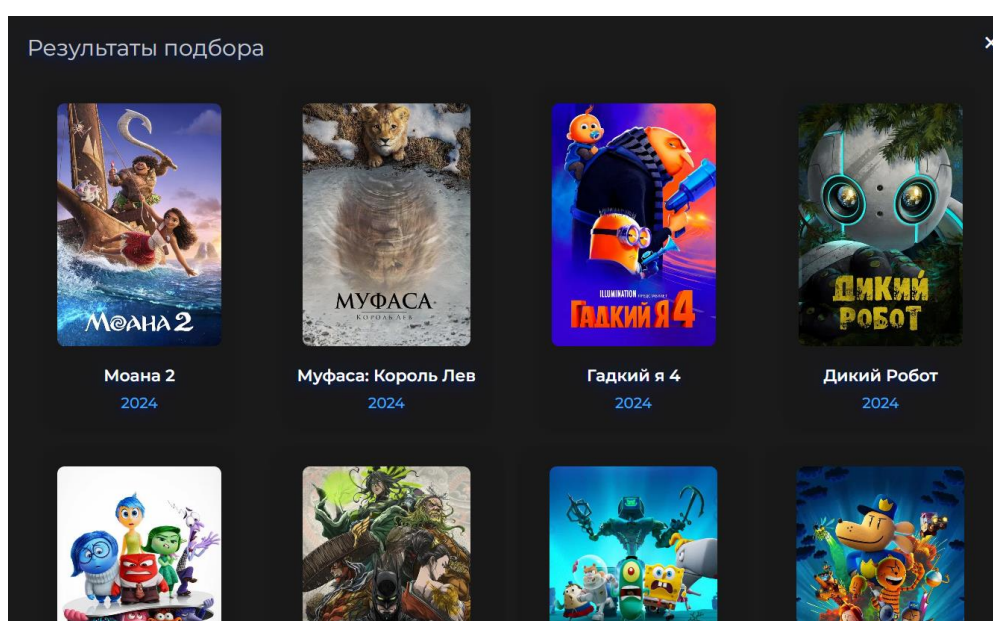


Рисунок 6.1 – Список подобранных фильмов

6.3.2 Похожие фильмы

На рисунке 6.2 представлена секция приложения, предназначенная для подбора похожих фильмов. Данная функция позволяет пользователю получить индивидуальные рекомендации на основе ранее просмотренных или понравившихся фильмов.

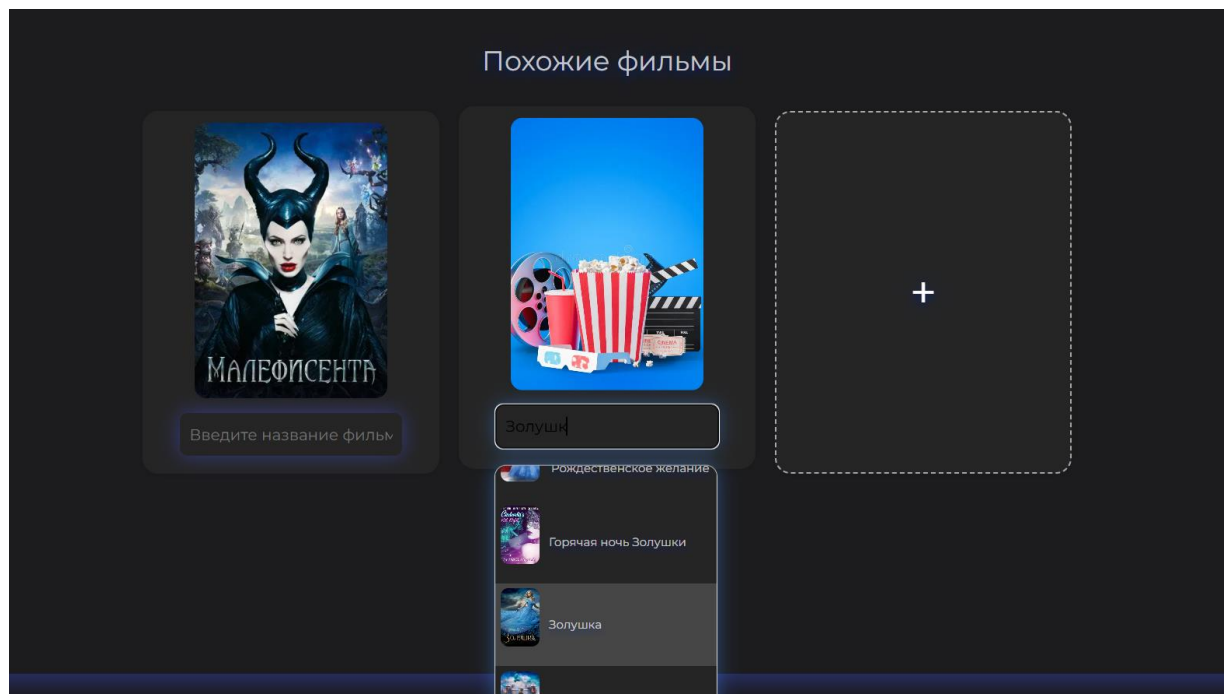


Рисунок 6.2 – Похожие фильмы

Для использования этой функции пользователь переходит в соответствующий раздел и может ввести от одного до трёх названий фильмов, которые ему интересны или запомнились. Это могут быть как известные блокбастеры, так и менее популярные, но значимые с точки зрения пользователя ленты. После нажатия на кнопку подтверждения, приложение инициирует алгоритм анализа и сопоставления фильмов, который использует множество критериев для подбора рекомендаций. Среди них – жанровая принадлежность, сюжетные особенности, стилистика повествования, атмосфера, актерский состав, работа режиссёра, а также такие численные параметры, как пользовательские рейтинги, оценки критиков, год выпуска, и другие мета-данные фильма. В совокупности эти характеристики формируют семантический и эстетический профиль фильма, на основе которого подбираются аналоги.

На основе этого анализа формируется список рекомендуемых фильмов, обладающих сходными признаками. Результатом работы алгоритма становится список рекомендуемых фильмов, обладающих высокой степенью сходства с исходными по ряду ключевых параметров. Данные результаты выводятся пользователю в наглядной форме – в виде карточек, содержащих подробную информацию. На рисунке 6.3 представлена визуализация полученного списка. Каждая карточка включает в себя следующие элементы:

- название фильма;
- постер или обложка;
- год выпуска;
- страна производства;
- длительность;
- жанры;
- основные актёры;
- режиссёры;
- средний рейтинг;
- краткое описание сюжета.



Рисунок 6.3 – Карточки с результатом рекомендаций

6.3.3 Поиск фильма

Одной из ключевых функций веб-приложения является поиск фильмов по названию, который позволяет пользователю быстро находить интересующий его фильм без необходимости вручную просматривать весь каталог. Поле ввода для поиска размещено в верхней части страницы, что делает его максимально доступным сразу после загрузки интерфейса. Оно сопровождается узнаваемой иконкой в виде лупы, что интуитивно подсказывает назначение элемента даже новым пользователям.

Поиск реализован в динамическом режиме, то есть результаты отображаются в реальном времени по мере ввода текста. Пользователь получает мгновенную обратную связь в виде выпадающего списка, содержащего релевантные совпадения — для каждого фильма отображается миниатюрная обложка, название и дата выхода (рисунок 6.4). Такой визуально-информативный подход позволяет быстро отличить нужный фильм даже при наличии похожих названий.

Механизм автоподсказок реализован с помощью асинхронных запросов к серверу, что обеспечивает быструю загрузку данных и снижение нагрузки на

сеть и ресурсы клиента. Обновление списка происходит плавно, без перезагрузки страницы, что значительно улучшает пользовательский опыт и делает взаимодействие с интерфейсом более приятным.

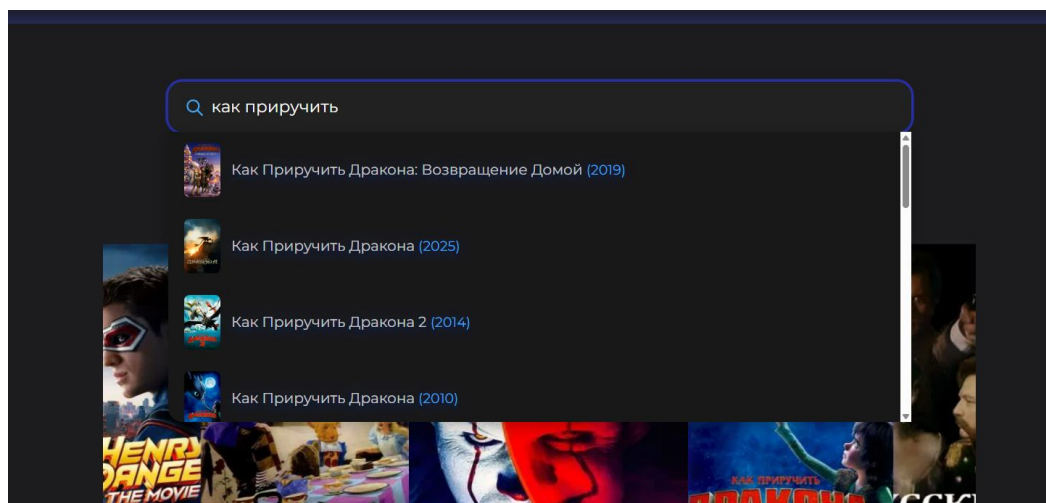


Рисунок 6.4 – Поле ввода

После выбора фильма пользователь переходит на отдельную страницу с подробной информацией (рисунок 6.5). Здесь представлены ключевые данные: постер, название, год выпуска, жанры, страна производства, продолжительность, актёры, режиссёр, рейтинг и краткое описание. На этой же странице доступна кнопка «Добавить в избранное», которая позволяет сохранить фильм в личной подборке. Эта возможность доступна только авторизованным пользователям, тогда как сам поиск можно использовать без входа в систему.

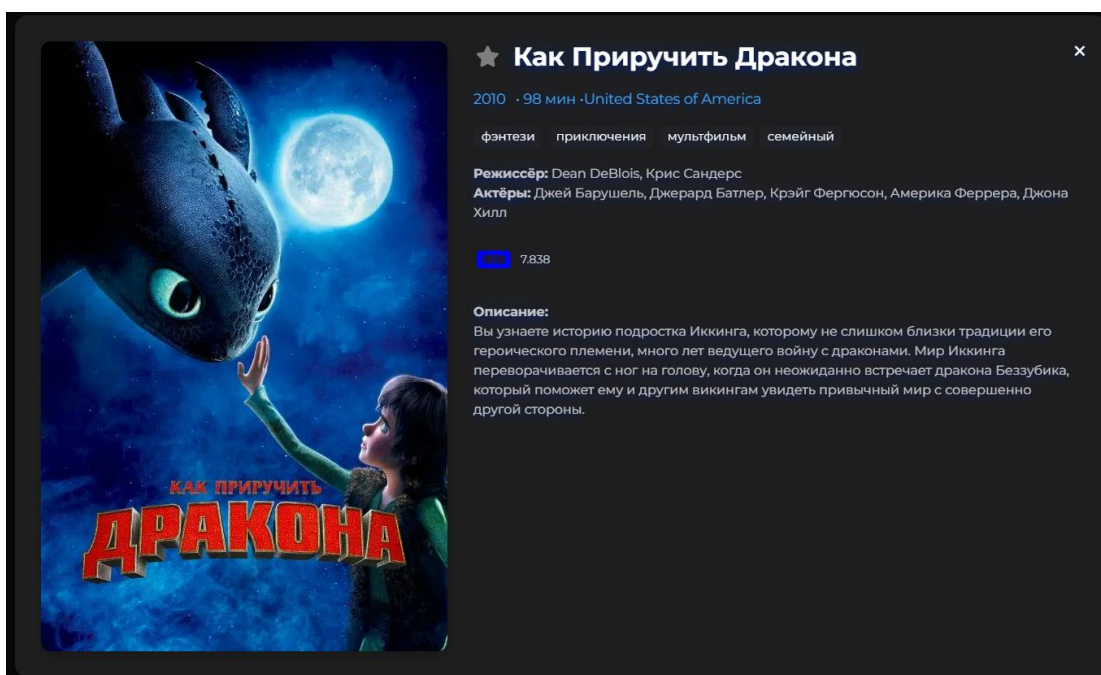


Рисунок 6.5 – Детальная информация о фильме

7 ТЕХНИКО-ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ И РЕАЛИЗАЦИИ НА РЫНКЕ ПРИЛОЖЕНИЯ «СЕРВИС РЕКОМЕНДАЦИЙ ФИЛЬМОВ С ИСПОЛЬЗОВАНИЕМ ИИ»

7.1 Характеристика программного средства, разрабатываемого для реализации на рынке

Целью разработки приложения является создание платформы, на которой пользователи смогут легко подобрать фильм исходя из собственных предпочтений. Приложение включает в себя несколько основных функций: подбор подходящей ленты на основе любимых фильмов и сохранение выбранных проектов в избранное.

Приложение будет иметь интуитивно понятный интерфейс, позволяющий пользователям быстро ориентироваться и находить нужные функции. Оно предоставит возможность поиска фильма по названию, после чего алгоритм будет осуществлять подбор схожих фильмов по жанрам, актерам, режиссерам и другим критериям, обеспечивая максимально точные рекомендации для пользователя.

Социальные функции позволят пользователям делиться фильмами и рекомендациями в социальных сетях, что обеспечит дополнительную рекламу проекта.

Веб-приложение будет доступно на различных устройствах, обеспечивая доступ к платформе из любого места через браузер. Безопасность данных пользователей будет обеспечена с помощью современных методов шифрования и аутентификации.

Целевая аудитория включает в себя людей возрастом от 12 до 60 лет в различных сферах деятельности. Существует достаточное количество рекомендательных платформ на рынке, однако данное приложение будет использовать искусственный интеллект для более точных прогнозов по предпочтениям различных пользователей, что делает его конкурентноспособным.

7.2 Расчёт инвестиций в разработку программного средства

7.2.1 Расчёт зарплат на основную заработную плату разработчиков

Расчёт затрат на основную заработную плату разработчиков производится исходя из количества людей, которые занимаются разработкой программного продукта, месячной зарплаты каждого участника процесса разработки и сложности выполняемой ими работы. Затраты на основную заработную плату рассчитаны по формуле:

$$Z_o = K_{\text{пр}} \sum_{i=1}^n Z_{\text{чи}} \cdot t_i, \quad (7.1)$$

где $K_{\text{пр}}$ – коэффициент премий и иных стимулирующих выплат;
 n – категории исполнителей, занятых разработкой программного средства;

$З_{\text{чи}}$ – часовая заработная плата исполнителя i -й категории, р;

t_i – трудоемкость работ, выполняемых исполнителем i -й категории, ч.

Разработкой всего приложения занимается инженер-программист, Обязанности тестирования приложения лежат на инженере-тестировщике. Задачами инженера-программиста, который занимается являются создание модели данных, графического интерфейса, связи между моделью данных и графическим интерфейсом. Инженер-тестировщик занимается выявлением неработоспособных частей приложения, а также оценивает пользовательский опыт, получаемый от использования приложения.

Месячная заработная плата основана на медианных показателях для Junior инженера-программиста за 2024 год по Республике Беларусь, которая составляет примерно 900 Долларов США в месяц, а для Junior инженера-тестировщика около 600 Долларов США. По состоянию на 5 марта 2025 года, 1 Доллар США по курсу Национального Банка Республики Беларусь составляет 3,2223 Белорусских рублей.

В перерасчёте на Белорусские рубли месячные оклады для инженера-программиста и инженера-тестировщика соответственно составляют 2 900,07 и 1 933,38 Белорусских рублей соответственно.

Часовой оклад исполнителей высчитывается путём деления их месячного оклада на количество рабочих часов в месяце, то есть 160 часов.

За количество рабочих часов в месяце для инженера-программиста и инженера-тестировщика принято соответственно 196 и 32 часа.

Коэффициент премии приравнивается к единице, так как она входит сумму заработной платы. Затраты на основную заработную плату:

Таблица 7.1 – Затраты на основную заработную плату

Категория исполнителя	Месячный оклад, р	Часовой оклад, р	Трудоемкость работ, ч	Итого, р
Инженер-программист	2 900,07	18,13	196	3 553,48
Инженер-тестировщик	1 933,38	12,08	32	386,56
Итого				3 940,04
Премия и иные стимулирующие выплаты (0%)				0
Всего затраты на основную заработную плату разработчиков				3 940,04

7.2.2 Расчёт затрат на дополнительную заработную плату разработчиков

Расчёт затрат на дополнительную заработную плату команды разработчиков рассчитывается по формуле:

$$З_{\text{д}} = \frac{З_{\text{о}} \cdot Н_{\text{д}}}{100}, \quad (7.2)$$

где N_d – норматив дополнительной заработной платы.
Значение норматива дополнительной заработной платы принимает за 10 %.

7.2.3 Расчёт отчислений на социальные нужды

Расчет отчислений на социальные нужды определяется согласно ставке отчислений, которая на апрель 2023 г. равняется 35%: 29% отчисляется на пенсионное страхование, 6% – на социальное страхование. Расчёт отчислений на социальные нужды вычисляется по формуле:

$$P_{\text{соц}} = \frac{(3_o + 3_d) \cdot N_{\text{соц}}}{100}, \quad (7.3)$$

где $N_{\text{соц}}$ – норматив отчислений в ФСЗН.

7.2.4 Расчёт прочих расходов

Расчёт затрат на прочие расходы определяется при помощи норматива прочих расчётов. Эта величина имеет значение 30%. Расчёт прочих расходов вычисляется по формуле:

$$P_{\text{пр}} = \frac{3_o \cdot N_{\text{пр}}}{100}, \quad (7.4)$$

где $N_{\text{пр}}$ – норматив прочих расходов.

7.2.5 Расчёт расходов на реализацию

Для того, чтобы рассчитать расходы на реализацию, необходимо знать норматив расходов на неё. Принимаем значение норматива равным 3%. Формула, которая использована для расчёта расходов на реализацию приведена ниже:

$$P_p = \frac{3_o \cdot N_p}{100}, \quad (7.5)$$

где N_p – норматив расходов на реализацию.

7.2.6 Расчёт общей суммы затрат на разработку и реализацию

Определяем общую сумму затрат на разработку и реализацию как сумму ранее вычисленных расходов: на основную заработную плату разработчиков, дополнительную заработную плату разработчиков, отчислений на социальные нужды, расходы на реализацию и прочие расходы. Значение определяется по формуле:

$$3_p = 3_o + 3_d + P_{\text{соц}} + P_{\text{пр}} + P_p \quad (7.6)$$

Таким образом, величина затрат на разработку программного средства высчитывается по указанной выше формуле и указана в таблице:

Таблица 7.2 – Затраты на разработку

Название статьи затрат	Формула/таблица для расчёта	Значение, р.
1 Основная заработная плата разработчиков	См. таблицу 7.1	3 940,04
2 Дополнительная заработная плата разработчиков	$З_d = \frac{3\,940,04 \cdot 10}{100}$	394,00
3 Отчисление на социальные нужды	$P_{\text{соц}} = \frac{(3\,940,04 + 394,00) \cdot 35}{100}$	1 516,91
4 Прочие расходы	$P_{\text{пр}} = \frac{3\,940,04 \cdot 30}{100}$	1 182,01
5 Расходы на реализацию	$P_p = \frac{3\,940,04 \cdot 3}{100}$	118,20
6 Общая сумма затрат на разработку и реализацию	$З_p = 3\,940,04 + 394,00 + 1\,516,91 + 1\,182,01 + 118,20$	7 151,16

7.3 Расчёт экономического эффекта от реализации программного средства на рынке

Для расчёта экономического эффекта организации-разработчика программного средства, а именно чистой прибыли, необходимо знать такие параметры как объем продаж, цену реализации и затраты на разработку.

Для оценки экономического эффекта программного продукта, предположим, что около 10 000 человек будут заинтересованы данным приложением. Из них 6 000 человек будут использовать его, а 2 500 приобретут расширенную версию. Цена на расширенную версию приложения составляет 4,99 долларов США. Таким образом, отпускная цена копии программного средства составляет 16,08 Белорусских рубля.

Для расчёта прироста чистой прибыли необходимо учесть налог на добавленную стоимость, который высчитывается по следующей формуле:

$$\text{НДС} = \frac{Ц_{\text{отп}} \cdot N \cdot Н_{\text{д.с}}}{100\% + Н_{\text{д.с}}}, \quad (7.7)$$

где N – количество копий(лицензий) программного продукта, реализуемое за год, шт.;

$Ц_{\text{отп}}$ – отпускная цена копии программного средства, р.;

$Н_{\text{д.с}}$ – ставка налога на добавленную стоимость, %.

Ставка налога на добавленную стоимость по состоянию на 5 марта 2025 года, в соответствии с действующим законодательством Республики Беларусь,

составляет 20%. Используя данное значение, посчитаем НДС:

$$\text{НДС} = \frac{16,08 \cdot 2\,500 \cdot 20\%}{100\% + 20\%}, = 6\,700,00 \text{ р.}$$

Посчитав налог на добавленную стоимость, можно рассчитать прирост чистой прибыли, которую получит разработчик от продажи программного продукта. Для этого используется формула:

$$\Delta\Pi_{\text{ч}}^{\text{р}} = (\text{Ц}_{\text{отп}} \cdot N - \text{НДС}) \cdot \text{Р}_{\text{пр}} \cdot \left(1 - \frac{\text{Н}_{\text{п}}}{100}\right), \quad (7.8)$$

где N – количество копий(лицензий) программного продукта, реализуемое за год, шт.;

$\text{Ц}_{\text{отп}}$ – отпускная цена копии программного средства, р.;

НДС – сумма налога на добавленную стоимость, р.;

$\text{Н}_{\text{п}}$ – ставка налога на прибыль, %;

$\text{Р}_{\text{пр}}$ – рентабельность продаж копий.

Ставка налога на прибыль, согласно действующему законодательству, по состоянию на 5.03.2025 равна 20%. Рентабельность продаж копий взята в размере 40%. Зная ставку налога и рентабельность продаж копий (лицензий), рассчитывается прирост чистой прибыли для разработчика:

$$\Delta\Pi_{\text{ч}}^{\text{р}} = (16,08 \cdot 2\,500 - 6\,700) \cdot 40\% \cdot \left(1 - \frac{20}{100}\right) = 10\,720,00$$

7.4 Расчёт показателей экономической эффективности разработки и реализации программного средства на рынке

Для того, чтобы оценить экономическую эффективность разработки и реализации программного средства на рынке, необходимо рассмотреть результат сравнения затрат на разработку данного программного продукта, а также полученный прирост чистой прибыли за год.

Сумма затрат на разработку меньше суммы годового экономического эффекта, поэтому можно сделать вывод, что такие инвестиции окупятся менее, чем за один год.

Таким образом, оценка экономической эффективности инвестиций производится при помощи расчёта рентабельности инвестиций (Return on Investment, ROI). Эта метрика позволяет понять, насколько выгодны вложения в разработку программного продукта и помогает принимать обоснованные решения о дальнейших инвестициях. Формула для расчёта ROI:

$$\text{ROI} = \frac{\Delta\Pi_{\text{ч}}^{\text{р}} - \text{З}_{\text{р}}}{\text{З}_{\text{р}}} \cdot 100\% \quad (7.9)$$

где $\Delta\P_{\text{ч}}^p$ – прирост чистой прибыли, полученной от реализации программного средства на рынке информационных технологий, р.;

Z_p – затраты на разработку и реализацию программного средства, р.

$$ROI = \frac{10\,720,00 - 7\,151,16}{7\,151,16} \cdot 100\% = 49,91\%$$

7.5 Вывод об экономической целесообразности реализации проектного решения

Проведённые расчёты технико-экономического обоснования позволяют сделать предварительный вывод о целесообразности разработки данного программного продукта. Общая сумма затрат на разработку и реализацию составила 7 151,16 Белорусских рублей, а отпускная цена была установлена на уровне 16,08 Белорусских рублей.

Прирост чистой прибыли за год, исходя из предполагаемого объёма продаж в размере 2500 расширенных версий в год, составляет 10 720,00 Белорусских рублей. Рентабельность инвестиций за год составляет 49,91%.

Это означает, что разработка данного программного продукта является целесообразной и реализация программного средства по установленной цене имеет смысл.

Однако, следует учитывать возможные риски, связанные с конкуренцией со стороны аналогов, что может привести к незамеченности продукта на рынке. Кроме того, высокая рентабельность связана с рисками, и расчётные результаты были получены при предполагаемом объёме продаж в 2500 копий в год.

Тем не менее, при поддержке проект может получить долгосрочное и успешное развитие, и количество проданных копий может превысить предполагаемое количество.

ЗАКЛЮЧЕНИЕ

В ходе выполнения дипломного проекта была успешно завершена разработка интеллектуального сервиса рекомендаций фильмов, основанного на современных подходах в области анализа данных и пользовательских предпочтений. На всех этапах – от постановки цели до реализации и тестирования системы – велась последовательная работа, направленная на создание эффективного, удобного и функционального программного продукта, способного удовлетворить потребности конечных пользователей.

Основной целью проекта являлось создание системы, способной не только предоставлять пользователю возможность поиска нужного фильма, но и формировать персонализированные рекомендации, опираясь на его интересы и поведенческую модель. Достигнутые результаты полностью соответствуют заявленным задачам: реализован сервис, сочетающий в себе удобный интерфейс, высокую производительность, устойчивость к нагрузкам и точность выдачи рекомендаций. Пользователь может легко взаимодействовать с системой, благодаря интуитивно понятному дизайну, поддержке различных масштабов отображения и динамическим элементам интерфейса.

Также в рамках проекта была проведена проработка архитектуры системы, разработаны диаграммы, отражающие её функциональную и логическую структуру, что способствовало ясному пониманию всех взаимосвязей между модулями. Особое внимание было уделено вопросам масштабируемости и адаптивности интерфейса, что обеспечивает корректное отображение информации при изменении параметров отображения на различных устройствах и экранах.

Отдельным этапом стало оформление экономического обоснования проекта, в котором проведена оценка затрат и расчет эффективности реализации системы. Это позволило не только обосновать целесообразность создания подобного программного решения, но и продемонстрировать его потенциальную устойчивость и востребованность в условиях рынка.

В целом, дипломный проект выполнен в полном объеме. Разработанная система полностью готова к использованию и может быть применена как в индивидуальных, так и в коммерческих целях. В ходе работы были приобретены важные навыки в сфере проектирования, программирования, работы с базами данных, построения пользовательского интерфейса, а также анализа и обработки пользовательских данных. Данный проект послужил не только результатом теоретических и практических знаний, полученных в процессе обучения, но и стал отправной точкой для дальнейшего профессионального роста и развития в сфере информационных технологий.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Medium [Электронный ресурс]. – The Movie Database (TMDb). – Режим доступа: <https://medium.com/@aalboughbar/the-movie-database-tmdb-a118f319ce10> – Дата доступа: 20.02.2025
- [2] TMDb [Электронный ресурс]. – Getting Started. – Режим доступа: <https://developer.themoviedb.org/reference/intro/getting-started> – Дата доступа: 20.02.2025
- [3] Хабр [Электронный ресурс]. – Коллаборативная фильтрация. – Режим доступа: <https://habr.com/ru/articles/150399/> – Дата доступа: 21.02.2025
- [4] Образование [Электронный ресурс]. – Матричная факторизация. – Режим доступа: <https://education.yandex.ru/handbook/ml/article/matrichnaya-faktorizaciya> – Дата доступа: 21.02.2025
- [5] The Clever Programmer [Электронный ресурс]. – Content-Based Filtering in Machine Learning. – Режим доступа: <https://thecleverprogrammer.com/2021/02/10/content-based-filtering-in-machine-learning/> – Дата доступа: 21.02.2025
- [6] Хабр [Электронный ресурс]. – Глубокое погружение в рекомендательную систему Netflix. – Режим доступа: <https://habr.com/ru/articles/677396/> – Дата доступа: 21.02.2025
- [7] TexTerra [Электронный ресурс]. – Рекомендации Кинопоиска. – Режим доступа: <https://texterra.ru/blog/rekomendatelnye-algoritmy-kinopoiska.html?ysclid=m8h8wk867w395375353> – Дата доступа: 15.02.2025
- [8] MovieTop [Электронный ресурс]. – Сервис быстрого подбора фильмов и сериалов. – Режим доступа: <https://movietop.org/about> – Дата доступа: 15.02.2025
- [9] WEB-сервис GitHub [Электронный ресурс]. – DIPLOMA. – Режим доступа: <https://github.com/Polkob/DIPLOMA> – Дата доступа: 15.05.2025

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг кода

Содержимое файла `app/app.py`:

```
1. from flask import Flask
2. from flask_sqlalchemy import SQLAlchemy
3. from flask_migrate import Migrate
4. from flask_restful import Api
5. from app.resources import UserListResource, UserResource,
MovieListResource, MovieResource, \
6. FavoriteListResource, GenreListResource,
ActorListResource, DirectorListResource, CountryListResource, \
7. MovieGenreResource, MovieActorResource,
MovieDirectorResource, MovieCountryResource, FavoritesResource,
SimilarMoviesResource
8.
9. app = Flask(__name__)
10. app.config.from_object('config.Config')
11. db = SQLAlchemy(app)
12. migrate = Migrate(app, db)
13. api = Api(app)
14.
15. api.add_resource(UserListResource, '/users')
16. api.add_resource(UserResource, '/users/<int:user_id>')
17. api.add_resource(MovieListResource, '/movies')
18. api.add_resource(MovieResource,
'/movies/<int:movie_id>')
19. api.add_resource(FavoriteListResource,
'/favorites/<int:user_id>')
20. api.add_resource(GenreListResource, '/genres')
21. api.add_resource(ActorListResource, '/actors')
22. api.add_resource(DirectorListResource, '/directors')
23. api.add_resource(CountryListResource, '/countries')
24. api.add_resource(MovieGenreResource,
'/movies/<int:movie_id>/genres')
25. api.add_resource(MovieActorResource,
'/movies/<int:movie_id>/actors')
26. api.add_resource(MovieDirectorResource,
'/movies/<int:movie_id>/directors')
27. api.add_resource(MovieCountryResource,
'/movies/<int:movie_id>/countries')
28. api.add_resource(FavoritesResource,
'/favorites/<int:user_id>')
29. api.add_resource(SimilarMoviesResource,
'/similars/<int:user_id>')
30.
31. if __name__ == '__main__':
32.     app.run(debug=True)
```

Содержимое файла app/__init__.py:

```
1. from flask import Flask
2. from flask_sqlalchemy import SQLAlchemy
3. from flask_restful import Api
4. from flask_migrate import Migrate
5. from flask_login import LoginManager
6. import sys
7. import os
8.
9.
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(
__file__))))
10.
11. from config import Config
12.
13. db = SQLAlchemy()
14. migrate = Migrate()
15. login_manager = LoginManager()
16.
17. def create_app(config_class=Config):
18.     app = Flask(__name__)
19.     app.config.from_object(config_class)
20.     app.config['SECRET_KEY'] = 'your-secret-key'
21.
22.     db.init_app(app)
23.     migrate.init_app(app, db)
24.     login_manager.init_app(app)
25.     login_manager.login_view = 'auth.login'
26.
27.     @login_manager.user_loader
28.     def load_user(user_id):
29.         from app.models import User
30.         return User.query.get(int(user_id))
31.
32.     from app.resources import UserListResource,
UserResource, MovieListResource, MovieResource
33.     api = Api(app)
34.     api.add_resource(UserListResource, '/users')
35.     api.add_resource(UserResource,
'/users/<int:user_id>')
36.     api.add_resource(MovieListResource, '/movies')
37.     api.add_resource(MovieResource,
'/movies/<int:movie_id>')
38.
39.     with app.app_context():
40.         from app.routes.auth import bp as auth_bp
41.         app.register_blueprint(auth_bp,
url_prefix='/api/auth')
42.
43.         from app.routes.recommendations import bp as
recommendations_bp, init_app as init_recommender
44.         app.register_blueprint(recommendations_bp,
```

```

url_prefix='/api/recommendations')
45.         init_recommender(app)
46.
47.         from app.routes.movie_picker import bp as
movie_picker_bp
48.         app.register_blueprint(movie_picker_bp,
url_prefix='/api/picker')
49.
50.         @app.route('/health')
51.         def health_check():
52.             return {'status': 'ok'}, 200
53.
54.         return app
55.
56. from app import models

```

Содержимое файла app/models.py:

```

1. from sqlalchemy.dialects.postgresql import JSONB
2. import json
3. from . import db
4. from flask_login import UserMixin
5. from werkzeug.security import generate_password_hash,
check_password_hash
6.
7. class User(UserMixin, db.Model):
8.     __tablename__ = 'users'
9.     id = db.Column(db.Integer, primary_key=True)
10.    username = db.Column(db.String(50), unique=True,
nullable=False)
11.    password_hash = db.Column(db.String(255))
12.    email = db.Column(db.String(100))
13.    created_at = db.Column(db.DateTime,
default=db.func.current_timestamp())
14.
15.    favorites = db.relationship('Favorite',
backref='user', lazy=True)
16.    search_history = db.relationship('SearchHistory',
backref='user', lazy=True)
17.    recommendation_logs =
db.relationship('RecommendationLog', backref='user', lazy=True)
18.
19.    def get_id(self):
20.        return str(self.id)
21.
22.    @property
23.    def is_active(self):
24.        return True
25.
26.    @property
27.    def is_authenticated(self):
28.        return True
29.

```

```

30.     @property
31.     def is_anonymous(self):
32.         return False
33.
34.     def set_password(self, password):
35.         self.password_hash =
generate_password_hash(password)
36.
37.     def check_password(self, password):
38.         return check_password_hash(self.password_hash,
password)
39.
40. class Movie(db.Model):
41.     __tablename__ = 'movies'
42.     id = db.Column(db.Integer, primary_key=True)
43.     tmdb_id = db.Column(db.Integer, unique=True,
nullable=False)
44.     title = db.Column(db.String(255), nullable=False)
45.     overview = db.Column(db.Text)
46.     poster_url = db.Column(db.String(255))
47.     release_date = db.Column(db.Date)
48.     rating = db.Column(db.Float)
49.
50.     genres = db.relationship('Genre',
secondary='movies_genres', back_populates="movies")
51.     actors = db.relationship('Actor',
secondary='movies_actors', back_populates="movies")
52.     directors = db.relationship('Director',
secondary='movies_directors', back_populates="movies")
53.     countries = db.relationship('Country',
secondary='movies_countries', back_populates="movies")
54.
55.     favorites = db.relationship('Favorite',
backref='movie', lazy=True)
56.     recommendation_logs =
db.relationship('RecommendationLog', backref='movie', lazy=True)
57.
58. class Genre(db.Model):
59.     __tablename__ = 'genres'
60.     id = db.Column(db.Integer, primary_key=True)
61.     name = db.Column(db.String(50), nullable=False,
unique=True)
62.
63.     movies = db.relationship('Movie',
secondary='movies_genres', back_populates="genres")
64.
65. class Actor(db.Model):
66.     __tablename__ = 'actors'
67.     id = db.Column(db.Integer, primary_key=True)
68.     name = db.Column(db.String(100), nullable=False)
69.
70.     movies = db.relationship('Movie',
secondary='movies_actors', back_populates="actors")

```

```

71.
72. class Director(db.Model):
73.     __tablename__ = 'directors'
74.     id = db.Column(db.Integer, primary_key=True)
75.     name = db.Column(db.String(100), nullable=False)
76.
77.     movies = db.relationship('Movie',
secondary='movies_directors', back_populates="directors")
78.
79. class Country(db.Model):
80.     __tablename__ = 'countries'
81.     id = db.Column(db.Integer, primary_key=True)
82.     name = db.Column(db.String(100), nullable=False,
unique=True)
83.
84.     movies = db.relationship('Movie',
secondary='movies_countries', back_populates="countries")
85.
86. class MovieGenre(db.Model):
87.     __tablename__ = 'movies_genres'
88.     movie_id = db.Column(db.Integer,
db.ForeignKey('movies.id'), primary_key=True)
89.     genre_id = db.Column(db.Integer,
db.ForeignKey('genres.id'), primary_key=True)
90.
91. class MovieActor(db.Model):
92.     __tablename__ = 'movies_actors'
93.     movie_id = db.Column(db.Integer,
db.ForeignKey('movies.id'), primary_key=True)
94.     actor_id = db.Column(db.Integer,
db.ForeignKey('actors.id'), primary_key=True)
95.
96. class MovieDirector(db.Model):
97.     __tablename__ = 'movies_directors'
98.     movie_id = db.Column(db.Integer,
db.ForeignKey('movies.id'), primary_key=True)
99.     director_id = db.Column(db.Integer,
db.ForeignKey('directors.id'), primary_key=True)
100.
101. class MovieCountry(db.Model):
102.     __tablename__ = 'movies_countries'
103.     movie_id = db.Column(db.Integer,
db.ForeignKey('movies.id'), primary_key=True)
104.     country_id = db.Column(db.Integer,
db.ForeignKey('countries.id'), primary_key=True)
105.
106. class Favorite(db.Model):
107.     __tablename__ = 'favorites'
108.     id = db.Column(db.Integer, primary_key=True)
109.     user_id = db.Column(db.Integer,
db.ForeignKey('users.id'), nullable=False)
110.     movie_id = db.Column(db.Integer,
db.ForeignKey('movies.id'), nullable=False)

```

```

111.         added_at = db.Column(db.DateTime,
default=db.func.current_timestamp())
112.
113. class SearchHistory(db.Model):
114.     __tablename__ = 'search_history'
115.     id = db.Column(db.Integer, primary_key=True)
116.     user_id = db.Column(db.Integer,
db.ForeignKey('users.id'), nullable=False)
117.     query = db.Column(db.Text)
118.     date = db.Column(db.DateTime,
default=db.func.current_timestamp())
119.
120. class RecommendationLog(db.Model):
121.     __tablename__ = 'recommendation_logs'
122.     id = db.Column(db.Integer, primary_key=True)
123.     user_id = db.Column(db.Integer,
db.ForeignKey('users.id'), nullable=False)
124.     movie_id = db.Column(db.Integer,
db.ForeignKey('movies.id'), nullable=False)
125.     based_on_movies = db.Column(JSONB)
126.     recommended_ids = db.Column(JSONB)
127.     created_at = db.Column(db.DateTime,
default=db.func.current_timestamp())

```

Содержимое файла app/routes/auth.py:

```

1. from flask import Blueprint, jsonify, request
2. from flask_login import login_user, logout_user,
login_required, current_user
3. from app.models import User
4. from app import db
5.
6. bp = Blueprint('auth', __name__)
7.
8. @bp.route('/register', methods=['POST'])
9. def register():
10.     data = request.get_json()
11.     username = data.get('username')
12.     password = data.get('password')
13.
14.     if not username or not password:
15.         return jsonify({'error': 'Missing username or
password'}), 400
16.
17.     if User.query.filter_by(username=username).first():
18.         return jsonify({'error': 'Username already
exists'}), 400
19.
20.     user = User(username=username)
21.     user.set_password(password)
22.     db.session.add(user)
23.     db.session.commit()
24.

```

```

25.         return jsonify({'message': 'User registered
successfully'}), 201
26.
27. @bp.route('/login', methods=['POST'])
28. def login():
29.     data = request.get_json()
30.     username = data.get('username')
31.     password = data.get('password')
32.
33.     if not username or not password:
34.         return jsonify({'error': 'Missing username or
password'}), 400
35.
36.     user =
User.query.filter_by(username=username).first()
37.     if user is None or not
user.check_password(password):
38.         return jsonify({'error': 'Invalid username or
password'}), 401
39.
40.     login_user(user)
41.     return jsonify({'message': 'Logged in
successfully'}), 200
42.
43. @bp.route('/logout', methods=['POST'])
44. @login_required
45. def logout():
46.     logout_user()
47.     return jsonify({'message': 'Logged out
successfully'}), 200
48.
49. @bp.route('/me', methods=['GET'])
50. @login_required
51. def get_current_user():
52.     return jsonify({
53.         'id': current_user.id,
54.         'username': current_user.username
55.     }), 200

```

Содержимое файла app/routes/favorites.py:

```

1. from flask import Blueprint, jsonify, request
2. from flask_login import login_required, current_user
3. from app.models import Movie, Favorite
4. from app import db
5.
6. bp = Blueprint('favorites', __name__)
7.
8. @bp.route('/', methods=['GET'])
9. @login_required
10. def get_favorites():
11.     favorites =
Favorite.query.filter_by(user_id=current_user.id).all()

```

```

12.     movies = [fav.movie for fav in favorites]
13.     return jsonify([
14.         'id': movie.id,
15.         'title': movie.title,
16.         'overview': movie.overview,
17.         'poster_path': movie.poster_url,
18.         'genres': [genre.name for genre in
movie.genres],
19.         'actors': [actor.name for actor in
movie.actors],
20.         'countries': [country.name for country in
movie.countries]
21.     } for movie in movies]), 200
22.
23. @bp.route('/<int:movie_id>', methods=['POST'])
24. @login_required
25. def add_to_favorites(movie_id):
26.     movie = Movie.query.get_or_404(movie_id)
27.
28.     if Favorite.query.filter_by(user_id=current_user.id,
movie_id=movie_id).first():
29.         return jsonify({'error': 'Movie already in
favorites'}), 400
30.
31.     favorite = Favorite(user_id=current_user.id,
movie_id=movie_id)
32.     db.session.add(favorite)
33.     db.session.commit()
34.
35.     return jsonify({'message': 'Movie added to
favorites'}), 201
36.
37. @bp.route('/<int:movie_id>', methods=['DELETE'])
38. @login_required
39. def remove_from_favorites(movie_id):
40.     favorite =
Favorite.query.filter_by(user_id=current_user.id,
movie_id=movie_id).first_or_404()
41.     db.session.delete(favorite)
42.     db.session.commit()
43.
44.     return jsonify({'message': 'Movie removed from
favorites'}), 200

```

Содержимое файла app/routes/movie_picker.py:

```

1. from flask import Blueprint, jsonify, request
2. from app.services.movie_picker import MoviePicker
3.
4. bp = Blueprint('movie_picker', __name__)
5.
6. @bp.route('/pick', methods=['GET'])
7. def pick_movies():

```

```

8.         try:
9.             genre = request.args.get('genre')
10.            year_start = request.args.get('year_start',
type=int)
11.            year_end = request.args.get('year_end', type=int)
12.            country = request.args.get('country')
13.            limit = request.args.get('limit', 20, type=int)
14.
15.            picker = MoviePicker()
16.            movies = (picker
17.                      .by_genre(genre)
18.                      .by_years(year_start, year_end)
19.                      .by_country(country)
20.                      .sort_by_rating()
21.                      .get_movies(limit))
22.
23.            return jsonify({
24.                'movies': [MoviePicker.format_movie(movie)
for movie in movies]
25.                }, 200)
26.        except Exception as e:
27.            return jsonify({'error': str(e)}), 500
28.
29. @bp.route('/genres', methods=['GET'])
30. def get_genres():
31.     try:
32.         genres = MoviePicker.get_available_genres()
33.         return jsonify({'genres': genres}), 200
34.     except Exception as e:
35.         return jsonify({'error': str(e)}), 500
36.
37. @bp.route('/countries', methods=['GET'])
38. def get_countries():
39.     try:
40.         countries =
MoviePicker.get_available_countries()
41.         return jsonify({'countries': countries}), 200
42.     except Exception as e:
43.         return jsonify({'error': str(e)}), 500

```

Содержимое файла app/routes/movies.py:

```

1. from flask import Blueprint, jsonify, request
2. from app.models import Movie, Genre, Actor, Country
3. from app import db
4.
5. bp = Blueprint('movies', __name__)
6.
7. @bp.route('/', methods=['GET'])
8. def get_movies():
9.     movies = Movie.query.all()
10.    return jsonify([
11.        {'id': movie.id,

```

```

12.         'title': movie.title,
13.         'overview': movie.overview,
14.         'poster_path': movie.poster_url,
15.         'genres': [genre.name for genre in movie.genres],
16.         'actors': [actor.name for actor in movie.actors],
17.         'countries': [country.name for country in
movie.countries]
18.     } for movie in movies]), 200
19.
20. @bp.route('/<int:movie_id>', methods=['GET'])
21. def get_movie(movie_id):
22.     movie = Movie.query.get_or_404(movie_id)
23.     return jsonify({
24.         'id': movie.id,
25.         'title': movie.title,
26.         'overview': movie.overview,
27.         'poster_path': movie.poster_url,
28.         'genres': [genre.name for genre in
movie.genres],
29.         'actors': [actor.name for actor in
movie.actors],
30.         'countries': [country.name for country in
movie.countries]
31.     }), 200
32.
33. @bp.route('/', methods=['POST'])
34. def create_movie():
35.     data = request.get_json()
36.
37.     genres = []
38.     for genre_name in data.get('genres', []):
39.         genre =
Genre.query.filter_by(name=genre_name).first()
40.         if not genre:
41.             genre = Genre(name=genre_name)
42.             db.session.add(genre)
43.             genres.append(genre)
44.
45.     actors = []
46.     for actor_name in data.get('actors', []):
47.         actor =
Actor.query.filter_by(name=actor_name).first()
48.         if not actor:
49.             actor = Actor(name=actor_name)
50.             db.session.add(actor)
51.             actors.append(actor)
52.
53.     countries = []
54.     for country_name in data.get('countries', []):
55.         country =
Country.query.filter_by(name=country_name).first()
56.         if not country:
57.             country = Country(name=country_name)

```

```

58.         db.session.add(country)
59.         countries.append(country)
60.
61.     movie = Movie(
62.         title=data['title'],
63.         overview=data.get('overview', ''),
64.         poster_path=data.get('poster_path', '')
65.     )
66.     movie.genres = genres
67.     movie.actors = actors
68.     movie.countries = countries
69.
70.     db.session.add(movie)
71.     db.session.commit()
72.
73.     return jsonify({
74.         'id': movie.id,
75.         'title': movie.title,
76.         'overview': movie.overview,
77.         'poster_path': movie.poster_url,
78.         'genres': [genre.name for genre in
movie.genres],
79.         'actors': [actor.name for actor in
movie.actors],
80.         'countries': [country.name for country in
movie.countries]
81.     }), 201

```

Содержимое файла app/routes/recommendations.py:

```

1. from flask import Blueprint, jsonify, request,
current_app
2. from app import db
3. from app.models import Movie, RecommendationLog
4. from recommender.hybrid_recommender import
HybridRecommender
5. from recommender.movie_recommender import
MovieRecommender
6. from flask_login import login_required, current_user
7. import os
8.
9. bp = Blueprint('recommendations', __name__)
10. recommender = HybridRecommender()
11.
12. current_dir =
os.path.dirname(os.path.dirname(os.path.abspath(
__file__)))
13. data_path = os.path.join(current_dir, 'recommender',
'data')
14.
15. if not os.path.exists(data_path):
16.     os.makedirs(data_path)
17.

```

```

18. movie_recommender =
MovieRecommender(data_path=data_path)
19.
20. def init_app(app):
21.     recommender.init_app(app)
22.     try:
23.         movie_recommender.load_model()
24.         print(f"Модель успешно загружена из
{data_path}")
25.     except FileNotFoundError:
26.         print(f"Создание новой модели в {data_path}")
27.         movie_recommender.load_data(
28.             credits_path=os.path.join(data_path,
'credits.csv'),
29.             movies_path=os.path.join(data_path,
'movies.csv')
30.         )
31.         movie_recommender.create_similarity_matrix()
32.         movie_recommender.save_model()
33.         print("Модель успешно создана и сохранена")
34.
35. @bp.route('/build-matrices', methods=['POST'])
36. def build_matrices():
37.     try:
38.         recommender.build_content_similarity_matrix()
39.         return jsonify({'message': 'Matrices built
successfully'}), 200
40.     except Exception as e:
41.         return jsonify({'error': str(e)}), 500
42.
43. @bp.route('/recommendations', methods=['GET'])
44. @login_required
45. def get_recommendations():
46.     try:
47.         movie_id = request.args.get('movie_id',
type=int)
48.         n_recommendations = request.args.get('n', 10,
type=int)
49.
50.         recommendations =
recommender.get_hybrid_recommendations(
51.             user_id=current_user.id,
52.             movie_id=movie_id,
53.             n_recommendations=n_recommendations
54.         )
55.
56.         return jsonify({
57.             'recommendations': [{
58.                 'id': movie.id,
59.                 'title': movie.title,
60.                 'overview': movie.overview,
61.                 'poster_url': movie.poster_url,
62.                 'rating': movie.rating,

```

```

63.             'genres': [genre.name for genre in
movie.genres],
64.             'actors': [actor.name for actor in
movie.actors[:5]],
65.             'countries': [country.name for country
in movie.countries]
66.         } for movie in recommendations]
67.     }), 200
68.     except Exception as e:
69.         return jsonify({'error': str(e)}), 500
70.
71. @bp.route('/similar-movies/<int:movie_id>',
methods=['GET'])
72. def get_similar_movies(movie_id):
73.     try:
74.         movie = Movie.query.get(movie_id)
75.         if not movie:
76.             return jsonify({'error': 'Movie not
found'}), 404
77.
78.         n_recommendations = request.args.get('n',
default=10, type=int)
79.         similar_movie_ids =
recommender.get_content_based_recommendations(movie_id,
n_recommendations)
80.
81.         similar_movies =
Movie.query.filter(Movie.id.in_(similar_movie_ids)).all()
82.
83.         return jsonify({
84.             'movie': {
85.                 'id': movie.id,
86.                 'title': movie.title,
87.                 'overview': movie.overview,
88.                 'poster_url': movie.poster_url
89.             },
90.             'similar_movies': [{
91.                 'id': m.id,
92.                 'title': m.title,
93.                 'overview': m.overview,
94.                 'poster_url': m.poster_url
95.             } for m in similar_movies]
96.         }), 200
97.     except Exception as e:
98.         return jsonify({'error': str(e)}), 500
99.
100. @bp.route('/recommendations/by-movies',
methods=['POST'])
101. def get_recommendations_by_movies():
102.     try:
103.         data = request.get_json()
104.         if not data or 'movies' not in data:
105.             return jsonify({'error': 'No movies

```

```

provided'}), 400
    106.
    107.         movies = data['movies']
    108.         if not isinstance(movies, list) or not 1 <=
len(movies) <= 3:
    109.             return jsonify({'error': 'Must provide 1-3
movies'}), 400
    110.
    111.         n_recommendations = data.get('n', 10)
    112.
    113.         recommendations =
movie_recommender.get_recommendations(movies, n_recommendations)
    114.
    115.         recommended_movies = []
    116.         for title, original_title in recommendations:
    117.             movie = Movie.query.filter(
    118.                 (Movie.title == title) |
(Movie.original_title == original_title)
    119.                 ).first()
    120.
    121.             if movie:
    122.                 recommended_movies.append({
    123.                     'id': movie.id,
    124.                     'title': movie.title,
    125.                     'original_title':
movie.original_title,
    126.                     'overview': movie.overview,
    127.                     'poster_url': movie.poster_url,
    128.                     'rating': movie.rating,
    129.                     'genres': [genre.name for genre in
movie.genres],
    130.                     'actors': [actor.name for actor in
movie.actors[:5]],
    131.                     'countries': [country.name for
country in movie.countries]
    132.                 })
    133.
    134.         return jsonify({
    135.             'recommendations': recommended_movies
    136.         }), 200
    137.     except Exception as e:
    138.         return jsonify({'error': str(e)}), 500
    139.
    140. @bp.route('/by-movies', methods=['GET'])
    141. def get_recommendations_by_movies_get():
    142.     try:
    143.         movies_str = request.args.get('movies', '')
    144.         if not movies_str:
    145.             return jsonify({'error': 'No movies
provided'}), 400
    146.
    147.         try:
    148.             movie_ids = [int(m.strip()) for m in

```

```

movies_str.split(',')])
149.         except ValueError:
150.             return jsonify({'error': 'Invalid movie IDs
provided'}), 400
151.
152.         if not 1 <= len(movie_ids) <= 3:
153.             return jsonify({'error': 'Must provide 1-3
movies'}), 400
154.
155.         n_recommendations = request.args.get('n', 10,
type=int)
156.
157.         recommended_ids =
movie_recommender.get_recommendations(movie_ids,
n_recommendations)
158.
159.         recommended_movies = []
160.         for movie_id in recommended_ids:
161.             movie = Movie.query.get(movie_id)
162.             if movie:
163.                 recommended_movies.append({
164.                     'id': movie.id,
165.                     'title': movie.title,
166.                     'original_title':
movie.original_title,
167.                     'overview': movie.overview,
168.                     'poster_url': movie.poster_url,
169.                     'rating': movie.rating,
170.                     'genres': [genre.name for genre in
movie.genres],
171.                     'actors': [actor.name for actor in
movie.actors[:5]], # Топ-5 актеров
172.                     'countries': [country.name for
country in movie.countries]
173.                 })
174.
175.         return jsonify({
176.             'recommendations': recommended_movies
177.         }), 200
178.     except Exception as e:
179.         return jsonify({'error': str(e)}), 500

```

Содержимое файла app/services/data_analyzer.py:

```

1. from app import db
2. from app.models import User, Movie, Favorite,
SearchHistory, RecommendationLog, Genre, Actor, Country
3. from sqlalchemy import func, desc
4. from collections import Counter
5. import numpy as np
6.
7. class DataAnalyzer:
8.     def __init__(self):

```

```

9.         self.db = db
10.
11.     def analyze_user_preferences(self, user_id):
12.         try:
13.             favorites =
Favorite.query.filter_by(user_id=user_id).all()
14.             favorite_movies = [fav.movie for fav in
favorites]
15.
16.             genres = []
17.             for movie in favorite_movies:
18.                 genres.extend([genre.name for genre in
movie.genres])
19.             genre_preferences =
Counter(genres).most_common()
20.
21.             actors = []
22.             for movie in favorite_movies:
23.                 actors.extend([actor.name for actor in
movie.actors])
24.             actor_preferences =
Counter(actors).most_common(10)
25.
26.             countries = []
27.             for movie in favorite_movies:
28.                 countries.extend([country.name for
country in movie.countries])
29.             country_preferences =
Counter(countries).most_common()
30.
31.             years = [movie.release_date.year for movie
in favorite_movies if movie.release_date]
32.             year_preferences =
Counter(years).most_common()
33.
34.             return {
35.                 'genre_preferences': genre_preferences,
36.                 'actor_preferences': actor_preferences,
37.                 'country_preferences':
country_preferences,
38.                 'year_preferences': year_preferences
39.             }
40.         except Exception as e:
41.             raise Exception(f"Error analyzing user
preferences: {str(e)}")
42.
43.     def analyze_movie_patterns(self):
44.         try:
45.             genre_counts = db.session.query(
46.                 Genre.name, func.count(Movie.id)
47.             ).join(Movie.genres).group_by(Genre.name).all()
48.

```

```

49.             actor_counts = db.session.query(
50.                 Actor.name, func.count(Movie.id)
51.             )
52.             .join(Movie.actors).group_by(Actor.name).all()
53.             country_counts = db.session.query(
54.                 Country.name, func.count(Movie.id)
55.             )
56.             .join(Movie.countries).group_by(Country.name).all()
57.             rating_stats = db.session.query(
58.                 func.avg(Movie.rating),
59.                 func.min(Movie.rating),
60.                 func.max(Movie.rating)
61.             ).first()
62.
63.             return {
64.                 'genre_patterns': genre_counts,
65.                 'actor_patterns': actor_counts,
66.                 'country_patterns': country_counts,
67.                 'rating_statistics': {
68.                     'average': rating_stats[0],
69.                     'minimum': rating_stats[1],
70.                     'maximum': rating_stats[2]
71.                 }
72.             }
73.         except Exception as e:
74.             raise Exception(f"Error analyzing movie
patterns: {str(e)}")
75.
76.     def generate_insights(self, user_id=None):
77.         try:
78.             insights = {
79.                 'global_patterns':
self.analyze_movie_patterns()
80.             }
81.
82.             if user_id:
83.                 insights['user_preferences'] =
self.analyze_user_preferences(user_id)
84.
85.             if user_id:
86.                 user_prefs =
insights['user_preferences']
87.                 global_patterns =
insights['global_patterns']
88.
89.                 recommended_genres = [
90.                     genre for genre, _ in
user_prefs['genre_preferences']
91.                     if genre in [g[0] for g in
global_patterns['genre_patterns']]
92.                 ]

```

```

93.
94.             recommended_actors = [
95.                 actor for actor, _ in
user_prefs['actor_preferences']
96.                 if actor in [a[0] for a in
global_patterns['actor_patterns']]
97.             ]
98.
99.             insights['recommendations'] = {
100.                 'genres': recommended_genres,
101.                 'actors': recommended_actors
102.             }
103.
104.             return insights
105.         except Exception as e:
106.             raise Exception(f"Error generating
insights: {str(e)}")
107.
108.         def get_trending_movies(self, limit=10):
109.             try:
110.                 trending = db.session.query(
111.                     Movie,
112.
func.count(Favorite.id).label('favorite_count')
113.
).outerjoin(Favorite).group_by(Movie.id).order_by(
114.                     desc('favorite_count')
115.                     ).limit(limit).all()
116.
117.                 return [movie for movie, _ in trending]
118.             except Exception as e:
119.                 raise Exception(f"Error getting trending
movies: {str(e)}")

```

Содержимое файла app/services/data_preprocessor.py:

```

1. from app import db
2. from app.models import Movie, Genre, Actor, Director,
Country
3. import pandas as pd
4. import numpy as np
5. from datetime import datetime
6. import re
7. import logging
8.
9. class DataPreprocessor:
10.     def __init__(self):
11.         self.logger = logging.getLogger(__name__)
12.
13.     def load_external_data(self, movies_path,
credits_path):
14.         try:
15.             movies_df = pd.read_csv(movies_path)

```

```

16.         credits_df = pd.read_csv(credits_path)
17.
18.         return {
19.             'movies': movies_df,
20.             'credits': credits_df
21.         }
22.     except Exception as e:
23.         self.logger.error(f"Error loading external
data: {str(e)}")
24.         raise
25.
26.     def preprocess_data(self, data):
27.         try:
28.             movies_df = data['movies']
29.             credits_df = data['credits']
30.
31.             movies_df = self._clean_movie_data(movies_df)
32.             credits_df =
self._clean_credits_data(credits_df)
33.
34.             merged_df = self._merge_data(movies_df,
credits_df)
35.
36.             processed_data =
self._transform_data(merged_df)
37.
38.             return processed_data
39.         except Exception as e:
40.             self.logger.error(f"Error preprocessing data:
{str(e)}")
41.             raise
42.
43.     def validate_data(self, data):
44.         try:
45.             validation_results = {
46.                 'movies':
self._validate_movies(data['movies']),
47.                 'credits':
self._validate_credits(data['credits'])
48.             }
49.
50.             if not all(validation_results.values()):
51.                 self.logger.warning("Data validation
failed")
52.                 return False
53.
54.             return True
55.         except Exception as e:
56.             self.logger.error(f"Error validating data:
{str(e)}")
57.             raise
58.
59.     def _clean_movie_data(self, movies_df):

```

```

60.         try:
61.             movies_df = movies_df.drop_duplicates()
62.
63.             movies_df['overview'] =
movies_df['overview'].fillna('')
64.             movies_df['poster_path'] =
movies_df['poster_path'].fillna('')
65.
66.             movies_df['release_date'] =
pd.to_datetime(movies_df['release_date'], errors='coerce')
67.
68.             movies_df['genres'] =
movies_df['genres'].apply(self._parse_json_list)
69.
70.             return movies_df
71.         except Exception as e:
72.             self.logger.error(f"Error cleaning movie
data: {str(e)}")
73.             raise
74.
75.     def _clean_credits_data(self, credits_df):
76.         try:
77.             credits_df = credits_df.drop_duplicates()
78.
79.             credits_df['cast'] =
credits_df['cast'].fillna('[]')
80.             credits_df['crew'] =
credits_df['crew'].fillna('[]')
81.
82.             credits_df['cast'] =
credits_df['cast'].apply(self._parse_json_list)
83.             credits_df['crew'] =
credits_df['crew'].apply(self._parse_json_list)
84.
85.             return credits_df
86.         except Exception as e:
87.             self.logger.error(f"Error cleaning credits
data: {str(e)}")
88.             raise
89.
90.     def _merge_data(self, movies_df, credits_df):
91.         try:
92.             merged_df = pd.merge(movies_df, credits_df,
on='id', how='left')
93.
94.             return merged_df
95.         except Exception as e:
96.             self.logger.error(f"Error merging data:
{str(e)}")
97.             raise
98.
99.     def _transform_data(self, merged_df):
100.        try:

```

```

101.            merged_df['actors'] =
merged_df['cast'].apply(
102.                lambda x: [actor['name'] for actor in
x[:5]] if isinstance(x, list) else []
103.            )
104.
105.            merged_df['directors'] =
merged_df['crew'].apply(
106.                lambda x: [crew['name'] for crew in x if
crew.get('job') == 'Director'] if isinstance(x, list) else []
107.            )
108.
109.            merged_df['title'] =
merged_df['title'].apply(self._clean_title)
110.
111.            return merged_df
112.        except Exception as e:
113.            self.logger.error(f"Error transforming data:
{str(e)}")
114.            raise
115.
116.        def _validate_movies(self, movies_df):
117.            try:
118.                required_fields = ['id', 'title',
'release_date']
119.                if not all(field in movies_df.columns for
field in required_fields):
120.                    return False
121.
122.                if not
pd.api.types.is_numeric_dtype(movies_df['id']):
123.                    return False
124.
125.                if not
pd.api.types.is_datetime64_any_dtype(movies_df['release_date']):
126.                    return False
127.
128.                return True
129.            except Exception as e:
130.                self.logger.error(f"Error validating movies:
{str(e)}")
131.                return False
132.
133.        def _validate_credits(self, credits_df):
134.            try:
135.                required_fields = ['id', 'cast', 'crew']
136.                if not all(field in credits_df.columns for
field in required_fields):
137.                    return False
138.
139.                if not
pd.api.types.is_numeric_dtype(credits_df['id']):
140.                    return False

```

```

141.
142.         return True
143.     except Exception as e:
144.         self.logger.error(f"Error validating
credits: {str(e)}")
145.         return False
146.
147.     def _parse_json_list(self, json_str):
148.         try:
149.             if pd.isna(json_str) or json_str == '[]':
150.                 return []
151.             return eval(json_str)
152.         except:
153.             return []
154.
155.     def _clean_title(self, title):
156.         try:
157.             title = re.sub(r'^\w\s', '', title)
158.             title = ' '.join(title.split())
159.             return title
160.         except:
161.             return title

```

Содержимое файла app/services/model_trainer.py:

```

1. from app import db
2. from app.models import Movie, User, Favorite,
RecommendationLog
3. from recommender.hybrid_recommender import
HybridRecommender
4. from recommender.movie_recommender import
MovieRecommender
5. import numpy as np
6. from sklearn.metrics import mean_squared_error
7. import joblib
8. import os
9. from datetime import datetime
10.
11. class ModelTrainer:
12.     def __init__(self, model_path='recommender/data'):
13.         self.model_path = model_path
14.         self.hybrid_recommender = HybridRecommender()
15.         self.movie_recommender =
MovieRecommender(data_path=model_path)
16.         self.metrics = {}
17.
18.     def prepare_training_data(self):
19.         try:
20.             favorites = Favorite.query.all()
21.
22.             user_movie_matrix = {}
23.             for fav in favorites:
24.                 if fav.user_id not in user_movie_matrix:

```

```

25.                 user_movie_matrix[fav.user_id] = []
26.
user_movie_matrix[fav.user_id].append(fav.movie_id)
27.
28.                 return user_movie_matrix
29.             except Exception as e:
30.                 raise Exception(f"Error preparing training
data: {str(e)}")
31.
32.         def train_recommendation_model(self):
33.             try:
34.                 training_data = self.prepare_training_data()
35.
36.                 self.hybrid_recommender.train(training_data)
37.
38.
self.movie_recommender.create_similarity_matrix()
39.
40.                 self.save_models()
41.
42.                 self.evaluate_model()
43.
44.                 return True
45.             except Exception as e:
46.                 raise Exception(f"Error training
recommendation model: {str(e)}")
47.
48.         def update_model(self, new_data):
49.             try:
50.                 training_data = self.prepare_training_data()
51.                 training_data.update(new_data)
52.
53.                 self.train_recommendation_model()
54.
55.                 self.log_model_update()
56.
57.                 return True
58.             except Exception as e:
59.                 raise Exception(f"Error updating model:
{str(e)}")
60.
61.         def evaluate_model(self):
62.             try:
63.                 test_data = self.prepare_training_data()
64.
65.                 hybrid_metrics =
self.hybrid_recommender.evaluate(test_data)
66.
67.                 content_metrics =
self.movie_recommender.evaluate()
68.
69.                 self.metrics = {
70.                     'hybrid': hybrid_metrics,

```

```

71.             'content': content_metrics,
72.             'timestamp': datetime.now().isoformat()
73.         }
74.
75.         return self.metrics
76.     except Exception as e:
77.         raise Exception(f"Error evaluating model:
{str(e)}")
78.
79.     def save_models(self):
80.         try:
81.             if not os.path.exists(self.model_path):
82.                 os.makedirs(self.model_path)
83.
84.             self.hybrid_recommender.save_model(
85.                 os.path.join(self.model_path,
'hybrid_model.pkl')
86.             )
87.
88.             self.movie_recommender.save_model()
89.
90.             return True
91.         except Exception as e:
92.             raise Exception(f"Error saving models:
{str(e)}")
93.
94.     def load_models(self):
95.         try:
96.             self.hybrid_recommender.load_model(
97.                 os.path.join(self.model_path,
'hybrid_model.pkl')
98.             )
99.
100.            self.movie_recommender.load_model()
101.
102.            return True
103.        except Exception as e:
104.            raise Exception(f"Error loading models:
{str(e)}")
105.
106.    def log_model_update(self):
107.        try:
108.            log = RecommendationLog(
109.                model_type='hybrid',
110.                metrics=self.metrics,
111.                created_at=datetime.now()
112.            )
113.            db.session.add(log)
114.            db.session.commit()
115.
116.            return True
117.        except Exception as e:
118.            raise Exception(f"Error logging model

```

```

update: {str(e)}")
119.
120.     def get_model_metrics(self):
121.         return self.metrics

```

Содержимое файла app/services/movie_picker.py:

```

1. from app import db
2. from app.models import Movie, Genre, Country
3. from sqlalchemy import and_, desc
4. from datetime import datetime
5.
6. class MoviePicker:
7.     def __init__(self):
8.         self.query = Movie.query
9.
10.    def by_genre(self, genre_name):
11.        if genre_name:
12.            self.query =
self.query.join(Movie.genres).filter(Genre.name == genre_name)
13.        return self
14.
15.    def by_years(self, start_year, end_year):
16.        if start_year:
17.            self.query =
self.query.filter(Movie.release_date >= f"{start_year}-01-01")
18.        if end_year:
19.            self.query =
self.query.filter(Movie.release_date <= f"{end_year}-12-31")
20.        return self
21.
22.    def by_country(self, country_name):
23.        if country_name:
24.            self.query =
self.query.join(Movie.countries).filter(Country.name ==
country_name)
25.        return self
26.
27.    def sort_by_rating(self):
28.        self.query =
self.query.order_by(desc(Movie.rating))
29.        return self
30.
31.    def get_movies(self, limit=20):
32.        return self.query.limit(limit).all()
33.
34.    @staticmethod
35.    def get_available_genres():
36.        return [genre.name for genre in
Genre.query.all()]
37.
38.    @staticmethod
39.    def get_available_countries():

```

```

40.         return [country.name for country in
Country.query.all()]
41.
42.     @staticmethod
43.     def format_movie(movie):
44.         return {
45.             'id': movie.id,
46.             'title': movie.title,
47.             'original_title': movie.original_title,
48.             'overview': movie.overview,
49.             'poster_url': movie.poster_url,
50.             'release_date':
movie.release_date.strftime('%Y-%m-%d') if movie.release_date
else None,
51.             'rating': movie.rating,
52.             'genres': [genre.name for genre in
movie.genres],
53.             'actors': [actor.name for actor in
movie.actors[:5]], # Топ-5 актеров
54.             'countries': [country.name for country in
movie.countries]
55.         }

```

Содержимое файла recommender/download_data_fast.py:

```

1. import httpx
2. import pandas as pd
3. import time
4. import os
5. import asyncio
6. from tqdm import tqdm
7. import json
8. from datetime import datetime
9. import random
10. import signal
11. import sys
12.
13. class TMDDBDownloader:
14.     def __init__(self, api_key):
15.         self.api_key = api_key.strip('"\'')
16.         self.base_url = "https://api.themoviedb.org/3"
17.         self.language = "ru-RU"
18.         self.movies_data = []
19.         self.error_log = []
20.         self.max_retries = 5
21.         self.retry_delay = 5
22.         self.current_page = 1
23.
24.         self.script_dir =
os.path.dirname(os.path.abspath(__file__))
25.         self.progress_file =
os.path.join(self.script_dir, 'download_progress.json')
26.         self.error_log_file =

```

```

os.path.join(self.script_dir, 'error_log.json')
27.         self.data_dir = os.path.join(self.script_dir,
'data')
28.
29.         self.last_save_time = time.time()
30.         self.save_interval = 30
31.
32.         self.headers = {
33.             'User-Agent': 'Mozilla/5.0 (Windows NT 10.0;
Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/91.0.4472.124 Safari/537.36',
34.             'Accept': 'application/json'
35.         }
36.
37.         signal.signal(signal.SIGINT,
self.handle_interrupt)
38.         signal.signal(signal.SIGTERM,
self.handle_interrupt)
39.
40.         def handle_interrupt(self, signum, frame):
41.             print("\nПолучен сигнал прерывания. Сохраняем
прогресс...")
42.             self.save_progress()
43.             print("Прогресс сохранен. Завершение работы...")
44.             sys.exit(0)
45.
46.         def save_progress(self):
47.             try:
48.                 with open(self.progress_file, 'w',
encoding='utf-8') as f:
49.                     json.dump({
50.                         'last_page': self.current_page,
51.                         'total_movies':
len(self.movies_data),
52.                         'movies_data': self.movies_data,
53.                         'timestamp':
datetime.now().isoformat()
54.                     }, f, ensure_ascii=False, indent=2)
55.                     print(f"\nПрогресс сохранен:
{len(self.movies_data)} фильмов")
56.             except Exception as e:
57.                 print(f"Ошибка при сохранении прогресса:
{str(e)}")
58.
59.         def log_error(self, error_type, details):
60.             error_entry = {
61.                 'timestamp': datetime.now().isoformat(),
62.                 'type': error_type,
63.                 'details': details
64.             }
65.             self.error_log.append(error_entry)
66.
67.             try:

```

```

68.         with open(self.error_log_file, 'w',
encoding='utf-8') as f:
69.             json.dump(self.error_log, f,
ensure_ascii=False, indent=2)
70.         except Exception as e:
71.             print(f"Ошибка при сохранении лога ошибок:
{str(e)}")
72.
73.     async def get_popular_movies(self, client, page=1):
74.         try:
75.             response = await client.get(
76.                 f"{self.base_url}/movie/popular",
77.                 params={
78.                     'api_key': self.api_key,
79.                     'language': self.language,
80.                     'page': page
81.                 }
82.             )
83.             response.raise_for_status()
84.             return response.json()
85.         except Exception as e:
86.             self.log_error('POPULAR_MOVIES_ERROR', {
87.                 'page': page,
88.                 'error': str(e)
89.             })
90.             raise
91.
92.     async def get_movie_details(self, client, movie_id):
93.         try:
94.             response = await client.get(
95.                 f"{self.base_url}/movie/{movie_id}",
96.                 params={
97.                     'api_key': self.api_key,
98.                     'language': self.language,
99.                     'append_to_response':
'credits,release_dates,external_ids,keywords,videos,images'
100.                 }
101.             )
102.             response.raise_for_status()
103.             return response.json()
104.         except Exception as e:
105.             self.log_error('MOVIE_DETAILS_ERROR', {
106.                 'movie_id': movie_id,
107.                 'error': str(e)
108.             })
109.             raise
110.
111.     async def process_movie_batch(self, client,
movie_ids):
112.         tasks = []
113.         for movie_id in movie_ids:
114.             tasks.append(self.get_movie_details(client,
movie_id))

```

```

115.         return await asyncio.gather(*tasks,
return_exceptions=True)
116.
117.     async def download_data(self, num_movies=5000):
118.         print("Начинаем скачивание данных...")
119.
120.         os.makedirs(self.data_dir, exist_ok=True)
121.
122.         if os.path.exists(self.progress_file):
123.             try:
124.                 with open(self.progress_file, 'r',
encoding='utf-8') as f:
125.                     progress = json.load(f)
126.                     self.movies_data =
progress.get('movies_data', [])
127.                     self.current_page =
progress.get('last_page', 1)
128.                     total_movies = len(self.movies_data)
129.                     print(f"Продолжаем с сохраненного
прогресса: {total_movies} фильмов, страница
{self.current_page}")
130.             except Exception as e:
131.                 print(f"Ошибка при загрузке прогресса:
{str(e)}")
132.                 self.current_page = 1
133.                 total_movies = 0
134.             else:
135.                 self.current_page = 1
136.                 total_movies = 0
137.
138.         async with httpx.AsyncClient(
139.             timeout=60.0,
140.             headers=self.headers,
141.
limits=httpx.Limits(max_keepalive_connections=20,
max_connections=40)
142.         ) as client:
143.             with tqdm(total=num_movies,
initial=total_movies, desc="Скачивание фильмов") as pbar:
144.                 while total_movies < num_movies:
145.                     try:
146.                         print(f"\nПолучение страницы
{self.current_page}...")
147.                         popular_movies = await
self.get_popular_movies(client, self.current_page)
148.
149.                         if not popular_movies or not
popular_movies.get('results'):
150.                             print(f"Нет результатов на
странице {self.current_page}")
151.                             break
152.
153.                         movie_ids = [movie['id'] for

```

```

movie in popular_movies['results']]
154.
155.                batch_size = 10
156.                for i in range(0,
len(movie_ids), batch_size):
157.                    batch_ids = movie_ids[i:i +
batch_size]
158.                    results = await
self.process_movie_batch(client, batch_ids)
159.
160.                    for movie_details in
results:
161.                        if
isinstance(movie_details, Exception):
162.                            continue
163.
164.                        if not movie_details:
165.                            continue
166.
167.
self.movies_data.append(movie_details)
168.
169.                total_movies += 1
170.                pbar.update(1)
171.
172.                if total_movies >=
num_movies:
173.                    break
174.
175.                self.save_progress()
176.
177.                await asyncio.sleep(0.1)
178.
179.                if total_movies >=
num_movies:
180.                    break
181.
182.                self.current_page += 1
183.
184.                except Exception as e:
185.                    print(f"\nОшибка при получении
страницы {self.current_page}: {str(e)}")
186.                    self.log_error('PAGE_ERROR', {
187.                        'page': self.current_page,
188.                        'error': str(e)
189.                    })
190.                    await
asyncio.sleep(self.retry_delay)
191.                    continue
192.
193.                if not self.movies_data:
194.                    print("Не удалось скачать данные о
фильмах")

```

```

195.                return
196.
197.                print("\nСохранение данных...")
198.                json_file = os.path.join(self.data_dir,
'movies_data.json')
199.                with open(json_file, 'w', encoding='utf-8')
as f:
200.                    json.dump(self.movies_data, f,
ensure_ascii=False, indent=2)
201.
202.                print("Данные успешно сохранены!")
203.                print(f"Скачано {len(self.movies_data)}
фильмов")
204.
205.                if os.path.exists(self.progress_file):
206.                    os.remove(self.progress_file)
207.
208. async def main():
209.     print("Для работы скрипта необходим API ключ от
TMDB.")
210.     print("Получить ключ можно на сайте:
https://www.themoviedb.org/settings/api")
211.     print("После регистрации перейдите в настройки
профиля -> API и запросите ключ")
212.     print("\nВведите ваш API ключ от TMDB:")
213.     api_key = input().strip()
214.
215.     if not api_key:
216.         print("API ключ не может быть пустым")
217.         return
218.
219.     downloader = TMDBDownloader(api_key)
220.     await downloader.download_data()
221.
222. if __name__ == "__main__":
223.     asyncio.run(main())

```

Содержимое файла recommender/movie_recommender.py:

```

1. import pandas as pd
2. import numpy as np
3. from sklearn.feature_extraction.text import
CountVectorizer
4. from sklearn.metrics.pairwise import cosine_similarity
5. import nltk
6. from nltk.corpus import stopwords
7. from nltk.stem.porter import PorterStemmer
8. import string
9. import pickle
10. import os
11.
12. class MovieRecommender:
13.     def __init__(self, data_path='data'):

```

```

14.         self.data_path = data_path
15.         self.movies = None
16.         self.movies_data = None
17.         self.similarity_matrix = None
18.         self.ps = PorterStemmer()
19.         nltk.download('stopwords')
20.
21.     def _preprocess_text(self, text):
22.         if pd.isna(text):
23.             return ''
24.
25.         words = text.split()
26.         stemmed = [self.ps.stem(word) for word in words]
27.         stop_words = set(stopwords.words('english'))
28.         filtered = [word.lower() for word in stemmed if
word.lower() not in stop_words]
29.
30.         result = []
31.         for word in filtered:
32.             word = word.translate(str.maketrans('', '',
string.punctuation))
33.             if len(word) > 2:
34.                 result.append(word)
35.
36.         return ' '.join(result)
37.
38.     def load_data(self, credits_path, movies_path):
39.         self.movies_data = pd.read_csv(movies_path)
40.
41.         self.movies_data['tags'] =
self.movies_data.apply(
42.             lambda row: ' '.join(filter(None, [
43.                 str(row['overview']) if
pd.notna(row['overview']) else '',
44.                 str(row['genres']) if
pd.notna(row['genres']) else '',
45.                 str(row['top_cast']) if
pd.notna(row['top_cast']) else '',
46.                 str(row['directors']) if
pd.notna(row['directors']) else '',
47.                 str(row['production_countries']) if
pd.notna(row['production_countries']) else '',
48.                 str(row['keywords']) if
pd.notna(row['keywords']) else ''
49.             ])),
50.             axis=1
51.         )
52.
53.         self.movies_data['tags'] =
self.movies_data['tags'].apply(self._preprocess_text)
54.         self.movies = self.movies_data[['id', 'title',
'original_title', 'tags']]
55.

```

```

56.         def create_similarity_matrix(self):
57.             cv = CountVectorizer(max_features=5000,
stop_words='english')
58.             vectors =
cv.fit_transform(self.movies['tags']).toarray()
59.             self.similarity_matrix =
cosine_similarity(vectors)
60.
61.         def save_model(self,
filename='movie_recommender.pkl'):
62.             model_data = {
63.                 'movies': self.movies,
64.                 'similarity_matrix': self.similarity_matrix
65.             }
66.             with open(os.path.join(self.data_path,
filename), 'wb') as f:
67.                 pickle.dump(model_data, f)
68.
69.         def load_model(self,
filename='movie_recommender.pkl'):
70.             with open(os.path.join(self.data_path,
filename), 'rb') as f:
71.                 model_data = pickle.load(f)
72.                 self.movies = model_data['movies']
73.                 self.similarity_matrix =
model_data['similarity_matrix']
74.
75.         def get_recommendations(self, movie_ids,
n_recommendations=10):
76.             if isinstance(movie_ids, int):
77.                 movie_ids = [movie_ids]
78.
79.             if not 1 <= len(movie_ids) <= 3:
80.                 raise ValueError("Количество фильмов должно
быть от 1 до 3")
81.
82.             movie_indices = []
83.             for movie_id in movie_ids:
84.                 try:
85.                     idx = self.movies[self.movies['id'] ==
movie_id].index[0]
86.                     movie_indices.append(idx)
87.                 except IndexError:
88.                     print(f"Фильм с ID {movie_id} не найден
в базе данных")
89.                     continue
90.
91.             if not movie_indices:
92.                 return []
93.
94.             similarity_scores = np.zeros(len(self.movies))
95.             for idx in movie_indices:
96.                 similarity_scores +=

```

```

self.similarity_matrix[idx]
97.
98.         movie_indices =
list(enumerate(similarity_scores))
99.         movie_indices = sorted(movie_indices, key=lambda
x: x[1], reverse=True)
100.        movie_indices = [i[0] for i in movie_indices if
i[0] not in movie_indices]
101.
102.        recommendations =
self.movies.iloc[movie_indices[:n_recommendations]]
103.        return recommendations['id'].tolist()

```

Содержимое файла recommender/hybrid_recommender.py:

```

1. import numpy as np
2. from sklearn.metrics.pairwise import cosine_similarity
3. from sklearn.feature_extraction.text import
TfidfVectorizer
4. from app.models import Movie, Genre, Actor, Country
5. from app import db
6.
7. class HybridRecommender:
8.     def __init__(self, app=None):
9.         self.app = app
10.        self.tfidf =
TfidfVectorizer(stop_words='english')
11.        self.content_similarity_matrix = None
12.        self.movie_id_to_index = {}
13.        self.index_to_movie_id = {}
14.
15.    def init_app(self, app):
16.        self.app = app
17.
18.    def _get_movie_features(self, movie):
19.        features = []
20.
21.        genres = [genre.name for genre in movie.genres]
22.        features.extend(genres)
23.
24.        actors = [actor.name for actor in movie.actors]
25.        features.extend(actors)
26.
27.        countries = [country.name for country in
movie.countries]
28.        features.extend(countries)
29.
30.        if movie.overview:
31.            features.append(movie.overview)
32.
33.        return ' '.join(features)
34.
35.    def build_content_similarity_matrix(self):

```

```

36.         with self.app.app_context():
37.             movies = Movie.query.all()
38.             print(f"\nBuilding content similarity matrix
for {len(movies)} movies")
39.
40.             movie_features =
[self._get_movie_features(movie) for movie in movies]
41.
42.             print("\nMovie features extracted")
43.
44.             tfidf_matrix =
self.tfidf.fit_transform(movie_features)
45.             print(f"TF-IDF matrix shape:
{tfidf_matrix.shape}")
46.
47.             self.content_similarity_matrix =
cosine_similarity(tfidf_matrix)
48.             print(f"Content similarity matrix shape:
{self.content_similarity_matrix.shape}")
49.
50.             self.movie_id_to_index = {movie.id: idx for
idx, movie in enumerate(movies)}
51.             self.index_to_movie_id = {idx: movie.id for
idx, movie in enumerate(movies)}
52.             print("Movie indices mapping created")
53.
54.             def get_content_based_recommendations(self, movie_id,
n_recommendations=10):
55.                 print(f"\nGetting content-based recommendations
for movie_id={movie_id}")
56.                 print(f"Available movie indices:
{self.movie_id_to_index}")
57.
58.                 if movie_id not in self.movie_id_to_index:
59.                     print(f"Movie ID {movie_id} not found in
index mapping")
60.                     return []
61.
62.                 movie_idx = self.movie_id_to_index[movie_id]
63.                 print(f"Movie index in similarity matrix:
{movie_idx}")
64.
65.                 similarity_scores =
self.content_similarity_matrix[movie_idx]
66.                 print(f"Similarity scores shape:
{similarity_scores.shape}")
67.                 print(f"Top 5 similarity scores:
{sorted(similarity_scores, reverse=True)[:5]}")
68.
69.                 similar_indices =
np.argsort(similarity_scores)[::-1][1:n_recommendations+1]
70.                 print(f"Similar movie indices:
{similar_indices}")

```

```
71.
72.         similar_movie_ids = [self.index_to_movie_id[idx]
for idx in similar_indices]
73.         print(f"Similar movie IDs: {similar_movie_ids}")
74.
75.         return similar_movie_ids
```

ПРИЛОЖЕНИЕ Б
(обязательное)

Спецификация

ПРИЛОЖЕНИЕ В
(обязательное)

Ведомость документов